Práticas de Engenharia de Software Contínua × fatores de QVT

Achados da etapa 2: como CI, CD, DevOps, *on-call* e práticas afins aumentam ou reduzem os fatores de qualidade de vida no trabalho de quem produz software, segundo o texto completo dos artigos

32
candidatos da busca 1 (aprovados e "talvez" no abstract)

26
lidos no texto completo

22
com elo prática → fator

65
elos prática → fator, todos com citação conferida

> **De onde vêm os elos.** Cada elo foi extraído do **texto completo** do artigo com as regras da skill *analise-esc-fatores*: só conta o que sai dos dados do próprio estudo; recomendação e trabalho citado não contam. Toda citação foi conferida por script contra o texto do PDF. **O que falta:** 6 candidatos ficaram sem PDF (seção 6), a busca 2 (790 registros) ainda não foi triada e os artigos ainda não estão no app.

Pergunta. Como a adoção de práticas de Engenharia de Software Contínua (ESC) aumenta ou reduz cada fator de QVT levantado na etapa 1? O que se registra é só o **primeiro elo**: o que a prática faz com a presença do fator. Se isso faz bem ou mal vem da polaridade do fator na etapa 1, mostrada entre parênteses ao lado do nome: *fator −* prejudica o bem-estar e *fator +* favorece.

Como ler os efeitos. **↑ aumenta** e **↓ reduz** descrevem a presença do fator, não o bem-estar. Reduzir *Interrupções no trabalho* (fator −) é bom; aumentar *Relações no trabalho* (fator +) também. **Evidência:** D = empírica direta (os dados ligam prática e fator: fala de participante, antes/depois, modelo); I = empírica indireta (co-ocorrência, interpretação dos autores); F = inferida (conclusão da discussão, sem dado que ligue os dois).

Base. Busca no Scopus (CAPES) em 2026-11-05; triagem dos abstracts e leitura do texto completo em 2026-11-06; catálogo de 64 fatores congelado em 24/09/2026. Protocolo em docs/busca-esc-qvt.md; dados em docs/esc/.

1. As práticas de Engenharia de Software Contínua

Engenharia de Software Contínua (ESC) é o conjunto de práticas que encurtam o caminho entre escrever software e colocá-lo nas mãos dos clientes, com automação e ciclos curtos — e que mudam também quem opera o sistema, como as falhas são tratadas e como as decisões são tomadas. DevOps é o nome mais conhecido desse movimento. Esta seção descreve cada prática do catálogo da pesquisa em linguagem não técnica: o que é, como funciona no dia a dia e por que importa para quem trabalha. São descrições gerais; o que os estudos encontraram está nas seções 4 e 5.

Núcleo técnico

Como o código sai da mesa do desenvolvedor e chega ao cliente: integração, testes, empacotamento e implantação feitos de forma automática e frequente.

Integração contínua — continuous integration, CI
O que é. Cada desenvolvedor junta o seu trabalho ao código comum do time várias vezes por dia, e um sistema automático verifica na hora se a junção quebrou alguma coisa. É o oposto de cada pessoa trabalhar semanas isolada e só juntar tudo no fim, quando os conflitos já se acumularam.
Na prática. A cada alteração enviada, um servidor compila o sistema e roda os testes automáticos em minutos. Se algo falha, o build fica "vermelho" e o time é avisado; a regra comum é consertar antes de seguir adiante.
Por que importa para as pessoas. Os problemas aparecem cedo e pequenos, em vez de tarde e grandes, o que reduz retrabalho e as "maratonas de integração" antes de cada entrega. Em troca, cria-se o compromisso de manter o build verde, e um build quebrado por outra pessoa pode interromper o time inteiro.
8 elos nesta pesquisa — ver achados

Pipeline de CI/CD — CI/CD pipeline, build pipeline, workflow (GitHub Actions, Jenkins, GitLab CI)
O que é. É a "linha de montagem" automatizada que leva uma alteração de código desde o envio pelo desenvolvedor até o ambiente de produção: compilar, testar, empacotar, aprovar e implantar, em etapas encadeadas. A prática aqui é o pipeline como algo que o time constrói, mantém e de que depende no dia a dia.
Na prática. O time descreve as etapas em arquivos de configuração; ferramentas como Jenkins ou GitHub Actions executam tudo sozinhas a cada mudança. Quando o pipeline é lento, instável ou difere do ambiente do desenvolvedor, alguém precisa parar para investigar e consertar.
Por que importa para as pessoas. Bem montado, tira do time tarefas manuais repetitivas e arriscadas e dá confiança para entregar. Mal mantido, vira uma fonte própria de trabalho: esperas, falhas sem explicação e manutenção que ninguém planejou.
12 elos nesta pesquisa — ver achados

Entrega contínua — continuous delivery, CD
O que é. O software é mantido o tempo todo em condição de ser lançado: qualquer versão que passou pelo pipeline pode ir para os clientes com um clique. A decisão de lançar passa a ser de negócio, não uma operação técnica de risco.
Na prática. Testes, empacotamento e preparação de ambientes são automatizados, e ambientes de teste se parecem com o de produção. O lançamento em si pode ainda exigir aprovação humana ou acontecer em janelas combinadas com o cliente.
Por que importa para as pessoas. Lançar deixa de ser um evento tenso, feito à noite ou no fim de semana com o time inteiro de prontidão. O custo está na transição: aprender ferramentas novas e reorganizar o trabalho exige esforço antes de os ganhos aparecerem.
9 elos nesta pesquisa — ver achados

Implantação contínua — continuous deployment
O que é. Um passo além da entrega contínua: toda alteração que passa nos testes automáticos vai para produção sozinha, sem aprovação manual. Empresas que a adotam lançam dezenas ou centenas de vezes por dia.
Na prática. O pipeline implanta, monitora os indicadores e, se algo piora, desfaz a mudança automaticamente. Para funcionar, depende de testes muito confiáveis e de mecanismos para ligar e desligar funcionalidades sem nova implantação.
Por que importa para as pessoas. O desenvolvedor vê o resultado do seu trabalho em produção no mesmo dia e não precisa acumular mudanças para um grande lançamento. Em compensação, passa a responder mais diretamente pelo que vai ao ar, e a automação pode exigir atenção constante.
2 elos nesta pesquisa — ver achados

Releases frequentes — **rapid releases, release cycle, release train**
O que é. Encurtar o intervalo entre versões do produto: em vez de uma grande versão por ano ou semestre, uma versão a cada poucas semanas ou dias, em calendário fixo. O navegador Firefox, por exemplo, passou de versões anuais para versões a cada seis semanas.
Na prática. Há datas de corte regulares (um "trem" que parte no horário); o que não estiver pronto vai no próximo. Isso troca o grande prazo final por muitos prazos pequenos e previsíveis.
Por que importa para as pessoas. Perder um corte custa pouco, porque o próximo está perto, o que alivia a corrida de última hora. Mas o ritmo constante pode virar pressão contínua se o calendário for tratado como cobrança permanente.
5 elos nesta pesquisa — ver achados

Automação de testes — **test automation, automated test suite, continuous testing, flaky tests**
O que é. Os testes que verificam se o software funciona são escritos como programas e executados automaticamente, a cada mudança, em vez de alguém testar à mão. É o que torna possível integrar e entregar com frequência sem perder qualidade.
Na prática. Milhares de testes rodam em minutos dentro do pipeline. Um problema conhecido são os testes instáveis ("flaky"): falham às vezes sem que haja defeito real, obrigando a repetir execuções e investigar alarmes falsos.
Por que importa para as pessoas. Dá segurança para alterar o código e libera as pessoas de testes manuais repetitivos. A suíte, porém, também precisa de manutenção, e testes pouco confiáveis geram frustração e desconfiança no próprio processo.
4 elos nesta pesquisa — ver achados

Desenvolvimento baseado em tronco — **trunk-based development, short-lived branches**
O que é. Todos trabalham sobre uma única linha principal do código (o "tronco"), com desvios que duram horas ou no máximo poucos dias. Evita versões paralelas do produto que divergem por semanas e depois são difíceis de reconciliar.
Na prática. Mudanças grandes são quebradas em pedaços pequenos, integrados todos os dias; o que ainda não está pronto fica escondido por chaves de funcionalidade.
Por que importa para as pessoas. Diminui os conflitos penosos de junção de código e a sensação de trabalho perdido. Exige disciplina para trabalhar em passos pequenos e expõe mais cedo o trabalho de cada um.
sem elo nos artigos lidos até agora

Feature toggles — **feature flags, feature toggles, dark launches**
O que é. "Interruptores" no próprio software que ligam ou desligam uma funcionalidade sem precisar de nova versão. Permitem separar o momento de implantar o código do momento de liberar a funcionalidade para os clientes.
Na prática. Uma funcionalidade nova vai para produção desligada, é ligada para poucos usuários, observada e então liberada para todos — ou desligada na hora se der problema.
Por que importa para as pessoas. Reduz o risco e o estresse dos lançamentos, porque o recuo é imediato. Interruptores esquecidos, porém, acumulam complexidade que alguém terá de limpar.
sem elo nos artigos lidos até agora

Infraestrutura como código — **infrastructure as code, IaC, ephemeral environments**
O que é. Servidores, redes e ambientes deixam de ser configurados à mão e passam a ser descritos em arquivos, como se fossem código, que uma ferramenta usa para criá-los automaticamente. O ambiente vira algo reproduzível e versionado.
Na prática. Criar um ambiente de teste idêntico ao de produção leva minutos e não depende de quem "sabe como foi configurado". Mudanças na infraestrutura passam pelo mesmo processo de revisão e teste do software.
Por que importa para as pessoas. Acaba com o "na minha máquina funciona" e com a dependência de poucos especialistas. Exige que os desenvolvedores aprendam um campo que antes era só da operação.
sem elo nos artigos lidos até agora

Operação

Como o software é mantido no ar depois de entregue: quem cuida dele, quem é acionado quando quebra e como a organização aprende com as falhas.

Responsabilidade de operação pelo time — **you build it, you run it; DevOps ownership**
O que é. O mesmo time que desenvolve o software também o mantém funcionando em produção, em vez de entregá-lo a um departamento separado de operações. É a ideia central do DevOps: "quem constrói, opera".
Na prática. O time implanta, monitora, atende incidentes e decide melhorias de estabilidade do seu próprio serviço. Isso costuma vir junto com escalas de plantão.
Por que importa para as pessoas. Dá mais autonomia, visão do resultado do próprio trabalho e menos dependência de outros departamentos. Também traz mais responsabilidade, trabalho fora de hora e uma variedade maior de tarefas a dominar.
3 elos nesta pesquisa — ver achados

On-call — **on-call rotation, pager duty, plantão**
O que é. Escala de plantão: em cada período, uma pessoa do time fica responsável por responder a alertas e incidentes do sistema, inclusive à noite e nos fins de semana. É a contrapartida operacional de manter serviços no ar 24 horas.
Na prática. Ferramentas de alerta acionam o plantonista pelo celular; ele precisa responder em minutos. O desenho da escala (quantas pessoas, com que frequência, com que compensação) varia muito entre empresas.
Por que importa para as pessoas. É um dos pontos mais sensíveis para o bem-estar: interrompe o descanso e a vida familiar, e alertas em excesso geram fadiga. Escalas divididas, alertas bem calibrados e compensação adequada mudam muito esse efeito.
1 elo nesta pesquisa — ver achados

Gestão de incidentes — **incident management, incident response, runbooks, troubleshooting guides (TSG), rollback**
O que é. O processo organizado para responder quando o sistema falha em produção: quem é acionado, quem coordena, como se diagnostica e como se restaura o serviço. Inclui os roteiros escritos (runbooks) que guiam o diagnóstico.
Na prática. Há níveis de gravidade, papéis definidos durante a crise e guias passo a passo para problemas conhecidos. A métrica típica é o tempo até restaurar o serviço.
Por que importa para as pessoas. Bons roteiros e papéis claros reduzem a pressão e a improvisação numa hora já tensa. Roteiros desatualizados ou confusos fazem o contrário: aumentam o esforço e espalham interrupções pelo time.
3 elos nesta pesquisa — ver achados

Postmortem sem culpados — **blameless postmortem, post-incident review, retrospective**
O que é. Depois de uma falha relevante, o time escreve uma análise do que aconteceu e por quê, com foco em causas do sistema e do processo, e não em achar um culpado. O objetivo é aprender e evitar a repetição.
Na prática. Uma reunião e um documento com a linha do tempo, as causas e as ações de melhoria, compartilhado com a organização.
Por que importa para as pessoas. Bem conduzido, cria segurança para relatar erros e registra conhecimento que antes se perdia. Se vira caça ao culpado ou burocracia, tem o efeito inverso e soma reuniões a uma agenda já cheia.
2 elos nesta pesquisa — ver achados

Monitoramento e observabilidade — **monitoring, observability, metrics, logs, alerts, SLOs**
O que é. Os instrumentos que mostram, em tempo real, se o sistema está saudável: métricas, registros (logs), rastreamento de requisições e alertas. Observabilidade é a capacidade de entender o que está acontecendo por dentro a partir desses sinais.
Na prática. Painéis acompanham indicadores como tempo de resposta e taxa de erro; metas de nível de serviço (SLOs) definem o que é aceitável; alertas disparam quando os limites são cruzados.
Por que importa para as pessoas. Permite agir antes de o cliente reclamar e diagnosticar mais rápido. Alertas demais ou mal calibrados geram ruído, fadiga e interrupções desnecessárias.
1 elo nesta pesquisa — ver achados

Site Reliability Engineering — **SRE, error budgets, toil**
O que é. Abordagem criada no Google que trata a operação como problema de engenharia: profissionais (SREs) usam software e automação para manter os sistemas confiáveis. Dois conceitos centrais são o "orçamento de erro" (quanto de falha é aceitável) e a eliminação de "toil" — o trabalho operacional manual, repetitivo e sem valor duradouro.
Na prática. Os SREs medem quanto tempo gastam em toil, automatizam o que é repetitivo e dividem o tempo entre operação e projetos de melhoria.
Por que importa para as pessoas. Eliminar toil libera tempo para trabalho mais interessante e reduz o desgaste. Pode também gerar insegurança ("se automatizar, meu trabalho some") e tirar dos novatos as tarefas simples com que aprendiam.
5 elos nesta pesquisa — ver achados

Negócio

Como as decisões de produto passam a ser tomadas em ciclos curtos, com base no uso real, e não em planos fechados no início do ano.

Experimentação contínua — continuous experimentation, A/B testing, online controlled experiments
O que é. Decisões de produto passam a ser testadas com usuários reais antes de serem adotadas: duas versões são mostradas a grupos diferentes e mede-se qual funciona melhor (teste A/B).
Na prática. O time lança variações pequenas, coleta dados de uso e decide com base nos resultados, em ciclos de dias ou semanas.
Por que importa para as pessoas. Dá ao time evidência para defender decisões e ver o efeito do próprio trabalho. Pode aumentar a cobrança por resultados mensuráveis e o volume de mudanças em produção.
sem elo nos artigos lidos até agora

Planejamento contínuo — continuous planning, rolling roadmap, continuous prioritization
O que é. O plano e as prioridades do produto são revistos continuamente, a cada ciclo curto, em vez de fixados num plano anual. Negócio e tecnologia ajustam a rota juntos à medida que aprendem.
Na prática. Roteiros de produto (roadmaps) com horizonte curto, priorização revisitada a cada poucas semanas e decisões baseadas no que foi entregue e medido.
Por que importa para as pessoas. Reduz o trabalho jogado fora por planos que envelheceram. Prioridades que mudam o tempo todo, porém, podem virar instabilidade e pressão permanente.
sem elo nos artigos lidos até agora

Transversais

Práticas que atravessam as demais: segurança, plataforma interna, medição e o desenho dos times.

DevSecOps — DevSecOps, shift-left security, security in the pipeline
O que é. A segurança deixa de ser uma auditoria no fim do projeto e passa a ser parte do fluxo contínuo: verificações automáticas de vulnerabilidades dentro do pipeline e responsabilidade compartilhada pelos times de desenvolvimento.
Na prática. Ferramentas analisam código e dependências a cada mudança e podem bloquear a entrega; o desenvolvedor corrige na hora, em vez de receber um relatório meses depois.
Por que importa para as pessoas. Problemas de segurança chegam mais cedo e mais baratos de corrigir. Para os times, significa mais um tipo de verificação, mais conhecimento exigido e, se mal introduzida, atrito entre segurança e desenvolvimento.
sem elo nos artigos lidos até agora

Engenharia de plataforma — platform engineering, internal developer platform, golden paths, self-service
O que é. Um time interno constrói uma “plataforma” — ferramentas, ambientes e caminhos prontos — que os demais times usam por autoatendimento, sem precisar montar sua própria infraestrutura. É tratar a infraestrutura interna como um produto para os desenvolvedores.
Na prática. Criar um serviço novo, um ambiente ou um pipeline vira uma operação padronizada de poucos cliques, seguindo “caminhos dourados” recomendados.
Por que importa para as pessoas. Reduz a carga cognitiva de quem desenvolve, que não precisa dominar toda a pilha técnica. A plataforma, porém, pode virar gargalo ou limitar a autonomia se não atender às necessidades reais dos times.
sem elo nos artigos lidos até agora

Métricas de entrega — DORA metrics, lead time, deployment frequency, change failure rate, MTTR
O que é. Indicadores padronizados de desempenho da entrega de software, popularizados pela pesquisa DORA: frequência de implantação, tempo entre a alteração e a produção, taxa de mudanças que causam falha e tempo para restaurar o serviço.
Na prática. Os números são coletados automaticamente do pipeline e usados para acompanhar a evolução dos times e orientar investimentos.
Por que importa para as pessoas. Dão visibilidade ao trabalho e ajudam a justificar melhorias. Usados como meta individual ou ranking entre times, tendem a gerar pressão e distorções.
sem elo nos artigos lidos até agora

Times multifuncionais Dev–Ops — **cross-functional DevOps teams, Dev and Ops collaboration**
O que é. Desenvolvedores e profissionais de operação passam a trabalhar no mesmo time, com objetivos comuns, em vez de em departamentos separados que se comunicam por chamados. É a dimensão organizacional e cultural do DevOps.
Na prática. Mesmo time, mesmas cerimônias, responsabilidade compartilhada pelo produto em produção; papéis antes distintos se misturam.
Por que importa para as pessoas. Melhora a colaboração e o reconhecimento mútuo entre áreas que antes se culpavam. A mistura de papéis pode gerar sobrecarga e conflito, por exemplo quando a mesma pessoa toca projeto e incidentes ao mesmo tempo.
4 elos nesta pesquisa — ver achados

Agregado e propostas

Categorias usadas quando o artigo não separa as práticas, e práticas novas propostas durante a leitura, ainda não incorporadas ao catálogo.

ESC/DevOps em geral — **DevOps, continuous software engineering**
O que é. Usada quando o estudo trata da adoção de DevOps ou de Engenharia de Software Contínua como um pacote, sem separar qual prática específica produziu o efeito.
Na prática. Por exemplo, a comparação entre times “DevOps” e times “ágeis” sem detalhar quais práticas cada um adota.
Por que importa para as pessoas. Mostra o efeito do conjunto, mas não permite dizer qual prática deve ser ajustada; por isso o catálogo a usa só como último recurso.
1 elo nesta pesquisa — ver achados

Bots de desenvolvimento no pipeline de CI/CD — **developer bots, DevBots (proposta na leitura de Asthana et al. 2023)**
O que é. Programas automáticos que atuam no fluxo de trabalho dos desenvolvedores — por exemplo, comentando nas revisões de código, apontando violações de padrão, sugerindo revisores ou respondendo dúvidas frequentes. Prática proposta durante a leitura, ainda não incorporada ao catálogo.
Na prática. O bot age sozinho a cada pull request ou chamado; o desenvolvedor recebe as mensagens e decide o que fazer com elas.
Por que importa para as pessoas. Podem orientar novatos e poupar especialistas de perguntas repetidas. Também geram ruído, mais coisas para ler e cobranças que competem com o prazo de quem está entregando.
5 elos nesta pesquisa — ver achados

2. O que o corpus da etapa 1 já dizia

Varredura do texto completo dos 65 artigos com fatores: 37 não mencionam nenhuma prática de ESC, 22 só de passagem (referência, cargo do participante, nome de ferramenta) e 6 tocam no elo, sempre como evidência secundária.

Artigo	O que diz sobre prática → fator
Trinkenreich et al. (2023)	*Postmortem* sem culpados e times multifuncionais levam à cultura generativa (Westrum/DORA), que reduz o *burnout* — mas a ligação prática → cultura é citada, não medida
Trinkenreich et al. (2025)	Recomenda *postmortems* sem culpados para reforçar a segurança psicológica e o pertencimento
Cerqueira et al. (2024)	Líder empático equilibra as escalas de *on-call*; *postmortem* honesto reduz a culpa — em recomendações apresentadas como além da análise
Jafarzadeh et al. (2025)	Ciclos de entrega contínua mantêm intensidade alta e sem pausa, caminho para o *burnout*
Kumar et al. (2025)	"Automatizar tudo para a entrega contínua" libera tempo para o trabalho que o desenvolvedor valoriza
Benlian et al. (2025)	Em projeto grande, atraso pequeno se propaga pelo *pipeline* de entrega contínua e gera estresse e sensação de fracasso

Piloto da extração pelo texto completo

Três desses artigos foram lidos inteiros com as regras da skill. Só um elo se sustenta: recomendação apresentada além dos dados, ou com a ligação vinda de trabalho citado, não conta.

> **Benlian et al. (2025)** — Entrega contínua **↑ aumenta** *desfecho:* estresse e sensação de fracasso. **Mecanismo:** em projeto grande, o *pipeline* de entrega contínua interliga o trabalho de muitos, e um atraso pequeno se propaga e é vivido como falha em escala maior. **Condições:** projetos ágeis de larga escala; em projeto pequeno o participante I30 não relata o efeito. **Evidência:** empírica direta.
> Supplementary Qualitative Insights (I15): "it creates extra stress because even minor delays have ripple effects on our continuous delivery pipeline and feel like a failure on a much larger scale".

Cerqueira et al. (2024) e Trinkenreich et al. (2023): nenhum elo. Westrum (2004) e o relatório DORA, citados por Trinkenreich, viram candidatos ao corpus.

3. Buscas, triagem e leitura

Busca	O que pega	Registros
1 — ESC × QVT	Práticas de ESC + termos de bem-estar **e dos fatores** (carga, pressão de prazo, interrupção, carga cognitiva, *toil*, horas de trabalho, segurança psicológica…) + quem produz software	187
2 — estudos empíricos de ESC	Estudos de adoção e desafios de CI/CD/DevOps com entrevista, *survey* ou estudo de caso, menos o que já cai na busca 1	790

Filtros: artigo, conferência, revisão ou capítulo; inglês ou português; sem Medicina e Enfermagem. Calibração com 12 artigos-semente: a união das buscas recupera 8. Os 4 que ficam fora não citam nenhuma prática de ESC no resumo e entram por *snowballing*.

Triagem dos abstracts da busca 1

Veredito	Registros
Aprovado	17
Talvez (precisa do texto completo)	15
Reprovado — ferramenta/técnica	68
Reprovado — fora de ES	43
Reprovado — sem elo	17
Reprovado — sem pessoas	10
Reprovado — secundário	9
Reprovado — sem elo: CI só como atributo do ágil; o estresse é atribuído às limitações do ágil	2
Reprovado — veículo: revista de prática (Dr. Dobb's); sem elo entre CI e pessoas	1
Reprovado — ferramenta/técnica: IA generativa, sem prática de ESC	1
Reprovado — ferramenta/técnica: estimativa com LLM, sem prática de ESC	1
Reprovado — sem elo: o efeito sobre equilíbrio trabalho-vida é da IA generativa, não de prática de ESC	1
Reprovado — sem elo: DevOps só como cargo em anúncios de vagas	1
Reprovado — sem elo: o efeito sobre carga cognitiva é das ferramentas de IA; CI/CD só como recomendação	1

Leitura do texto completo dos 32 candidatos

Os abstracts afirmavam 22 elos; o texto completo sustenta 65, em 22 artigos. Vários elos do abstract caíram (o efeito estava só no resumo, ou vinha de trabalho citado) e muitos novos apareceram nas falas, tabelas e comparações antes/depois.

Resultado	Artigos	Quais
Influencia fatores: aprovado com elo	22	**Ahmad et al. (2023)**, **Anandayuvaraj et al. (2026)**, **Andreyev (2025)**, **Asthana et al. (2023)**, **Claes et al. (2017)**, **Fluri et al. (2024)**, **Hemon et al. (2018)**, **Itkonen et al. (2016)**, **Jeyam et al. (2026)**, **Kafoe et al. (2026)**, **Kassab et al. (2026)**, **Kärpänoja et al. (2016)**, **Limoncelli (2018)**, **Parry et al. (2022)**, **Pinto et al. (2017)**, **Riedl et al. (2025)**, **Rossi et al. (2016)**, **Shetty et al. (2022)**, **Soltren e Mulligan (2023)**, **Souza e Silva (2017)**, **Valenzuela-Toledo et al. (2024)**, **Zhang et al. (2018)**
Aprovado, mas sem elo no texto	1	**Fluri et al. (2023)**
Reprovado na leitura	3	**Gummadi et al. (2026)**, **Lertpongrujikorn et al. (2026)**, **Tennakoon e Jayatissa (2025)**
Sem PDF	6	**Ferazza e Mersinas (2026)**, **Claassen e Boekhorst (2015)**, **Hemon-Hildgen et al. (2020)**, **Jothi et al. (2026)**, **Kirpitsas e Pachidis (2026)**, **Kulas et al. (2014)**

4. Os elos: prática × fator

Cada linha liga uma prática de ESC a um fator do catálogo (ou a um fator emergente, ou direto a um desfecho). ↑ aumenta, ↓ reduz, ↕ ambivalente; o número é a quantidade de artigos. A última coluna dá os artigos e, entre colchetes, a força da evidência (D, I, F).

Prática	Fator	↑	↓	↕	Artigos
Automação de testes	Manutenção do ambiente de desenvolvimento (fator −)	3			**Ahmad et al. (2023)** [D]; **Kärpänoja et al. (2016)** [D]; **Parry et al. (2022)** [D]
Automação de testes	Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)	1			**Andreyev (2025)** [I]
Bots de desenvolvimento no pipeline de CI/CD	Carga cognitiva (fator −)	1			**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Carga de trabalho (fator −)			1	**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Conflito de papéis (fator −)	1			**Asthana et al. (2023)** [F]
Bots de desenvolvimento no pipeline de CI/CD	Documentação interna (fator +)	1			**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Integração de novatos (fator +)	1			**Asthana et al. (2023)** [D]
ESC/DevOps em geral	Desenho do trabalho (fator −, nome neutro*)	1			**Hemon et al. (2018)** [I]
Entrega contínua	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		2		**Kärpänoja et al. (2016)** [D]; **Limoncelli (2018)** [F]
Entrega contínua	Conflito trabalho-família (fator −)		2		**Kärpänoja et al. (2016)** [I]; **Limoncelli (2018)** [F]
Entrega contínua	Carga de trabalho (fator −)			1	**Itkonen et al. (2016)** [D]
Entrega contínua	Desenho do trabalho (fator −, nome neutro*)	1			**Itkonen et al. (2016)** [D]
Entrega contínua	Interação com clientes (fator −, nome neutro*)		1		**Itkonen et al. (2016)** [I]
Entrega contínua	Relações no trabalho (fator +)	1			**Kärpänoja et al. (2016)** [I]
Entrega contínua	Risco de falha na implantação (fator emergente, Técnico e ferramentas)		1		**Itkonen et al. (2016)** [D]
Gestão de incidentes	Carga cognitiva (fator −)	1			**Shetty et al. (2022)** [D]
Gestão de incidentes	Carga de trabalho (fator −)		1		**Shetty et al. (2022)** [I]
Gestão de incidentes	Interrupções no trabalho (fator −)	1			**Shetty et al. (2022)** [D]
Implantação contínua	Carga cognitiva (fator −)	1			**Hemon et al. (2018)** [D]
Implantação contínua	Carga de trabalho (fator −)		1		**Zhang et al. (2018)** [D]
Integração contínua	Carga de trabalho (fator −)	2			**Pinto et al. (2017)** [D]; **Soltren e Mulligan (2023)** [F]
Integração contínua	Alinhamento do uso do tempo (fator +)		1		**Pinto et al. (2017)** [D]

Prática	Fator	↑	↓	↕	Artigos
Integração contínua	Confiança para alterar o código (fator emergente, Técnico e ferramentas)	1			**Pinto et al. (2017)** [D]
Integração contínua	Integração de novatos (fator +)		1		**Pinto et al. (2017)** [D]
Integração contínua	Manutenção do ambiente de desenvolvimento (fator −)		1		**Soltren e Mulligan (2023)** [I]
Integração contínua	Percepção de recompensa (fator emergente, Organizacional)	1			**Riedl et al. (2025)** [F]
Integração contínua	Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)	1			**Pinto et al. (2017)** [D]
Monitoramento e observabilidade	Carga cognitiva (fator −)			1	**Jeyam et al. (2026)** [I]
On-call	Conflito trabalho-família (fator −)	1			**Jeyam et al. (2026)** [D]
Pipeline de CI/CD	Manutenção do ambiente de desenvolvimento (fator −)	3		1	**Parry et al. (2022)** [D]; **Pinto et al. (2017)** [D]; **Valenzuela-Toledo et al. (2024)** [I]; **Zhang et al. (2018)** [D]
Pipeline de CI/CD	Carga cognitiva (fator −)		1	1	**Fluri et al. (2024)** [D]; **Jeyam et al. (2026)** [I]
Pipeline de CI/CD	Interrupções no trabalho (fator −)	1	1		**Fluri et al. (2024)** [D]; **Kassab et al. (2026)** [F]
Pipeline de CI/CD	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		1		**Fluri et al. (2024)** [D]
Pipeline de CI/CD	Desenho do trabalho (fator −, nome neutro*)	1			**Fluri et al. (2024)** [D]
Pipeline de CI/CD	*desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração)	1			**Kassab et al. (2026)** [I]
Pipeline de CI/CD	*desfecho:* sentimento negativo expresso nas mensagens de commit	1			**Souza e Silva (2017)** [D]
Postmortem sem culpados	Carga de trabalho (fator −)			1	**Anandayuvaraj et al. (2026)** [I]
Postmortem sem culpados	Documentação interna (fator +)	1			**Anandayuvaraj et al. (2026)** [I]
Releases frequentes	Pressão de prazos (fator −)	1	1		**Hemon et al. (2018)** [D]; **Rossi et al. (2016)** [I]
Releases frequentes	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		1		**Kärpänoja et al. (2016)** [D]
Releases frequentes	Carga cognitiva (fator −)		1		**Kärpänoja et al. (2016)** [D]
Releases frequentes	Conflito trabalho-família (fator −)		1		**Claes et al. (2017)** [D]
Responsabilidade de operação pelo time	Conflito trabalho-família (fator −)	1			**Jeyam et al. (2026)** [D]
Responsabilidade de operação pelo time	Desenho do trabalho (fator −, nome neutro*)	1			**Kärpänoja et al. (2016)** [I]
Responsabilidade de operação pelo time	Pressão por atualização (fator −)	1			**Jeyam et al. (2026)** [D]

Prática	Fator	↑	↓	↕	Artigos
Site Reliability Engineering	Alinhamento do uso do tempo (fator +)	1			**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Desenho do trabalho (fator –, nome neutro*)	1			**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Desenvolvimento de competências (fator emergente, Individual)			1	**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Insegurança no emprego (fator –)	1			**Kafoe et al. (2026)** [I]
Site Reliability Engineering	Senso de realização (fator +)			1	**Kafoe et al. (2026)** [D]
Times multifuncionais Dev–Ops	Apoio do gestor (fator +)		1		**Hemon et al. (2018)** [I]
Times multifuncionais Dev–Ops	Conflito de papéis (fator –)	1			**Hemon et al. (2018)** [D]
Times multifuncionais Dev–Ops	Reconhecimento no trabalho (fator +)			1	**Hemon et al. (2018)** [D]
Times multifuncionais Dev–Ops	Relações no trabalho (fator +)	1			**Hemon et al. (2018)** [D]

* **Desenho do trabalho**: a descrição é neutra (variedade, autonomia, retorno), mas na etapa 1 só aparece o caso ruim (4 artigos, todos negativos). Nos elos desta etapa, "aumenta" quer dizer trabalho **mais bem desenhado** — o que atenua o fator negativo, ao contrário do que a polaridade sugere.

* **Interação com clientes**: o nome é neutro, mas a descrição e a polaridade da etapa 1 (8 negativos) tratam do desgaste com o cliente. Nos elos desta etapa o efeito se refere a esse desgaste: "reduz" quer dizer menos retrabalho e espera pelo cliente.

Por fator

A mesma informação vista do lado dos fatores: quais práticas mexem em cada um.

Fator	Práticas que mexem nele
Manutenção do ambiente de desenvolvimento (fator –)	Automação de testes ↑ ×3; Pipeline de CI/CD ↑ ×3; Integração contínua ↓; Pipeline de CI/CD ↕
Carga cognitiva (fator –)	Bots de desenvolvimento no pipeline de CI/CD ↑; Pipeline de CI/CD ↓; Implantação contínua ↑; Monitoramento e observabilidade ↕; Pipeline de CI/CD ↕; Releases frequentes ↓; Gestão de incidentes ↑
Carga de trabalho (fator –)	Postmortem sem culpados ↕; Bots de desenvolvimento no pipeline de CI/CD ↕; Entrega contínua ↕; Integração contínua ↑ ×2; Gestão de incidentes ↓; Implantação contínua ↓
Conflito trabalho-família (fator –)	Releases frequentes ↓; On-call ↑; Responsabilidade de operação pelo time ↑; Entrega contínua ↓ ×2
Desenho do trabalho (fator –, nome neutro*)	Pipeline de CI/CD ↑; ESC/DevOps em geral ↑; Entrega contínua ↑; Site Reliability Engineering ↑; Responsabilidade de operação pelo time ↑
Ansiedade de implantação (fator emergente, Técnico e ferramentas)	Pipeline de CI/CD ↓; Entrega contínua ↓ ×2; Releases frequentes ↓
Interrupções no trabalho (fator –)	Pipeline de CI/CD ↓; Pipeline de CI/CD ↑; Gestão de incidentes ↑
Alinhamento do uso do tempo (fator +)	Site Reliability Engineering ↑; Integração contínua ↓
Conflito de papéis (fator –)	Bots de desenvolvimento no pipeline de CI/CD ↑; Times multifuncionais Dev–Ops ↑
Documentação interna (fator +)	Postmortem sem culpados ↑; Bots de desenvolvimento no pipeline de CI/CD ↑
Integração de novatos (fator +)	Bots de desenvolvimento no pipeline de CI/CD ↑; Integração contínua ↓
Pressão de prazos (fator –)	Releases frequentes ↑; Releases frequentes ↓

Fator	Práticas que mexem nele
Relações no trabalho (fator +)	Times multifuncionais Dev–Ops ↑; Entrega contínua ↑
Apoio do gestor (fator +)	Times multifuncionais Dev–Ops ↓
Confiança para alterar o código (fator emergente, Técnico e ferramentas)	Integração contínua ↑
Desenvolvimento de competências (fator emergente, Individual)	Site Reliability Engineering ↕
Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)	Automação de testes ↑
Insegurança no emprego (fator −)	Site Reliability Engineering ↑
Interação com clientes (fator −, nome neutro*)	Entrega contínua ↓
Percepção de recompensa (fator emergente, Organizacional)	Integração contínua ↑
Pressão por atualização (fator −)	Responsabilidade de operação pelo time ↑
Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)	Integração contínua ↑
Reconhecimento no trabalho (fator +)	Times multifuncionais Dev–Ops ↕
Risco de falha na implantação (fator emergente, Técnico e ferramentas)	Entrega contínua ↓
Senso de realização (fator +)	Site Reliability Engineering ↕
desfecho: emoção negativa expressa nas discussões (sobretudo tristeza/frustração)	Pipeline de CI/CD ↑
desfecho: sentimento negativo expresso nas mensagens de commit	Pipeline de CI/CD ↑

5. Achados por prática

Cada elo com o mecanismo e as condições segundo o artigo e a citação do texto completo. O nome do artigo leva à ficha dele (seção 6); o nome da prática, à descrição dela (seção 1).

Pipeline de CI/CD

12 elos em 8 artigo(s)

> **O que é.** É a "linha de montagem" automatizada que leva uma alteração de código desde o envio pelo desenvolvedor até o ambiente de produção: compilar, testar, empacotar, aprovar e implantar, em etapas encadeadas. A prática aqui é o pipeline como algo que o time constrói, mantém e de que depende no dia a dia. Descrição completa na seção 1.

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*

Contexto: Mesmo estudo de três casos com pipeline de CI/CD em bancos Oracle e entrevistas antes e depois resumidas pelos autores. O estresse de implantar só foi relatado antes no UC3-RET, back-end de logística de varejo com 35 desenvolvedores, onde o código era alterado diretamente em produção, sem validação prévia, e os erros de implantação tinham de ser corrigidos na hora; reduzir esse estresse era um dos objetivos do time. Depois, com controle de versão, análise estática e migrações executadas num banco de CI antes da produção, os três casos relataram mais confiança para implantar, e a Tabela 4 mostra queda das falhas em produção no UC1-DWH (de 30% para 3%) e no UC3-RET (de 10% para 2%). O elo combina, assim, o relato de antes de um só caso com o de depois dos três, sem falas literais nem medida de estresse. Os autores atribuem a confiança aos testes automáticos do pipeline, mas no UC1-DWH e no UC3-RET não havia testes unitários e a parcela de código testado automaticamente continuou em 0%.

Mecanismo: antes, mudar código direto em produção sem validação gerava medo de implantar; o feedback imediato do CI e os testes automáticos do pipeline dão confiança de que a mudança funciona antes de chegar à produção. **Condições:** o estresse de implantar foi relatado antes só no UC3-RET, que alterava código direto em produção; o aumento de confiança foi relatado nos três casos. **Evidência:** empírica direta.

> Seção 6.3, UC3-RET (antes do pipeline): *"In UC3-RET, developers reported high stress levels when altering code in production directly. Their goal was to reduce this stress, so changes can be deployed with confidence."*
>
> Síntese dos autores sobre as entrevistas anteriores ao pipeline no UC3-RET: os desenvolvedores relataram níveis altos de estresse ao alterar código diretamente em produção e queriam reduzi-lo para poder implantar mudanças com confiança. É o estado antes da prática e o único relato explícito de estresse de implantação no artigo; a causa é implantar sem nenhuma etapa intermediária de validação.

> Seção 7.1 (discussão): *"The instant feedback the developers get from the CI increases trust in deploying features to production."*
>
> Na discussão, os autores atribuem ao feedback imediato do CI (que, a cada commit, aplica as migrações num banco de CI e roda as verificações) o aumento da confiança em implantar features em produção. É o mecanismo do elo: o desenvolvedor sabe se a mudança funciona antes que ela chegue à produção.

> Seção 7.1 (discussão), três casos: *"In all three use cases, developer confidence in adding changes increased because they could rely on the automated pipeline to test them thoroughly"*
>
> Os autores generalizam para os três casos: a confiança dos desenvolvedores em fazer mudanças aumentou porque podiam contar com o pipeline automatizado para testá-las a fundo, o que, acrescentam, reduziu a carga cognitiva. É o depois do elo; pela Tabela 4, porém, o UC1-DWH e o UC3-RET seguiram com 0% de código testado automaticamente, de modo que nesses casos a verificação do pipeline se apoia mais na execução das migrações no banco de CI, nos testes de sistema e nas checagens pós-migração do que em testes unitários.

↓ **reduz** Carga cognitiva (fator −)

Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*

Contexto: O artigo combina uma revisão sinóptica da literatura com três estudos de caso industriais em projetos de banco de dados Oracle, nos quais os autores (uma delas consultora das empresas) implantaram um pipeline de CI/CD, compararam métricas de 5 semanas antes e 5 depois e entrevistaram dois desenvolvedores por caso antes e depois (12 entrevistas de 90 minutos). Os casos são o UC1-DWH (data warehouse de uma seguradora suíça, 20 desenvolvedores), o UC2-BE (back-end de banco de dados de uma administração pública europeia, 7 desenvolvedores) e o UC3-RET (back-end de uma organização internacional de logística de varejo, 35 desenvolvedores). Antes, integração e implantação eram manuais: no UC1-DWH e no UC2-BE um coordenador de release juntava e aplicava as mudanças, e no UC3-RET os desenvolvedores alteravam o código diretamente em produção. O elo vem das entrevistas, que os autores apenas resumem, sem falas literais, por sigilo; não há medida de carga cognitiva, só o relato. A redução foi relatada nos três casos, mas no UC3-RET a mudança trouxe uma curva de aprendizado íngreme, que é uma carga nova durante a transição.

Mecanismo: o pipeline automatiza integração, release e implantação que antes eram manuais e testa cada mudança, então desenvolvedores e coordenadores de release deixam de manter várias features em paralelo prontas para consertar e passam a confiar no pipeline para implantar. **Condições:** relatado nos três casos; no UC3-RET a troca para controle de versão e novo processo de release trouxe uma curva de aprendizado íngreme na transição. **Evidência:** empírica direta.

> Seção 5.2, UC2-BE (antes do pipeline): *“Having several features in the pipeline in parallel also increased the cognitive load on the developers because they always had to be prepared to fix packages as they got promoted.”*
>
> Fecha o terceiro problema que o time do UC2-BE relatou na entrevista anterior ao pipeline, cuja primeira parte diz que a carga cognitiva do coordenador de release era alta por somar tarefas de desenvolvimento a muitas tarefas manuais de integração, release e implantação. A citação acrescenta os desenvolvedores: com várias features em andamento ao mesmo tempo, tinham de estar sempre prontos para consertar pacotes à medida que eram promovidos de um ambiente para o seguinte. Descreve o estado antes da prática e o tipo de carga que o pipeline viria a retirar.

> Seção 6.3, três casos (depois do pipeline): *“The feedback from the three use cases after the pipeline implementation revealed that developers and release coordinators experience a lighter cognitive load in daily work.”*
>
> Síntese dos autores sobre as entrevistas feitas depois da implantação do pipeline nos três casos: desenvolvedores e coordenadores de release passaram a ter uma carga cognitiva mais leve no dia a dia. É a evidência direta do elo no momento posterior, ainda que na voz dos autores e não dos entrevistados.

> Seção 6.3, três casos (depois do pipeline): *“They can now concentrate on development and trust the pipeline to deploy the changes correctly.”*
>
> Continuação da frase anterior, que explica de onde vem o alívio: os entrevistados podem agora se concentrar no desenvolvimento e confiar que o pipeline implantará as mudanças corretamente. É o mecanismo do elo: a tarefa de acompanhar e garantir a implantação sai da cabeça das pessoas e passa para a automação.

> Seção 7.1 (discussão): *“The interviews showed that developers now have fewer parallel tasks and can test features independently”*
>
> Ao discutir o impacto do CI/CD, os autores dizem que as entrevistas mostraram menos tarefas paralelas e a possibilidade de testar cada feature de forma independente. Liga o depois ao problema descrito no UC2-BE antes (várias features em paralelo exigindo prontidão para consertos); a mesma seção conclui que a confiança no pipeline reduziu a carga cognitiva.

> Seção 6.3, UC3-RET (depois do pipeline): *“the usage of a version control system together with a new release process introduces a steep learning curve.”*
>
> Relato do UC3-RET depois do pipeline, no fim da análise qualitativa: o novo processo é um desafio para os desenvolvedores, porque usar controle de versão junto com um novo processo de release exige uma curva de aprendizado íngreme. Nesse caso, antes, o código era alterado direto em produção e só copiado para o controle de versão uma vez por noite. É a condição do elo: na transição, o pipeline acrescenta carga de aprendizado antes de reduzir a carga do dia a dia.

↕ **ambivalente** Carga cognitiva (fator −)

Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*

Contexto: Estudo qualitativo do ICSE-SEIP 2026 com 26 entrevistas semiestruturadas (45 a 60 minutos, em alemão, citações traduzidas para o inglês pelos autores) com membros de times DevOps de duas empresas suíças reguladas: uma seguradora global (Caso 1, participantes 1 a 13) e um banco (Caso 2, participantes 14 a 26), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança e incerteza tecnológicas). A heterogeneidade dos pipelines de CI/CD aparece em dois pontos: como tecno-complexidade, quando partes da infraestrutura ainda exigem implantação manual e obrigam a alternar entre processos manuais e automatizados, e como tecno-incerteza, porque a maturidade dos pipelines varia muito entre times e departamentos, alguns com a cadeia inteira automatizada e outros fazendo quase tudo à mão. Como coping no nível organizacional, um desenvolvedor do banco diz que seria mais fácil se tudo fosse padronizado, e os autores concluem que a padronização alivia a carga cognitiva. O elo se apoia em falas e na interpretação dos autores; não há medida de carga, e os próprios autores lembram, na discussão, que padronizar ferramentas pode reduzir flexibilidade e inovação.

Mecanismo: pipelines heterogêneos entre times e etapas de implantação ainda manuais obrigam a trocar de tecnologia e de modo de trabalho, o que exige troca de contexto mental; pipelines padronizados e automatizados aliviam essa carga.

Condições: aumenta quando a maturidade dos pipelines varia entre times e parte da implantação é manual; reduz com padronização das tecnologias de pipeline e automação de build, teste e implantação. **Evidência:** empírica indireta.

> Seção 4.2 (tecno-incerteza), Application Manager, Nr. 21: *“They have also set up pipelines and mapped out the entire chain. Some are very advanced, and some are still in the Stone Age. They do everything by hand”*
>
> Fala do participante 21, gerente de aplicação do banco (Caso 2) há mais de 15 anos na função, sobre a variação entre times: alguns montaram pipelines e mapearam toda a cadeia, outros ainda estão na idade da pedra e fazem tudo à mão. Mostra a condição em que o elo aumenta a carga: maturidade desigual dos pipelines na mesma organização.

> Seção 4.2 (tecno-complexidade), síntese dos autores: *“still required manual deployment, which can lead to mental context shifts”*
>
> Frase dos autores sobre infraestruturas híbridas: algumas plataformas não estavam totalmente digitalizadas e ainda exigiam implantação manual, o que pode causar trocas de contexto mentais e dificultar o controle de dependências ocultas. É o mecanismo cognitivo do elo, formulado pelos autores.

> Seção 4.2, SE (Dev), Nr. 2: *“This is another manual; it is not automatized. Someone has to log in and perform this step in order to proceed to the next step”*
>
> Fala do participante 2, desenvolvedor da seguradora (Caso 1), descrevendo uma etapa manual no meio da cadeia: não é automatizada, alguém precisa entrar no sistema e executar o passo para que o processo siga. Ilustra a implantação parcialmente manual que interrompe o fluxo automatizado e exige a troca de contexto.

> Seção 4.3 (coping organizacional), SE (Dev), Nr. 20: *“It would be easier if you had everything standardized”*
>
> Primeira parte da fala do participante 20, desenvolvedor do banco (Caso 2) com 1 a 5 anos de experiência: seria mais fácil se tudo estivesse padronizado. Apresenta a padronização como saída desejada, o lado em que a prática bem feita reduz a carga.

> Seção 4.3, SE (Dev), Nr. 20 (continuação da mesma fala, após a elipse dos autores): *“and didn’t have to switch between pipeline technologies.”*
>
> Continuação da mesma fala, depois de um trecho omitido pelos autores: e não ter de alternar entre tecnologias de pipeline. Especifica o que a padronização evitaria, a troca entre ferramentas de pipeline diferentes de um time para outro.

> Seção 4.3, interpretação dos autores: *“standardization alleviates the cognitive load on DevOps members”*
>
> Conclusão dos autores a partir da fala do participante 20: ao reduzir a necessidade de se adaptar o tempo todo a ferramentas e plataformas diferentes, a padronização alivia a carga cognitiva dos membros de DevOps. Sustenta o sentido de redução, mas é interpretação dos autores, não medida.

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*

Contexto: Mesmo estudo de três casos com pipeline de CI/CD implantado pelos autores, métricas de 5 semanas antes e 5 depois (Tabela 4) e entrevistas resumidas. Antes, no UC1-DWH e no UC2-BE, um coordenador central de release reunia as mudanças e as aplicava (no UC1-DWH, num pacote único enviado por e-mail ao DBA), e era um ponto único de falha: sem ele, nada chegava à produção. Com o pipeline, os desenvolvedores passaram a promover as próprias mudanças pelos ambientes; no UC1-DWH o número de features por implantação caiu de 5 para 1 e o de pessoas capazes de implantar em produção subiu de 1 para 30. O ganho de autonomia, porém, dependeu de decisões da organização: no UC2-BE a produção continuou restrita ao desenvolvedor líder, porque exigia VPN do cliente, e no UC3-RET os 35 desenvolvedores já alteravam a produção diretamente antes, de modo que ali o pipeline acrescentou etapas de controle em vez de autonomia (o lead time subiu de 3 para 4 dias). O elo se apoia, portanto, sobretudo no UC1-DWH e, nos ambientes de teste e integração, no UC2-BE.
Mecanismo: o pipeline permite que cada desenvolvedor promova e implante a própria mudança e teste a feature de forma independente, sem passar por um coordenador central de release (mais autonomia e entrega do trabalho inteiro).
Condições: depende de a organização dar permissão de implantação e confiar nos desenvolvedores: só no UC1-DWH todos passaram a implantar em produção (1 para 30 pessoas); no UC2-BE a produção seguiu restrita ao desenvolvedor líder por exigência de VPN do cliente; não vale para o UC3-RET, onde os 35 desenvolvedores já alteravam a produção diretamente antes do pipeline (Tabela 4: 35 → 35) e passaram a depender do controle de versão e das verificações automáticas.
Evidência: empírica direta.

> Seção 7.1 (discussão): *“The ability to deploy features for all developers also removed the release coordinator as a single point of failure and dependency for the teams in two use cases.”*
> Ao discutir o impacto do CI/CD, os autores afirmam que a possibilidade de todos os desenvolvedores implantarem suas features eliminou o coordenador de release como ponto único de falha e de dependência em dois dos casos, que pelo resto do artigo são o UC1-DWH e o UC2-BE, os únicos com esse papel. Para o elo, mostra o redesenho do trabalho: a entrega da mudança deixa de passar por outra pessoa e volta para quem a desenvolveu.

> Seção 6.2 (análise quantitativa): *“only UC1-DWH enabled all developers to execute production deployments”*
> Depois de dizer que dar a todos o poder de implantar ajudou a aumentar o número de implantações, os autores ressalvam que só o UC1-DWH permitiu a todos os desenvolvedores implantar em produção. É a condição do elo: no UC2-BE só o desenvolvedor líder tinha o acesso por VPN exigido pelo cliente, e a autonomia ficou limitada aos ambientes de teste e integração.

> Seção 7.1, desafios organizacionais: *“the need to trust all developers to promote their changes through the pipeline without a central coordinator.”*
> Item da lista de desafios organizacionais que os times relataram durante a implantação do pipeline, ao lado da necessidade de permissão para disparar implantações em produção a partir de um processo automatizado. Mostra que a autonomia não vem só da ferramenta: a organização precisa confiar nos desenvolvedores para que promovam suas mudanças sem um coordenador central.

> Tabela 4, linha 'People able to deploy to production': *“People able to deploy to production 1 30 1 1 35 35”*
> Linha da Tabela 4, que compara as métricas antes (manual) e depois (automated) em cada caso, na ordem UC1-DWH manual e automatizado, UC2-BE manual e automatizado, UC3-RET manual e automatizado. Os números dizem que no UC1-DWH as pessoas capazes de implantar em produção passaram de 1 (o coordenador) para 30 (mais que os 20 desenvolvedores citados na descrição do caso; o artigo não explica a diferença), no UC2-BE ficaram em 1 e no UC3-RET ficaram em 35, porque ali todos já alteravam a produção diretamente. Só o primeiro caso mostra o aumento de autonomia que o elo descreve.

↓ **reduz** Interrupções no trabalho (fator −)

Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*

Contexto: Mesmo estudo de três casos de banco de dados Oracle, com pipeline de CI/CD implantado pelos autores e entrevistas antes e depois, resumidas sem falas literais. Antes do pipeline, no UC1-DWH (data warehouse de uma seguradora) e no UC2-BE (back-end de uma administração pública), os scripts de migração dos desenvolvedores eram reunidos e executados manualmente por um coordenador de release, e os erros só apareciam nessa execução, sem feedback imediato para quem os escreveu. Com o pipeline, cada commit dispara análise estática, execução das migrações num banco de CI e testes, de modo que o erro aparece cedo e para o próprio autor. O elo vem da comparação entre as entrevistas de antes, que descrevem a troca de contexto provocada pelos scripts que falhavam, e as de depois, em que os três casos relatam mais fluxo; não há contagem de interrupções.
Mecanismo: com a implantação manual, o coordenador de release voltava ao desenvolvedor a cada script que falhava, obrigando-o a largar o que fazia; com o pipeline o erro aparece cedo e o troubleshooting de integração e implantação deixa de interromper o trabalho. **Condições:** comparação antes/depois nos casos com coordenador central de release (UC1-DWH e

UC2-BE) e relato de mais fluxo nos três casos. **Evidência:** empírica direta.

> Seção 6.3, UC1-DWH e UC2-BE (antes do pipeline): *"manual deployments involved a high level of context switching, because the release coordinator needed to get back to the developers if installations failed, causing them to pause their current work for bug fixes."*
>
> Síntese dos autores sobre as entrevistas anteriores ao pipeline no UC1-DWH e no UC2-BE: a implantação manual envolvia muita troca de contexto, porque o coordenador de release precisava voltar aos desenvolvedores quando uma instalação falhava, e eles tinham de interromper o trabalho em curso para corrigir o erro. Descreve o estado antes da prática e o caminho exato da interrupção: falha descoberta tarde, por outra pessoa, que volta ao autor da mudança.

> Seção 5.2, UC2-BE (antes do pipeline): *"Not getting instant feedback if the change scripts worked caused a lot of context switching when the change coordinator needed information about a failed script."*
>
> Segundo problema relatado pelo time do UC2-BE antes do pipeline: sem saber na hora se os scripts de mudança funcionavam, havia muita troca de contexto quando o coordenador (aqui chamado change coordinator, o mesmo papel do coordenador de release) precisava de informações sobre um script que falhou. Confirma, num caso concreto, que a falta de feedback imediato é a origem das interrupções.

> Seção 6.3, três casos (depois do pipeline): *"They also mentioned that their daily work had a lot more flow since they were not interrupted by integration and deployment troubleshooting as much as before"*
>
> Relato das entrevistas posteriores ao pipeline: os desenvolvedores disseram que o trabalho diário passou a ter muito mais fluxo, porque já não eram interrompidos tanto quanto antes pelo troubleshooting (investigação e correção de falhas) de integração e implantação. O texto continua dizendo que isso eliminou a necessidade de trocas de contexto frequentes; é o depois do elo.

↑ **aumenta** Interrupções no trabalho (fator −)

Kassab et al. (2026) *— The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes*

Contexto: Mesmo estudo de mineração de 73.125 comentários de issues de Hadoop, Kafka e Flink, com temas obtidos por LDA e emoção atribuída por classificador automático. Os dados medem só o tom emocional dos comentários de cada tema; nada no estudo mede interrupção, troca de contexto ou tempo de trabalho. O elo vem da Discussão (Seções 5.1 e 5.2), em que os autores explicam por que os temas de NEI mais alto, entre eles o de testes e CI/CD, seriam os mais frustrantes: falhas de CI quebrariam o ciclo de feedback e forçariam trabalho reativo. Para isso apoiam-se em literatura anterior, como um estudo do Google sobre testes flaky, e não em dado próprio. É, portanto, interpretação dos autores, por isso a evidência é classificada como inferida.
Mecanismo: falhas de CI e de testes quebram o ciclo de feedback e o ritmo de desenvolvimento, forçando troca de contexto repetida e trabalho reativo. **Condições:** pipeline com dívida (builds quebrados, testes flaky); interpretação dos autores sobre os temas de NEI alto, sem dado que meça interrupção. **Evidência:** inferida.

> Seção 5.2: *"The high-NEI themes identified in our study—particularly CI/test failures and observability or configuration issues—are precisely the kinds of concerns that interrupt development rhythm and force repeated context switching"*
>
> Trecho da discussão sobre produtividade e bem-estar: os autores afirmam que os temas de NEI alto, sobretudo falhas de CI e de testes e problemas de observabilidade ou configuração, são justamente os que interrompem o ritmo de desenvolvimento e forçam troca de contexto repetida. É a formulação explícita do mecanismo do elo, mas como explicação para o resultado emocional, não como achado medido.

> Seção 5.1: *"Operational and infrastructure-related debt frequently disrupt feedback loops (e.g., broken pipelines, opaque logs, misconfigurations)"*
>
> Trecho anterior da discussão: a dívida operacional e de infraestrutura interromperia com frequência os ciclos de feedback, com exemplos como pipelines quebrados, logs opacos e configurações erradas; a frase continua dizendo que isso empurra os desenvolvedores para trabalho reativo e repetitivo. Mostra como o pipeline quebrado se ligaria à interrupção do trabalho, ainda no registro de interpretação dos autores.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*

Contexto: O estudo é um survey online com 158 desenvolvedores (taxa de resposta de 14,4%) sorteados entre 1.100 pessoas que já haviam quebrado pelo menos um build em 666 projetos open source populares do GitHub configurados com o Travis CI; as respostas abertas foram codificadas qualitativamente pelos autores. O elo vem da pergunta aberta sobre problemas do uso de CI (Q15), cujo problema mais citado, por 31 respondentes, foi a dificuldade de configurar o ambiente de build. As citações mostram tanto o lado administrável do custo quanto os casos em que ele pesa: depurar uma falha que só acontece no servidor e conviver com testes instáveis. Os próprios autores ponderam, com base em outro estudo, que muitos projetos mudam a configuração de CI cinco vezes ou menos, de modo que esse esforço talvez não seja um fardo constante; as duas últimas citações vêm cada uma de um único respondente.

Mecanismo: usar CI exige configurar e manter o ambiente de build; quando o build falha no servidor e não localmente fica difícil depurar, e testes instáveis frustram. **Condições:** parte dos respondentes considera o custo de configuração administrável; pesa mais em testes distribuídos ou com múltiplos componentes. **Evidência:** empírica direta.

> Seção IV.B, síntese dos autores (31 respondentes): *"we found that 31 respondents are having a hard time configuring the build environment"*
>
> É a contagem feita pelos autores ao codificar as respostas sobre problemas da CI: 31 dos 158 respondentes disseram ter dificuldade para configurar o ambiente de build, o problema mais frequente da lista. Mostra que o custo de montar e manter a infraestrutura de build é a queixa principal de quem usa CI nesses projetos.

> Seção IV.B, respondente anônimo: *"Some extra overhead and complexity for setting them up / maintaining the configuration. Not usually a big problem"*
>
> Um respondente reconhece que montar e manter a configuração de CI traz sobrecarga e complexidade extras, mas diz que em geral não é um grande problema. A citação confirma o custo de manutenção do ambiente e, ao mesmo tempo, sustenta a condição do elo: para parte dos usuários o esforço é administrável.

> Seção IV.B, um respondente (ambientes múltiplos): *"If build fails it may be hard to debug it, especially if the problem doesn't occur in the local environment"*
>
> Um respondente aponta que rodar o build em ambientes diferentes do local, apresentado antes como benefício da CI, vira problema quando o build falha: a falha que não se reproduz na máquina do desenvolvedor é difícil de depurar. Mostra o mecanismo pelo qual a CI acrescenta trabalho de investigação sobre o próprio ambiente de build.

> Seção IV.B, um respondente (testes instáveis): *"Tests can sometimes be flaky, which is frustrating"*
>
> Um respondente cita os testes flaky, isto é, testes que ora passam ora falham sem mudança no código, como fonte de frustração no uso da CI. Indica que a manutenção do ambiente de build inclui lidar com testes não determinísticos, que geram falhas espúrias no pipeline.

↕ **ambivalente** Manutenção do ambiente de desenvolvimento (fator −)

Zhang et al. (2018) — *One size does not fit all: An empirical study of containerized continuous deployment workflows*

Contexto: No mesmo estudo do ESEC/FSE 2018, o survey com 168 donos de projetos de código aberto que publicam imagens Docker teve perguntas abertas sobre motivações, necessidades não atendidas (83 respostas válidas) e mudanças de workflow ao longo do tempo (71 respostas), seguidas de regressões de efeitos mistos sobre 855 projetos que comparam dois modos de montar o pipeline: o auto-build do Docker Hub (DHW, de Docker Hub workflow), em que o próprio registro constrói a imagem quando o código muda, e o workflow via serviço de CI (CIW, de CI workflow), em que Travis CI ou CircleCI constroem, testam e publicam a imagem. O elo é ambivalente porque as falas mostram os dois lados: uma minoria adota a prática para gastar menos tempo configurando ambientes, mas outros relatam builds de horas, configuração penosa e scripts necessários para o pipeline funcionar, e 45,8% já trocaram de workflow, sobretudo por dificuldade de configurar e manter. A regressão mostra que o CIW tem mais builds com erro que o DHW, e os autores leem isso como o custo de manutenção do pipeline mais testado. Como ressalvas, 89,9% dos que responderam sobre necessidades se dizem satisfeitos com o workflow atual, a motivação de economizar manutenção aparece em só 6,4% das respostas, e as regressões medem builds, não o tempo ou o esforço das pessoas.

Mecanismo: o pipeline automatiza a configuração de ambientes e libera tempo para desenvolver, mas ele próprio exige configuração difícil, builds lentos que travam o trabalho e ciclos de tentativa e rebuild quando quebra. **Condições:** depende do modo de adoção: o workflow via CI (mais testes, mais configurável) tem mais builds com erro e exige mais manutenção e experiência; o auto-build do Docker Hub é mais simples e de manutenção menor, porém com latência maior; a latência de build cresce com o tempo; vários respondentes trocaram de workflow por ser difícil de configurar e manter. **Evidência:** empírica direta.

> Seção 3.2, Tabela 1 (motivações para fazer CD, N=140): *"Lets us spend less time on maintenance and configuration 9 6.4%"*

Linha da tabela de motivações para adotar a implantação contínua: 9 das 140 respostas (6,4%) citam gastar menos tempo com manutenção e configuração. Mostra o lado em que a prática reduz o trabalho de manter o ambiente, mas é uma motivação pouco frequente na amostra.

Seção 3.2, [M7], respondente R64: *"to optimize time, focusing on developing the actual application"*

O respondente R64 ilustra a motivação M7 (gastar menos tempo com manutenção e configuração): usa a prática para aproveitar melhor o tempo e se concentrar em desenvolver a aplicação. Os autores ligam essa motivação à configuração automática de ambientes pelo pipeline, que tira do desenvolvedor o trabalho de montar e consertar ambientes de teste.

Seção 3.4, [N1], respondente R4: *"one dockerfile takes more than 2 hours to build and timeouts"*

Na pergunta sobre necessidades não atendidas, o respondente R4 relata que, no pipeline dele, a construção de um Dockerfile (o arquivo que define a imagem) passa de duas horas e estoura o tempo limite. É o lado negativo do elo: o próprio pipeline vira fonte de espera e de falhas que o desenvolvedor precisa contornar; a necessidade de builds mais rápidos foi a mais citada (21,7%).

Seção 3.4, [N1], comentário dos autores: *"The CD processing speed will affect the software release and developers' work efficiency."*

Os autores comentam a fala de R4 e generalizam: a velocidade do pipeline afeta tanto a entrega quanto a eficiência do trabalho dos desenvolvedores. É a interpretação dos autores que conecta a lentidão do build ao trabalho das pessoas; eles acrescentam que a latência tende a crescer à medida que se somam testes e artefatos.

Seção 3.4, [N2], respondente R109: *"sometimes, circleCI config and setup is pain. Docs sometimes doesn't help"*

O respondente R109, que usa o CircleCI (um serviço de CI) para construir e publicar imagens, diz que configurar o serviço às vezes é penoso e a documentação nem sempre ajuda. Ilustra a necessidade N2, de ferramentas mais fáceis de aprender e configurar (16,9% das respostas), e mostra o trabalho de configuração que o pipeline cria.

Seção 3.5, [B1], respondente R119: *"the old CD pipeline is a little harder to setup. It was necessary to write several scripts to get everything working properly. The new CD pipeline is easier to setup and maintain"*

Na pergunta sobre mudanças de workflow, o respondente R119 explica por que trocou de pipeline: o antigo exigia escrever vários scripts para funcionar, e o novo é mais fácil de configurar e manter. A dificuldade de configurar e manter foi o motivo de troca mais citado (35,2%); a fala mostra que a manutenção depende do modo de adoção, que é a condição do elo.

Seção 5.2, trade-off confiabilidade (CIW) × manutenção (DHW): *"More testing means higher reliability, but also more errored builds, i.e., more maintenance."*

Na discussão, os autores resumem um dos trade-offs entre os workflows, apoiados no survey e no modelo de regressão de builds com erro (o CIW teve significativamente mais builds com erro que o DHW). O pipeline via CI, com mais testes, é mais confiável, mas gera mais builds com erro para corrigir, ou seja, mais manutenção; é a interpretação dos autores que dá a condição da ambivalência.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Parry et al. (2022) — *Surveying the Developer Experience of Flaky Tests*

Contexto: Mesmo survey com 170 respondentes e análise de 38 threads do StackOverflow sobre testes flaky. Aqui o elo vem de duas fontes: a afirmação fechada mais endossada entre os impactos (testes flaky atrapalham a CI) e as respostas abertas sobre outras causas de instabilidade (SQ6) e comentários livres (SQ11), em que vários participantes situam o problema no ambiente de CI. As respostas abertas foram codificadas em temas por análise temática indutiva, feita em conjunto pelos quatro autores; um desses temas, diferenças de ambiente entre a máquina local e a máquina remota de build, trata quase só de CI. Ressalvas: o artigo não informa quantas respostas compõem cada tema, e o sentido do elo, de que a CI gera trabalho de manutenção, vem de percepção e de falas individuais, sem medida do tempo gasto.
Mecanismo: diferenças entre a máquina local e o ambiente remoto de CI, e dependências externas baixadas no build, fazem testes falharem só na CI, o que leva a reexecutar builds e investigar falhas que não são do código. **Condições:** quando o ambiente de CI difere do local (padrões da JVM, latência de sistema de arquivos, concorrência) ou depende de serviços e bibliotecas externos. **Evidência:** empírica direta.

RQ2, SQ4: *"developers strongly agree with the notion that flaky tests hinder CI"*

Resultado da afirmação SQ4.5, que teve a maior concordância entre os oito impactos propostos (média 2,63 numa escala de 0 a 3), com a mesma nota entre quem vê testes flaky com frequência e quem os vê raramente. Mostra que o atrito se concentra na CI: é no pipeline compartilhado que a falha falsa bloqueia o fluxo. Sustenta que a CI é onde recai o custo dos testes instáveis.

RQ3, SQ6t2 (P21): *"environmental differences in local vs CI like different Java Virtual Machine (JVM) defaults."*

Resposta do participante P21 à pergunta aberta SQ6, sobre causas de testes flaky não listadas no questionário. Ele aponta diferenças entre o ambiente local e o de CI, como padrões diferentes da máquina virtual Java (JVM), como origem da instabilidade. Ilustra o mecanismo do elo: o teste passa na máquina do desenvolvedor e falha só no ambiente de CI, por configuração do ambiente e não por erro no código.

RQ3, SQ6t2: *"Almost all the responses in this theme made reference to CI."*

Observação dos autores sobre o tema SQ6t2 (diferenças entre as máquinas locais de desenvolvimento e as máquinas remotas de build): quase todas as respostas desse tema mencionavam a CI. Mostra que o caso de P21 não é isolado; o artigo cita também P97, que atribui o comportamento diferente na CI a latência do sistema de arquivos e a concorrência. Não informa quantas respostas compõem o tema.

Tabela 1, SQ11t7 (P31): *"Recently, seen a lot of flaky tests when running CI on Azure due to failures to download libraries"*

Fala de P31 na pergunta aberta final, exemplo do tema SQ11t7 (serviços externos): ele tem visto muitos testes flaky ao rodar a CI no Azure por falhas ao baixar bibliotecas. Mostra a segunda condição do elo: o build de CI que depende de serviços e bibliotecas externos falha por motivos alheios ao código, e cabe ao desenvolvedor lidar com essas falhas.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Valenzuela-Toledo et al. (2024) *— The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance*

Contexto: O estudo minera o histórico de commits de 183 projetos de código aberto grandes e maduros do GitHub, em dez linguagens, todos com GitHub Actions (GA), a plataforma do GitHub em que arquivos de workflow em YAML descrevem o build, os testes e o deploy executados automaticamente a cada push ou pull request. Os autores partem da hipótese de que essa automação exige intervenção humana para corrigir defeitos, atualizar dependências e reorganizar os workflows, e medem, desde o primeiro commit de GA até o fim de 2023, quanto os arquivos de workflow mudam por mês, com que frequência mudam junto com código de produção e de teste, por que motivo (codificação aberta e card sorting de 200 commits) e quem faz essas mudanças. O elo vem da leitura dos autores sobre esses dados: os workflows mudam menos que o código de produção e de teste, mas mudam todo mês, sobretudo para melhorar o próprio CI/CD e corrigir bugs, e só cerca de 5% dos desenvolvedores mexem neles. Nenhuma pessoa foi ouvida ou medida diretamente, não há comparação com projetos sem a automação, e os próprios autores reconhecem que contar mudanças é só um indicador aproximado do esforço de manutenção, já que uma mudança pequena pode exigir muito esforço cognitivo.
Mecanismo: os arquivos de workflow que automatizam build, teste e deploy não ficam estáveis: exigem mudança manual recorrente para consertar defeitos, atualizar dependências e melhorar o próprio CI/CD, inclusive quando uma alteração de infraestrutura faz todos os pull requests falharem. **Condições:** projetos open source grandes e maduros no GitHub Actions; o esforço é medido por commits, que os autores admitem ser proxy imperfeito (mudança pequena pode custar muito esforço cognitivo); a manutenção recai sobre poucos desenvolvedores (cerca de 5%), que também mexem em código de produção e teste. **Evidência:** empírica indireta.

Seção IV-B (RQ2, taxa de mudança), Finding 2: *"In the multi-workflow group, a mean of 8.2% of the workflow files undergo at least one monthly change"*

É a medida da taxa de mudança (churn) dos workflows: nos 156 projetos com mais de um arquivo de workflow, em média 8,2% desses arquivos são alterados a cada mês (6,9% nos projetos com um só workflow). O mesmo resultado mostra que o código de produção muda muito mais (74,12%), então a citação sustenta que os workflows exigem mudança contínua, não que exigem mais trabalho que o resto do código.

Seção IV-D (RQ4), categoria 1.b, Dependency and Build Management: *"involves reverting a change to Debian 11 (bullseye) that caused all pull requests to fail"*

É um dos commits usados pelos autores para ilustrar a categoria de gestão de dependências e de build, na taxonomia dos motivos pelos quais workflows mudam junto com outros arquivos: a troca da imagem base dos contêineres para o Debian 11 fez todos os pull requests falharem e teve de ser desfeita. O exemplo mostra o tipo de manutenção manual que a infraestrutura do pipeline gera e o impacto de um erro nela sobre todo o projeto; é um caso ilustrativo, de uma categoria que responde por 6% a 7% dos commits analisados.

Seção V (a), discussão de RQ1 e RQ2: *"Nonetheless, they are far from being stable, which confirms our hypothesis that manual effort is needed for maintaining workflow files over time"*

Na discussão, os autores interpretam a taxa de mudança: embora os workflows mudem menos que o código de produção e de teste, continuam mudando ao longo do tempo, o que, para eles, confirma que mantê-los exige esforço manual. É a afirmação central do elo, uma inferência a partir das contagens de commits, não uma medida do esforço das pessoas.

Seção V (b), discussão de RQ3 e RQ4: *"thorough investigation of commits that contribute to the logical coupling reveals that they are primarily updated for CI/CD improvements and bug fixes"*

Os autores resumem a análise qualitativa dos 200 commits em que workflows mudaram junto com código de produção ou de teste: a maior parte das mudanças serve para melhorar o próprio pipeline de CI/CD (61% dos casos com teste e 33%

dos casos com produção) e para corrigir bugs e problemas de compatibilidade (12% e 18%). A citação indica de que é feito o trabalho de manutenção que o pipeline demanda.

Seção VIII (conclusão): *"However, our research reveals that implementing and maintaining these workflow files is not without additional costs"*

Na conclusão, os autores contrapõem os ganhos da automação com GA ao custo que encontraram: implementar e manter os workflows traz custos adicionais, que eles chamam de custos escondidos da automação e que, segundo eles, exigem planejar tempo e pessoas para essa manutenção. É a síntese que dá o sentido do elo, de que o pipeline aumenta o trabalho de manter o ambiente.

↑ **aumenta** *desfecho:* sentimento negativo expresso nas mensagens de commit

Souza e Silva (2017) *— Sentiment Analysis of Travis CI Builds*

Contexto: O estudo é um artigo curto do desafio de mineração do MSR 2017 que usa o TravisTorrent, base pública com os builds de 1.262 projetos de código aberto em Java e Ruby hospedados no GitHub e integrados pelo Travis CI, um serviço de integração contínua que constrói e testa o código a cada envio; ao todo, 609 mil builds e cerca de 1 milhão de commits, com 26,5% de builds quebrados (código que não compila ou não passa nos testes automatizados). O sentimento das mensagens de commit foi medido automaticamente com o SentiStrength, que dá a cada texto uma nota positiva (de 1 a 5) e uma negativa (de -1 a -5) a partir de um dicionário de palavras; os autores retiraram do dicionário termos técnicos como bug e failure, que inflavam o negativo. Para este elo, a pergunta foi se os commits do build seguinte a um build quebrado são mais negativos que os que vêm depois de um build bem-sucedido, testada com Mann-Whitney e tamanho de efeito delta de Cliff. A diferença é estatisticamente significativa, mas o efeito é muito pequeno (delta de -0,014), e os autores avisam que a imprecisão da ferramenta, que não reconhece sarcasmo e cujo dicionário ajustado não é completo, pode fazer o efeito real ser ainda menor. O sentimento expresso em mensagem de commit é um indicador indireto do que a pessoa sente, e as mensagens citadas são exemplos escolhidos pelos autores.
Mecanismo: o build quebrado no servidor de CI desperta frustração que aparece no commit seguinte, e quebras repetidas antes de conseguir consertar levam o desenvolvedor a odiar o bug, num ciclo de realimentação entre sentimento e quebra.
Condições: commit imediatamente após um build quebrado (Travis CI, projetos de código aberto); efeito estatisticamente significativo mas muito pequeno, medido por análise de sentimento automática sujeita a erro e sarcasmo. **Evidência:** empírica direta.

Seção III.B (RQ2, Figura 2): *"The proportion of builds with more extreme negative scores is higher after broken builds"*

É o resultado da comparação mostrada na Figura 2: entre os builds que vêm logo depois de um build quebrado, a proporção com notas negativas mais extremas no SentiStrength é maior do que entre os que vêm depois de um build bem-sucedido. A citação sustenta o sentido do elo, o build quebrado associado a mais sentimento negativo no commit seguinte.

Seção III.B (RQ2): *"although the effect size is small (Cliff's delta = −0.014)"*

Logo depois do resultado, os autores informam o tamanho do efeito: delta de Cliff de -0,014, muito abaixo do limiar que eles mesmos usam para efeito pequeno (0,33). A citação marca a principal ressalva do elo: a diferença existe estatisticamente, numa amostra enorme, mas é quase desprezível na prática.

Seção III.B (síntese da RQ2): *"Commits following a build breakage tend to be more negative and less positive, although the effect size is small."*

É a frase de síntese da RQ2, que junta o resultado do sentimento negativo ao do positivo (Figura 3, com delta de -0,009): depois de um build quebrado, os commits ficam um pouco mais negativos e um pouco menos positivos. Os autores repetem a ressalva do efeito pequeno.

Seção III.B, Tabela III, commit (e): *"multiple builds failed before the developer managed to fix bug #320, causing him to hate the bug"*

Os autores comentam um dos exemplos da Tabela III, de commits negativos feitos depois de builds quebrados: a mensagem diz que o desenvolvedor está odiando o bug #320, e eles explicam que vários builds falharam antes que ele conseguisse corrigi-lo. É o exemplo que ilustra o mecanismo do elo, a frustração acumulada por quebras repetidas aparecendo no texto do commit; é um caso escolhido, não uma medida.

Seção III.D (discussão): *"We also found that broken builds are more likely to be followed by negative sentiment, creating a feedback loop."*

Na discussão, os autores juntam as duas direções que encontraram: commits negativos quebram o build um pouco mais (RQ1) e builds quebrados são seguidos de commits um pouco mais negativos (RQ2). Daí a ideia de um ciclo de realimentação entre sentimento e quebra, que é interpretação dos autores; logo em seguida eles dizem que os dois efeitos pequenos sugerem papel menor do sentimento no sucesso do build.

Seção III.D (ameaças à validade): *"the actual effect sizes may be even smaller than reported"*

Os autores apontam a precisão do SentiStrength como a principal ameaça à validade: nas análises preliminares, com o dicionário original, o efeito parecia bem maior, e a lista de termos técnicos removidos não é completa. A citação diz que o efeito real pode ser ainda menor que o relatado, o que enfraquece o elo.

↑ **aumenta** *desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração)
Kassab et al. (2026) *— The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes*
Contexto: Estudo de mineração de repositórios (FSE Companion 2026) com 73.125 comentários de 7.410 issues do Jira de três projetos Apache de código aberto (Hadoop, Kafka e Flink), selecionados por um léxico de palavras-chave de dívida técnica. Os autores agruparam os comentários em 7 temas por LDA (modelagem de tópicos) e classificaram a emoção de cada comentário com um modelo RoBERTa pré-treinado, calculando o Negative Emotion Index (NEI), a fração da probabilidade atribuída a raiva, nojo, medo e tristeza. O elo vem da comparação entre temas: o Tópico 4, de dívida de testes e de infraestrutura de CI/CD, tem NEI de 15,6%, acima da média de 14,4% e dos temas de API/SDK (12,4%) e de código (12,9%), dos quais difere significativamente. Não há antes/depois nem comparação com projetos sem CI: o que se compara é a dívida acumulada no pipeline com outros tipos de dívida. Ressalvas fortes: o efeito global é pequeno (Cliff's δ = 0,06), o classificador mede o tom expresso no texto e não o estado afetivo das pessoas, e os autores declaram não afirmar relação causal.
Mecanismo: a dívida acumulada no pipeline e na suíte de testes (testes flaky, builds instáveis, falhas dependentes de plataforma) mina a confiança na automação, e as discussões sobre ela carregam mais emoção negativa que as de dívida de API ou de implementação. **Condições:** pipeline e suíte de testes com dívida (testes flaky, build instável); comparação relativa a outros temas de dívida, não a projetos sem CI; efeito estatisticamente significativo mas pequeno, sem pretensão causal; três projetos Apache de código aberto, emoção inferida por classificador a partir do texto. **Evidência:** empírica indireta.

Seção 4.1: *"Topic 4 focuses on testing and CI/CD infrastructure debt"*
Descrição, pelos autores, de um dos 7 temas latentes que o LDA extraiu dos comentários. O Tópico 4 reúne a dívida de testes e de infraestrutura de CI/CD: 5.917 comentários de 721 issues (Tabela 1), com palavras como test, fail, build, flaky e junit. A citação identifica qual parte dos dados corresponde à prática do elo.

Seção 4.1: *"flaky tests, unstable build pipelines, and platform-dependent failures that undermine trust in automation"*
Continuação da descrição do Tópico 4: o tema é dominado por testes flaky (que passam ou falham sem mudança no código), pipelines de build instáveis e falhas dependentes da plataforma, que minam a confiança na automação. Mostra que o tema não trata da CI em geral, e sim da dívida acumulada nela, que é a condição do elo. O rótulo é interpretação dos autores a partir das palavras mais frequentes do tópico.

Seção 4.3: *"Topic 4 (Testing and CI/CD) also shows elevated NEI"*
Resultado da RQ2, sobre o tamanho das diferenças: depois do Tópico 3 (observabilidade e conectividade, o mais negativo, com 18,6%), o tema de testes e CI/CD também tem NEI elevado, 15,6%, e difere significativamente ($p < 0,001$) dos Tópicos 2 (API/SDK) e 6 (código), que ficam em 12–13%. É o dado que sustenta o sentido do elo, mas a diferença é de cerca de três pontos percentuais e o próprio artigo pede interpretação cautelosa.

Seção 4.3: *"Conversely, Topics 3 and 4 lean neutral-to-negative (EBI ≤ 0), consistent with frustration associated with monitoring failures and unstable automation"*
Segunda métrica: o EBI (Emotional Balance Index, emoções positivas menos negativas, numa escala de −1 a +1) fica em zero ou abaixo nos Tópicos 3 e 4 (−0,032 e −0,005), enquanto os temas de API e de código ficam positivos. Os autores leem isso como frustração associada a falhas de monitoramento e a automação instável. Reforça a direção do elo por uma medida diferente do NEI, mas a leitura como frustração é interpretação dos autores.

Entrega contínua

9 elos em 3 artigo(s)

> **O que é.** O software é mantido o tempo todo em condição de ser lançado: qualquer versão que passou pelo pipeline pode ir para os clientes com um clique. A decisão de lançar passa a ser de negócio, não uma operação técnica de risco. Descrição completa na seção 1.

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*

Contexto: O artigo é um relato de workshop baseado em três entrevistas semiestruturadas, de cerca de duas horas cada, com os projetistas líderes de três projetos web pequenos (4 a 7 pessoas) da Solita, empresa finlandesa de software. Os três projetos adotaram a entrega contínua desde o início, e não a partir de um processo tradicional, de modo que não há medição de antes e depois: a comparação vem da experiência anterior dos entrevistados com outras formas de trabalho. Em todos, cada mudança dispara automaticamente build, testes e implantação no ambiente de teste, e a implantação em produção é feita em um passo e seria possível todos os dias. O elo resulta da síntese dos autores sobre as entrevistas (feitas em finlandês, traduzidas por eles e sem transcrição completa), e não de falas literais; os próprios autores advertem que três pessoas abertas a mudanças são pouco para generalizar e que pessoas mais conservadoras poderiam sentir mais estresse com a mudança.

Mecanismo: cada mudança é construída, testada e implantada automaticamente no ambiente de teste, o que dá confiança de que vai funcionar em produção; e como corrigir e implantar a correção é simples e imediato, bugs e implantações deixam de gerar estresse. **Condições:** relatado nos três projetos, todos com pipeline automatizado e implantação diária possível; os autores ressalvam que as implantações precisam ser também frequentes (ver o registro de Releases frequentes) e que pessoas mais conservadoras poderiam sentir mais estresse com a mudança. **Evidência:** empírica direta.

> Seção 4.1 (os três projetos): *“Because the software was built, tested, and deployed to test environment automatically after each change”*
>
> É a descrição, pelos autores, de como o pipeline funcionava nos três projetos: build, testes e implantação no ambiente de teste disparados automaticamente a cada mudança. A frase abre a explicação causal do elo, porque é dessa automação que vem a confiança descrita na citação seguinte.

> Seção 4.1 (os três projetos): *“team had high trust for the application to work expectedly in production”*
>
> Continuação da mesma frase: por causa da automação, o time tinha grande confiança de que a aplicação funcionaria como esperado em produção. É o lado psicológico do elo: a confiança no software, construída antes de ele chegar à produção, é o oposto do receio de implantar.

> Seção 4.1 (os três projetos): *“they even felt that the shelter brought by the tests was so precious they would like to spend even more time writing tests”*
>
> Os autores relatam que os programadores não achavam penoso escrever testes e chegavam a sentir que a proteção (shelter) dada por eles era tão valiosa que gostariam de dedicar ainda mais tempo a escrevê-los. Para o elo, mostra que a segurança percebida vem em boa parte dos testes automatizados do pipeline, que funcionam como rede de proteção contra falhas em produção.

> Seção 4.2 (os três projetos): *“In all cases, the developers did not feel any stress due to bugs, because fixing and deploying the fixes to production was so straightforward.”*
>
> Trecho da seção sobre a prontidão para implantar em produção: dois entrevistados disseram que bugs raramente chegavam à produção e um disse que eram bem comuns, mas em nenhum dos três projetos os desenvolvedores sentiam estresse por causa deles. A razão apontada é que corrigir e implantar a correção era simples; é o segundo mecanismo do elo, que não depende de não haver erros, e sim de poder consertá-los depressa.

> Seção 5 (discussão): *“feel stress over deployments or bugs that would appear in the production, because they were always able to deploy fixes to production – immediately, if needed.”*
>
> Na discussão, os autores resumem o achado: os desenvolvedores não sentiam estresse com implantações nem com bugs que aparecessem em produção, porque sempre podiam implantar correções, imediatamente se fosse preciso. A citação junta implantações e bugs na mesma ausência de estresse e é a formulação mais direta do elo, ainda que seja a interpretação dos autores sobre as três entrevistas.

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Limoncelli (2018) *— SQL is no excuse to avoid DevOps*

Contexto: A coluna de Thomas Limoncelli no ACM Queue é opinião de um gerente de SRE, sem método nem dados. O amigo do autor, que trabalha numa empresa com banco SQL e implantações manuais algumas vezes por ano, diz que cada implantação arrisca os dados da empresa e o próprio emprego e que esse processo é a maior fonte de estresse do lugar, a ponto de achar irresponsável implantar com mais frequência. O autor argumenta, retomando o princípio dos lotes pequenos, que fazer a tarefa arriscada com mais frequência força a automação e o teste do processo e dá prática às pessoas, e descreve as duas técnicas (esquema atualizado pela aplicação e código compatível com vários esquemas). O elo com a ansiedade de implantação junta o estado anterior relatado pelo amigo à conclusão do autor de que a automação transforma a implantação em evento rotineiro; não há caso observado depois da adoção, e o próprio autor ressalva que aumentar a frequência sem baixar o custo fixo da implantação seria prejudicial.

Mecanismo: a implantação manual e rara é vivida como risco pessoal (dados da empresa e o próprio emprego); automatizada e testada muitas vezes antes de produção, ela vira evento rotineiro e sem incidente. **Condições:** a frequência maior só ajuda se o processo for automatizado e testado; com o processo manual, mais implantações significariam mais estresse. **Evidência:** inferida.

> Relato do amigo (meio da coluna): *“these deployments are risky. Every time we do one I risk the company’s data and, to be honest, my job.”*
>
> O amigo do autor descreve como vive a implantação manual atual: cada uma põe em risco os dados da empresa e o emprego dele. É o estado anterior à prática e mostra o conteúdo do medo, que é pessoal e não só técnico.

> Relato do amigo (início da coluna): *“Doing this every few months is painful and the number-one source of stress around here.”*
>
> No mesmo relato, o amigo diz que a implantação feita a cada poucos meses é dolorosa e a principal fonte de estresse no trabalho dele. A citação dá a intensidade do problema antes da automação, sempre na percepção de uma única pessoa.

> Summary: *“Automating releases liberates us. It turns a worrisome, stressful, manual upgrade process into a regular event that happens without incident.”*
>
> Na conclusão, o autor afirma o efeito da prática: com a liberação automatizada, a atualização deixa de ser um processo manual preocupante e estressante e vira evento regular, sem incidentes. É a tese do autor, apoiada na experiência dele como SRE, e descreve o mecanismo de redução da ansiedade.

> Summary: *“It reduces business risk but, more importantly, creates a more sustainable workplace.”*
>
> A frase final do resumo põe o ganho para as pessoas acima do ganho para o negócio: a automação cria um local de trabalho mais sustentável. Reforça que o autor vê o efeito sobre quem implanta como o principal, embora sem dado que o meça.

↕ **ambivalente** Carga de trabalho (fator −)

Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*

Contexto: Mesmo estudo de caso na Solita Oy. O aumento de carga vem dos desenvolvedores, que relatam que montar as ferramentas e práticas novas custou esforço grande; mesmo assim, todos disseram que a mudança valeu a pena. A redução citada vem da seção sobre ganhos do cliente: antes, a equipe do cliente preparava, avisava os usuários, redirecionava requisições e acompanhava implantações que podiam durar a noite inteira, e a automação melhorou isso. As implantações em produção continuam fora do horário de expediente e com breve janela de manutenção. As citações são sínteses dos autores, sem falas literais nem medida de horas.

Mecanismo: durante a adoção, montar ferramentas e práticas novas exigiu esforço grande, às vezes acima do previsto, por ter de aprender ferramentas desconhecidas; depois, a automação de implantação encurtou implantações que antes podiam tomar a noite inteira da equipe do cliente. **Condições:** aumenta na transição (aprendizado das ferramentas) e reduz depois que a automação está pronta, com menos tarefas repetitivas de operação para os desenvolvedores; as implantações em produção continuam fora do expediente e ainda exigem janela de manutenção. **Evidência:** empírica direta.

> Seção 3.1.1: *“establishing the new tools and practices required substantial effort, sometimes more than expected, due to learning unfamiliar tools and techniques”*
>
> Síntese dos autores sobre o lado do fornecedor: estabelecer as ferramentas e práticas novas exigiu esforço substancial, às vezes maior que o previsto, porque era preciso aprender ferramentas e técnicas desconhecidas. É o aumento de carga durante a transição, vivido pelos desenvolvedores.

> Seção 3.1.1: *“The increasing level of automation decreased the developer effort needed for repetitive, menial tasks related to operations”*
>
> Primeira frase da seção sobre os ganhos de produtividade do fornecedor, logo antes de os autores registrarem o esforço da transição. É o lado “reduz” do elo para quem produz o software: a automação do pipeline tirou dos desenvolvedores

tarefas repetitivas de operação, que antes eram feitas à mão.

Seção 3.8 (síntese dos benefícios): *"the developers spend less time on non value adding tasks"*
Na síntese dos benefícios, os autores dizem que quase todos os entrevistados perceberam ganho de produtividade e que os desenvolvedores passam menos tempo em tarefas que não agregam valor. Confirma, para os desenvolvedores, a redução de carga depois que a automação ficou pronta.

Seção 3.1.1: *"have to take place outside of office hours"*
Na seção sobre o fornecedor, os autores dizem que os desenvolvedores já podem implantar correções no mesmo dia e que a única restrição que resta é as implantações terem de ocorrer fora do horário de expediente. É a condição que limita a redução: o trabalho fora de hora continua, embora o artigo não diga quem executa a implantação em produção.

↓ **reduz** Conflito trabalho-família (fator −)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Contexto: Mesmo estudo de três entrevistas na Solita. Nos três projetos, cada um com um único pacote implantável e implantação em produção em um passo (incluindo migrações de banco de dados), as implantações para QA e produção eram roteirizadas e disparadas manualmente, e um time chegou a programá-las para rodar sozinhas à noite, decidindo durante o dia o que seria implantado. O elo é indireto: o artigo não mede conflito entre trabalho e família, e sim relata que as implantações aconteciam no expediente, sem hora extra nem janelas de manutenção à noite ou no fim de semana, o que remove uma fonte de invasão do tempo pessoal. Como os projetos adotaram a entrega contínua desde o início, não há situação anterior descrita para comparar; os autores contrastam apenas com a visão tradicional de implantações raras e penosas e com os 'scary Saturdays' (sábados assustadores) da empresa de jogos estudada por Neely e Stolt, que dizem não existir na Solita.
Mecanismo: com implantação automatizada e frequente, implantar vira rotina que não exige ninguém acompanhando nem acordado; não há janela de manutenção à noite ou no fim de semana, e as implantações cabem no expediente sem hora extra. **Condições:** projetos pequenos com um único pacote implantável e implantação em produção em um passo; um time chegou a agendar a implantação automática para a noite, decidida durante o dia. **Evidência:** empírica indireta.

Seção 4.1 (os três projetos): *"When deliveries are frequent and automated, the deployment becomes a normal process which does not require anyone to be monitoring or even awake at the time."*
Logo depois de contar que as implantações em produção eram tão estáveis que um time as automatizou para ocorrer à noite, os autores generalizam: com entregas frequentes e automatizadas, implantar vira um processo normal que não exige ninguém monitorando, nem mesmo acordado, naquele momento. Para o elo, mostra que a automação desobriga o desenvolvedor de estar presente fora do horário, mesmo quando a implantação acontece à noite.

Seção 4.1 (os três projetos): *"Deployments were done during standard working hours and required no overtime or particular scheduling."*
Conclusão do mesmo parágrafo, depois de os autores dizerem que nenhum projeto viu necessidade de longas paradas de manutenção nem de marcar implantações para noites ou fins de semana. É a observação concreta que sustenta o elo: as implantações cabiam no horário normal de trabalho e não exigiam hora extra nem agendamento especial.

Seção 5 (discussão): *"They did not work overtime"*
Na discussão, os autores atribuem à melhora de qualidade e estabilidade trazida pela entrega contínua o fato de os desenvolvedores não fazerem hora extra (nem sentirem estresse com implantações). Reforça o elo pela ausência de hora extra, mas é uma síntese dos autores, sem número de horas nem comparação com outro período.

↓ **reduz** Conflito trabalho-família (fator −)
Limoncelli (2018) *— SQL is no excuse to avoid DevOps*
Contexto: O texto é uma coluna de opinião publicada no ACM Queue por Thomas Limoncelli, gerente de SRE (engenharia de confiabilidade) da Stack Overflow; não há método, amostra nem dados coletados. O ponto de partida é a conversa com um amigo, não identificado, que diz não poder adotar DevOps porque a empresa usa banco SQL: a cada poucos meses a aplicação é desligada, os DBAs (administradores de banco de dados) alteram o esquema à mão e a nova versão é instalada, num processo de várias horas marcado depois de semanas de negociação entre times e que acaba caindo no fim de semana. O autor responde com duas técnicas, a atualização automática de esquema gerida pela própria aplicação (Técnica 1) e o código que funciona com mais de um esquema ao mesmo tempo, conhecido como expand/contract (Técnica 2), que dispensam a parada da aplicação. O elo com o conflito trabalho-família vem da justaposição entre o relato do amigo sobre os fins de semana perdidos e a afirmação do autor de que, com as técnicas, os DBAs recuperam os fins de semana. É uma previsão do autor baseada na experiência profissional, sem nenhum caso observado depois da adoção, e o relato do amigo é de uma única pessoa.
Mecanismo: com mudança de esquema automatizada e código compatível com vários esquemas, a implantação deixa de exigir parada da aplicação e janela de manutenção negociada entre times, que acabava caindo no fim de semana; o DBA

deixa de participar pessoalmente de cada atualização. **Condições:** depende de automatizar o esquema e de reduzir o custo fixo da implantação; sem isso, o amigo do autor prevê que releases semanais acabariam com os fins de semana e o autor chama de irresponsável aumentar a frequência sem baixar esse custo. **Evidência:** inferida.

> Relato do amigo (início da coluna): *"The process takes many hours, so we tend to do it on the weekend... which I hate."*
> O amigo do autor descreve o processo de implantação manual atual, sem a prática: parar a aplicação, mudar o esquema e instalar a nova versão leva horas. Por isso o trabalho é feito no fim de semana, e ele diz detestar isso; é o estado anterior à prática que mostra a invasão do tempo pessoal.

> Relato do amigo (início da coluna): *"Just scheduling such an event takes weeks of negotiation. We usually lose the negotiation, which is why we end up doing it on the weekend."*
> Ainda no relato do amigo, ele explica por que a implantação cai no fim de semana: a janela de parada precisa ser negociada durante semanas com outros times, e o time dele costuma perder essa negociação. A citação mostra a causa organizacional do problema, a janela de manutenção, que é justamente o que a Técnica 2 elimina.

> Technique 1: Automated Schema Updates: *"This should bring new confidence to the process, reduce the risk of schema upgrades, and decouple the DBAs' personal involvement in upgrades. They will appreciate getting their weekends back."*
> É o autor falando, ao fim da descrição da Técnica 1, em que a aplicação aplica sozinha as mudanças de esquema e esse processo é testado centenas de vezes no ambiente de integração contínua antes de chegar à produção. Ele afirma que os DBAs deixam de participar pessoalmente de cada atualização e recuperam os fins de semana; é a previsão do efeito da prática, não uma observação.

> Technique 2: Coding for Multiple Schemas: *"While in the past upgrades required finding a maintenance window compatible with multiple teams, now the process is decoupled and all parties can work in a coordinated way but not in lockstep."*
> O autor descreve o efeito da Técnica 2: como o código passa a funcionar com o esquema antigo e o novo, o campo novo pode ser criado antes e usado depois, sem parar tudo ao mesmo tempo. Com isso some a necessidade de uma janela de manutenção comum a vários times, que no relato do amigo era o que empurrava o trabalho para o fim de semana.

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)
Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*
Contexto: Mesmo estudo de caso na Solita Oy (entrevistas com três pessoas do fornecedor e duas do cliente). Em 2010 quase não havia automação de teste, integração ou implantação: a cada release os artefatos eram gerados para um ambiente específico e levados ao cliente num pendrive. Nos cinco anos seguintes vieram servidor de integração contínua (2011), scripts de implantação, builds independentes de ambiente, configuração automatizada dos servidores com Ansible e Vagrant (2014) e implantação automática no ambiente de teste do cliente. A melhora do moral dos desenvolvedores foi reconhecida só pelos desenvolvedores, não pelo cliente (Tabela 3). O elo vem das sínteses dos autores sobre as entrevistas com três desenvolvedores; é percepção, sem medida.
Mecanismo: a automação de implantação e de configuração tira do desenvolvedor tarefas manuais repetitivas, e o retorno sobre o que foi implantado chega bem mais rápido; o artigo liga as duas coisas ao aumento de satisfação (aqui 'aumenta' quer dizer melhora do desenho: menos tarefa mecânica, mais retorno do resultado). **Condições:** melhorias propostas de baixo para cima por desenvolvedores incomodados com tarefas manuais e decididas coletivamente; parte da automação é de configuração de servidores (infraestrutura como código), que o artigo não separa da entrega contínua. **Evidência:** empírica direta.

> Seção 3.1.1: *"Not only has the automation of tasks decreased the amount of repetitive work, such as manual"*
> Síntese dos autores sobre os ganhos de produtividade do fornecedor: a automação diminuiu o trabalho repetitivo, como implantações e configurações manuais (a frase continua no texto, 'manual deployments and configuration'). Mostra a mudança no conteúdo do trabalho, com menos tarefa mecânica.

> Seção 3.7: *"of repetitive, menial tasks. Interviewees reported that many of the ideas that led to changes simply stemmed from individual developers being annoyed with having to perform some task manually"*
> Na seção sobre o moral, os autores falam da redução de tarefas repetitivas e braçais ('menial') e relatam que muitas das mudanças nasceram de desenvolvedores incomodados por ter de fazer algo à mão, decididas em geral coletivamente entre eles. Mostra o estado anterior que incomodava e a condição de adoção de baixo para cima.

> Seção 3.7: *"These kinds of bottom-up changes have resulted in increased developer satisfaction"*
> Conclusão dos autores no mesmo parágrafo: essas mudanças de baixo para cima resultaram em maior satisfação dos desenvolvedores. É o que liga a remoção das tarefas manuais ao desfecho satisfação.

> Seção 3.2.1: *"the feedback on deployed changes, which now arrives notably faster than five years ago"*

Na seção sobre colaboração do lado do fornecedor, os autores registram que o retorno sobre as mudanças implantadas chega bem mais rápido do que cinco anos antes. Cobre a outra parte do elo, o retorno sobre o resultado do próprio trabalho, que é uma das dimensões do desenho do trabalho.

↓ **reduz** Interação com clientes (fator −, nome neutro*)

Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*

Contexto: Estudo de caso exploratório (ESEM 2016) na Solita Oy, fornecedora finlandesa de serviços digitais, numa unidade de 13 desenvolvedores que mantém 16 aplicações para um único cliente do setor público; os autores entrevistaram dois desenvolvedores e o desenvolvedor líder do fornecedor e, do cliente, um arquiteto de TI e o usuário-chave de um sistema, e fizeram análise temática sobre a transição de cinco anos (2010 a 2015). No início, o cliente escrevia especificações pesadas, os desenvolvedores implementavam isolados e muita funcionalidade era refeita quando se descobria que não atendia à necessidade. Ao longo da transição vieram, juntos, práticas de colaboração (diálogo contínuo sobre especificações, desenvolvimento colocalizado, sistema de tickets comum, mensagem instantânea), prototipação e um pipeline que, a partir de 2014, leva cada commit bem-sucedido ao servidor de teste de aceitação do cliente. O artigo não traz falas literais: todas as citações abaixo são sínteses dos autores a partir das entrevistas. A amostra é de cinco pessoas e o artigo não separa o efeito do pipeline do efeito das práticas de colaboração.

Mecanismo: o pipeline de implantação permite prototipar continuamente e levar cada mudança ao ambiente de teste do cliente, então a divergência de entendimento do requisito aparece cedo e o trabalho feito contra especificação mal interpretada deixa de virar retrabalho; o cliente também ficou mais fácil de consultar e a espera por ele caiu. O contato com o cliente aumentou; o que diminuiu foi o desgaste que o fator descreve. **Condições:** junto com práticas de colaboração adotadas na mesma transição (diálogo contínuo sobre especificações, desenvolvimento colocalizado, sistema de tickets comum, mensagem instantânea), que o artigo não separa do pipeline; o feedback do usuário continua limitado pela agenda do cliente. **Evidência:** empírica indireta.

Seção 2.4.1 (estado inicial): *“lead to considerable amounts of wasted work”*

Descrição dos autores da situação em 2010, quando o fornecedor assumiu as aplicações do cliente. Refazer funcionalidades depois de descobrir que não atendiam à necessidade do cliente gerava muito trabalho perdido. É o estado anterior à prática, o desgaste que o fator descreve.

Seção 3.1.1: *“specs is now largely avoided through continuous”*

Trecho da seção sobre ganhos de produtividade do fornecedor: o esforço desperdiçado por desenvolver contra especificações mal feitas ou mal interpretadas agora é em grande parte evitado pela discussão contínua e pela prototipação. É o estado depois; note que aqui os autores atribuem a melhora ao jeito mais ágil de especificar, não ao pipeline diretamente.

Seção 3.7: *“the continuous prototyping, enabled by the deployment pipeline and related automation, allows developers to get faster validation of their interpretation of the requirements”*

Na seção sobre o moral dos desenvolvedores, os autores dizem que a prototipação contínua, possibilitada pelo pipeline de implantação e pela automação, permite validar mais cedo a interpretação que o desenvolvedor fez do requisito. É a citação que liga a prática (o pipeline) ao fator, explicando por que o mal-entendido aparece antes de virar retrabalho.

Seção 3.2.1: *“Developers also have a much lower threshold for contacting the customer”*

Na seção sobre melhorias de colaboração do lado do fornecedor, os autores relatam que os desenvolvedores agora hesitam muito menos em contatar o cliente para tirar dúvidas pequenas durante o desenvolvimento. Mostra que o contato aumentou e que o cliente ficou mais acessível, o que reduz a parte do fator ligada à indisponibilidade do cliente.

Seção 3.1.1: *“the time spent waiting for deployment decisions, feedback, and customer”*

Os autores dizem que o tempo gasto esperando decisões de implantação, retorno e comunicação com o cliente (a citação para em 'customer'; no texto segue 'communication') caiu bastante, graças às práticas de colaboração mais próximas e às ferramentas adequadas. Mostra a redução da espera pelo cliente, atribuída sobretudo à colaboração, e não só ao pipeline.

Seção 3.8: *“User feedback can be gained more quickly, but is limited by the schedules of the customer stakeholders.”*

Na discussão, os autores ponderam que o retorno dos usuários chega mais rápido, mas continua limitado pela agenda das pessoas do cliente. É a condição que restringe o elo: a dependência da disponibilidade do cliente diminuiu, mas não sumiu.

↑ **aumenta** Relações no trabalho (fator +)

Kärpänoja et al. (2016) — *Exploring peopleware in continuous delivery*

Contexto: Mesmo estudo de três entrevistas com projetistas líderes de projetos da Solita, cada um feito por um único time de 4 a 7 pessoas. Para manter o ramo principal sempre implantável, os projetos adotaram a regra de que fazer merge no master significa declarar o código pronto para produção, e cada programador era responsável por testar a própria implementação. O elo é uma interpretação dos autores a partir das entrevistas: essa regra obriga cada um a confiar que as mudanças dos colegas também podem ir para produção, e daí surgiria um time unido que responde pelo produto inteiro. Não há medida de coesão nem falas literais, e os autores ressalvam que os desenvolvedores da Solita são abertos a mudança, o que pode ter favorecido o resultado.

Mecanismo: como todo merge no ramo principal precisa estar pronto para produção e cada um testa a própria implementação, os desenvolvedores passam a confiar que as mudanças dos colegas também são implantáveis, o que forma um time unido que responde junto pelo produto. **Condições:** depende de o time assumir a responsabilidade pela qualidade e pelos testes; os autores notam que os desenvolvedores da Solita são abertos a mudança. **Evidência:** empírica indireta.

> Seção 4.1 (os três projetos): *“they also had to trust their colleges that their changes are deployable, too. This created a unified team that was ready to take responsibility for the whole product.”*
>
> Os autores explicam que, como o merge no ramo principal equivale a declarar o código pronto para produção e cada programador testa a própria implementação, todos têm de confiar que as mudanças dos colegas também podem ser implantadas ('colleges' no original é erro por 'colleagues'). Segundo eles, isso criou um time unido, disposto a assumir a responsabilidade pelo produto todo; é o mecanismo central do elo, a confiança mútua imposta pela prática.

> Seção 4.1 (os três projetos): *“When teams had more responsibility, they clearly did not consider themselves as just programmers but rather as full members in a solid team.”*
>
> Frase do parágrafo sobre a automação da configuração da infraestrutura: com mais responsabilidade (por testes, qualidade e infraestrutura), os membros dos times não se viam como simples programadores, e sim como membros plenos de um time sólido. Para o elo, mostra o sentimento de pertencer ao time, embora a frase o ligue à responsabilidade ampliada em geral e não só à regra de merge.

↓ **reduz** Risco de falha na implantação (fator emergente, Técnico e ferramentas)

Itkonen et al. (2016) — *Perceived Benefits of Adopting Continuous Delivery Practices*

Contexto: Mesmo estudo de caso na Solita Oy. No início, cada release era montada para um ambiente específico, levada num pendrive e implantada no local, com muitos passos manuais; a seção 3.4 abre lembrando que passos manuais trazem risco de erro humano. A transição trouxe builds independentes de ambiente, teste automático das mudanças, menos passos para acessar os ambientes do cliente e ambientes de desenvolvimento parecidos com produção. A ligação com o estresse vem dos relatos dos desenvolvedores resumidos na seção 3.7. Ressalvas: a redução do risco foi percebida pelos desenvolvedores, mas não pelos representantes do cliente (Tabela 3 e conclusões), e o monitoramento de produção ainda era incipiente.

Mecanismo: builds independentes de ambiente, teste automático de toda mudança, menos passos para acessar os ambientes do cliente e ambientes parecidos com produção diminuem a chance de a implantação falhar e o trabalho extra que a falha exige; os desenvolvedores atribuem a isso boa parte da queda do estresse. **Condições:** organização ainda em adoção de CD, com monitoramento de produção incipiente. **Evidência:** empírica direta.

> Seção 3.4: *“Four factors were identified as having decreased release and deployment risk significantly”*
>
> Frase de abertura dos autores na seção 'risco reduzido de falha na release': quatro fatores diminuíram bastante o risco de release e implantação. Os quatro são builds independentes de ambiente, teste automático de toda mudança, menos passos para acessar os ambientes do cliente e ambientes parecidos com produção. Define o que, na prática, reduziu o risco.

> Seção 3.4: *“the automatic testing of all changes has reduced the risk of uncovering bugs in the applications after production deployments”*
>
> O segundo desses fatores, na descrição dos autores: o teste automático de todas as mudanças reduziu o risco de descobrir defeitos na aplicação depois da implantação em produção. Ilustra o mecanismo técnico mais diretamente ligado à entrega contínua.

> Seção 3.7: *“Much of the reduction in stress levels results from the reduced risk of failing deployments and the additional work that those require”*
>
> Na seção sobre o moral, os autores relatam que os desenvolvedores descreveram um ambiente de trabalho menos estressante e atribuem boa parte da queda do estresse ao menor risco de implantações falharem e ao trabalho extra que essas falhas exigiam. É o que liga o fator emergente ao desfecho estresse.

Integração contínua

8 elos em 3 artigo(s)

> **O que é.** Cada desenvolvedor junta o seu trabalho ao código comum do time várias vezes por dia, e um sistema automático verifica na hora se a junção quebrou alguma coisa. É o oposto de cada pessoa trabalhar semanas isolada e só juntar tudo no fim, quando os conflitos já se acumularam. Descrição completa na seção 1.

↓ **reduz** Alinhamento do uso do tempo (fator +)

Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*

Contexto: O elo vem das respostas abertas do survey com 158 usuários de Travis CI sobre problemas da integração contínua, logo depois da menção à disciplina exigida pela CI. Os autores registram que um único respondente associou essa disciplina a uma perda de foco: o desenvolvedor deixa de se concentrar só no código da aplicação e passa a cuidar também do sistema de build. O elo se apoia inteiramente nessa fala, interpretada pelos autores como "falta de foco". É a percepção de uma só pessoa, sem contagem nem confirmação por outros respondentes.
Mecanismo: a disciplina da CI desvia a atenção do código da aplicação para o código do sistema de build, e o engenheiro de software passa a fazer trabalho de infraestrutura, que considera bem menos interessante. **Condições:** relato de um respondente. **Evidência:** empírica direta.

> Seção IV.B, um respondente: *“instead of only focusing on the code of your application, you also focus on the code of your build system”*
>
> O respondente descreve o deslocamento de atenção: além do código da aplicação, passa a ser preciso cuidar do código do sistema de build. Mostra o mecanismo do elo, em que parte do tempo do desenvolvedor migra para a infraestrutura.

> Seção IV.B, o mesmo respondente: *“becomes an infrastructure engineer, which is much less interesting”*
>
> Continuação da mesma fala: o engenheiro de software vira engenheiro de infraestrutura, trabalho que o respondente considera bem menos interessante. É o que liga a prática ao fator: o tempo passa a ser gasto em tarefas que a pessoa valoriza menos do que desenvolver a aplicação.

↑ **aumenta** Carga de trabalho (fator −)

Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*

Contexto: O dado vem do mesmo survey com 158 usuários de Travis CI em projetos open source populares, na análise das respostas abertas sobre problemas da integração contínua. Os autores agrupam sob "esforço adicional" as respostas de apenas três respondentes, além dos que já haviam mencionado disciplina. O elo se apoia, portanto, numa fala ilustrativa de um respondente que lista as tarefas extras que a CI cria para o time: compartilhar, documentar, ajudar a resolver problemas e garantir que os builds passem. É uma evidência fraca em número, baseada só na percepção de poucos participantes, e o artigo não diz o tamanho dos projetos nem dos times desses respondentes.
Mecanismo: manter os builds passando exige esforço adicional de compartilhar, documentar e ajudar a resolver problemas, o que segundo os respondentes pede mais gente. **Evidência:** empírica direta.

> Seção IV.B, síntese dos autores (3 respondentes): *“additional effort was also evidenced by 3 other respondents”*
>
> É a contagem dos autores: além dos que falaram de disciplina, três respondentes relataram que a CI exige esforço adicional. O número mostra que o achado é minoritário na amostra de 158 pessoas, o que limita o peso do elo.

> Seção IV.B, um respondente: *“Need more people sharing / Documenting and helping resolve problems / Making sure builds pass”*
>
> Exemplo dado pelos autores para o esforço adicional: o respondente diz que é preciso mais gente para compartilhar conhecimento, documentar, ajudar a resolver problemas e garantir que os builds passem. A fala descreve as tarefas que se somam ao trabalho de desenvolvimento quando o projeto adota CI, e é daí que vem o aumento da carga de trabalho.

↑ **aumenta** Carga de trabalho (fator −)

Soltren e Mulligan (2023) — *Enabling Pre-Merge CI on your TV*

Contexto: No mesmo resumo de palestra da Roku, depois de listar as mudanças técnicas no build do RokuOS, os autores dedicam um parágrafo aos desafios organizacionais: desenvolvedores acostumados ao Perforce, mudanças de arquitetura para usar submódulos, treinamento para ler merge commits e o processo extra criado pelo CI pré-merge obrigatório. O elo com a carga de trabalho vem dessa frase dos autores, que reconhecem que tornar obrigatória a aprovação no CI acrescentou passos a quem faz commit sempre que o merge falha. Não há medida de tempo, volume de trabalho ou percepção dos desenvolvedores: é uma observação de passagem de quem implantou a prática, e não se sabe quanto esse processo extra pesa nem com que frequência ocorre.

Mecanismo: tornar obrigatória a aprovação no CI pré-merge acrescentou passos ao processo de quem faz commit sempre que o merge falha nas verificações. **Condições:** quando o merge falha nas verificações obrigatórias. **Evidência:** inferida.

> Corpo, desafios organizacionais: *"CI checks resulted in additional process for committers when merges failed."*
>
> Os autores, engenheiros de Developer Experience que conduziram a adoção, apontam um custo da prática para os desenvolvedores que fazem commit (committers): quando o merge não passa nas verificações obrigatórias de CI, eles têm de cumprir passos adicionais antes de integrar a mudança. A citação é a única base do elo e indica o mecanismo e a condição (falha no merge), sem quantificar o acréscimo.

↑ **aumenta** Confiança para alterar o código (fator emergente, Técnico e ferramentas)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Contexto: No survey com 158 usuários de Travis CI em projetos open source, 10 respondentes apontaram como benefício da CI a confiança para fazer as mudanças necessárias no código, porque o build automatizado com testes indica que o comportamento existente não foi quebrado. O próprio respondente citado pelos autores ressalva que essa confiança depende da cobertura de testes. Em sentido contrário, 25 respondentes descreveram uma falsa sensação de confiança, quando se confia cegamente nos testes e se aprova código sem revisão cuidadosa. O elo, portanto, se apoia na percepção de uma minoria e vem acompanhado, no mesmo artigo, de um contraponto mais frequente; a fala sobre não ter medo de quebrar o build foi dada pelos autores como uma das razões sociais para a quebra de build.
Mecanismo: o build automatizado com testes dá segurança de que a mudança não quebrou o comportamento existente, e tira o medo de quebrar o build num ramo. **Condições:** depende da cobertura de testes; 25 respondentes apontam falsa sensação de confiança quando se confia cegamente nos testes. **Evidência:** empírica direta.

> Seção IV.A, síntese dos autores (10 respondentes): *"continuous integration gives more confidence to perform required code changes"*
>
> Os autores relatam que 10 respondentes veem como benefício da CI mais confiança para realizar as mudanças necessárias no código. É a evidência principal do elo, apresentada entre os benefícios da prática.

> Seção IV.B, síntese dos autores (25 ocorrências): *"Another recurring problem is the false sense of confidence"*
>
> Os autores introduzem o problema da falsa sensação de confiança, citado 25 vezes: desenvolvedores confiam cegamente nos testes, e um build aprovado pode levar a mesclar código sem revisão cuidadosa. A citação sustenta a condição do elo: a confiança dada pela CI só é bem fundada quando a cobertura de testes é suficiente.

↓ **reduz** Integração de novatos (fator +)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Contexto: O survey ouviu 158 desenvolvedores que já quebraram builds em projetos open source populares com Travis CI, um ambiente em que contribuidores novos e casuais chegam com frequência. Ao listar os problemas da CI, os autores observam que superar as dificuldades técnicas de configuração é especialmente difícil para integrar novatos, citado em 9 respostas. Antes, ao discutir causas técnicas de quebra de build, eles já haviam notado que contribuidores casuais, sem orientação ou documentação, podem nem saber do fluxo do projeto. O elo combina essas duas passagens: a contagem e a fala sobre novatos, e um relato em primeira pessoa de quem só descobriu o linter e a CI depois de enviar o patch.
Mecanismo: os problemas técnicos de configuração e o fluxo de CI pouco explicado dificultam a entrada de novatos e contribuidores casuais, que não entendem o que a CI faz e só descobrem o fluxo depois de submeter o patch. **Condições:** projetos open source sem orientação ou documentação para quem chega. **Evidência:** empírica direta.

> Seção IV.B, síntese dos autores (9 ocorrências): *"Overcoming such technical problems is particularly challenging for onboarding newcomers"*
>
> Os autores afirmam que os problemas técnicos de configuração da CI pesam em particular na entrada de novatos, tema que apareceu 9 vezes nas respostas. Estabelece o elo entre a complexidade da CI e a dificuldade de integrar quem chega ao projeto.

> Seção IV.B, respondentes sobre novatos: *"do not understand what CI does and what it doesn't"*
>
> Trecho de resposta citado pelos autores para explicar a dificuldade: os novatos não entendem o que a CI faz e o que ela não faz. Mostra que a barreira não é só técnica, mas de compreensão do papel da CI no fluxo do projeto.

> Seção III.A, um respondente (causas técnicas de quebra): *"I didn't know about the linter (or that CI was being used) until after I submitted my patch."*
>
> Um respondente conta que só soube da existência do linter (ferramenta que verifica regras de estilo do código) e do uso de CI depois de submeter o patch. Os autores usam o relato para mostrar que contribuidores casuais, sem orientação nem documentação, desconhecem o fluxo do projeto; para o elo, é o retrato concreto do novato surpreendido pela CI.

↓ **reduz** Manutenção do ambiente de desenvolvimento (fator −)
Soltren e Mulligan (2023) *— Enabling Pre-Merge CI on your TV*
Contexto: O texto é o resumo estendido, de uma página, de uma palestra no ICSTW 2023 em que dois engenheiros da área de Developer Experience da Roku relatam mudanças no build do RokuOS, o firmware Linux que roda em dezenas de modelos de TV, player e caixa de som. Antes, o código ficava num monorepo no Perforce e só havia verificação depois do commit (pós-submit); com o volume de mudanças e a variedade de plataformas, quando uma mudança quebrava o build o desenvolvimento chegava a parar enquanto se desfaziam mudanças e se resolviam conflitos à mão. A empresa migrou para git com submódulos, passou a aceitar merge commits, criou verificações de CI antes do merge (pré-merge) e as tornou obrigatórias, com uma camada de emulação para testar rápido sem hardware. O elo vem da comparação antes e depois feita pelos autores com uma métrica técnica, a frequência de builds quebrados, sem nenhum dado sobre pessoas. A ressalva é que a queda medida acompanhou várias mudanças simultâneas, não só o CI pré-merge, e o efeito sobre o trabalho dos desenvolvedores é inferido da métrica de build.
Mecanismo: o CI pré-merge obrigatório barra a mudança que quebraria o build, então os builds quebrados caem e o desenvolvimento deixa de parar para desfazer mudanças e resolver conflitos à mão. **Condições:** CI pré-merge tornado obrigatório e somado ao pós-merge, com camada de emulação para acelerar a verificação; base de código grande e multiplataforma. **Evidência:** empírica indireta.

> Corpo, situação antes (só verificação pós-submit): *“development would sometimes come to a standstill as we backed out changes or resolved conflicts manually”*
>
> Os autores, que cuidam da infraestrutura de build, descrevem como era o trabalho quando só existia a verificação depois do commit: uma mudança quebrada parava o desenvolvimento até alguém desfazê-la ou resolver os conflitos manualmente. É o estado anterior à prática e mostra o esforço de manter o ambiente funcionando que o CI pré-merge pretende evitar.

> Corpo, resultado: *“the frequency of build breakages was reduced from 8.70 percent to 4.85 percent in a period of three months”*
>
> É a métrica que os autores usaram para justificar as mudanças: a proporção de builds quebrados caiu de 8,70% para 4,85% em três meses, cerca de 50% de melhora na confiabilidade. Para o elo, ela sustenta que houve menos quebras a consertar, mas mede o build e não o trabalho das pessoas, e cobre o conjunto das mudanças, não só o CI pré-merge.

↑ **aumenta** Percepção de recompensa (fator emergente, Organizacional)
Riedl et al. (2025) *— How an Agile Software Process Increases Developers’ Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model*
Contexto: O estudo é um survey online transversal, aplicado em 2021 por divulgação nas redes pessoais dos autores e em comunidades de software da Áustria (amostra de conveniência); dos 217 desenvolvedores que completaram o questionário, foram analisados os 178 que disseram trabalhar em time ágil. A agilidade foi medida pela frequência e intensidade de uso de oito práticas do instrumento de Maruping et al. (2009), entre elas a integração contínua; o estresse foi medido como desequilíbrio esforço-recompensa (ERI, de effort-reward imbalance, modelo de Siegrist), isto é, a razão entre o esforço que a pessoa investe e a recompensa que percebe (estima, reconhecimento, segurança no emprego, perspectiva de carreira). Com modelos de mediação (PROCESS, modelos 4 e 7 de Hayes), os autores estimaram o efeito de cada prática sobre o estresse e, por meio dele, sobre a satisfação no trabalho, e decompuseram o efeito em esforço e recompensa só para a agilidade como um todo. O elo é inferência dos autores: a integração contínua tem efeito no limite da significância sobre o estresse (β = -0,0151, p = 0,0553, com intervalo de confiança que chega a 0,0002) e efeito indireto significativo sobre a satisfação, mas a afirmação de que ela aumenta a recompensa percebida transfere para a prática o resultado da agilidade agregada, sem detalhar o mecanismo. Os próprios autores ressalvam que a amostra não é grande, que dados transversais podem produzir efeitos de mediação que não se confirmariam ao longo do tempo e que só controlaram fatores individuais, não organizacionais nem de projeto; além disso, os efeitos são estatisticamente pequenos (R^2 de 0,017 no modelo da CI) e o benefício da agilidade some entre os muito sobrecomprometidos.
Mecanismo: segundo os autores, a CI, junto das demais práticas técnicas, torna a razão esforço-recompensa mais favorável porque aumenta a recompensa percebida (estima, reconhecimento, segurança, carreira), e não porque reduz o esforço. O artigo não detalha como a CI em particular gera essa recompensa. **Condições:** o efeito da CI sobre o estresse é marginal (p = 0,0553); só o efeito indireto sobre a satisfação é significativo. A redução por recompensa foi testada apenas para a agilidade agregada (Tabela 7). Na agilidade agregada, o benefício diminui com o sobrecomprometimento e desaparece no nível alto; incluir o moderador não muda os resultados por prática. Amostra de times que se declaram ágeis. **Evidência:** inferida.

> Seção 5, Tabela 6 (resultados por prática, modelo 4): *“the statistics show that requirements change and customer relation have a stress-increasing effect, while the other six dimensions have a stress-reducing effect”*
>
> Os autores resumem a Tabela 6, que repete o modelo de mediação para cada uma das oito práticas medidas: mudança de requisitos e relação com o cliente aumentam o desequilíbrio esforço-recompensa, e as outras seis, entre elas a integração

contínua, o reduzem. Para a CI, o coeficiente é negativo (-0,0151), mas com p = 0,0553, ou seja, fora do limiar usual de 5%; o resumo dos autores trata como redutor um efeito que, isoladamente, fica no limite.

Seção 5, Tabela 7 (decomposição em esforço e recompensa, modelo 4): *"agility affects stress predominantly via the reward component"*
A Tabela 7 separa os dois componentes do estresse no modelo ERI e mostra que a agilidade, medida como a soma das oito práticas, não muda o esforço percebido (p = 0,61), mas aumenta a recompensa percebida (β = 0,127, p < 0,001). É desse resultado que vem o fator do elo; a decomposição não foi feita prática por prática, então a citação sustenta a recompensa para a agilidade em geral, não para a integração contínua em particular.

Seção 6.1 (contribuição científica): *"continuous integration, refactoring, unit testing, collective ownership, and coding standards are agile practices that increase developers' reward perceptions"*
Na discussão, os autores juntam os dois resultados anteriores e afirmam que a integração contínua e outras práticas técnicas aumentam a recompensa percebida pelos desenvolvedores (a frase original começa com programação em par). É a interpretação dos autores que liga a CI ao fator, e não um teste específico; na mesma lista entra o teste unitário, cujo efeito sobre o estresse não foi significativo, o que mostra que a afirmação generaliza além do que a tabela por prática sustenta.

↑ **aumenta** Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)
Pinto et al. (2017) *— Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Contexto: No survey com 158 usuários de Travis CI em projetos open source, 12 respondentes disseram que as técnicas de integração contínua exigem certo nível de disciplina dos desenvolvedores. A norma em questão é a de que o ramo principal (master) deve compilar sempre, de modo que cada integração precisa deixar o build verde. O elo se baseia na síntese dos autores e numa fala de um respondente que descreve essa norma como trabalhar sob pressão. O fator é emergente (não está no catálogo) e o artigo não mede a pressão de outro modo além do relato.
Mecanismo: a norma de que o ramo principal tem de compilar sempre impõe disciplina e auto-organização constantes, vividas como trabalhar sob pressão. **Evidência:** empírica direta.

Seção IV.B, síntese dos autores (12 respondentes): *"techniques require developers a certain level of discipline"*
Os autores resumem o que 12 respondentes disseram: usar CI requer dos desenvolvedores um certo nível de disciplina. Mostra que a exigência não é um caso isolado e situa a disciplina como um custo humano da prática, apresentado entre os problemas da CI.

Seção IV.B, um respondente: *"it takes a lot of self-organization to work under pressure of 'master must build'"*
Um respondente diz que é preciso muita auto-organização para trabalhar sob a pressão de "master must build", isto é, a regra de que o ramo principal tem de compilar sempre. É a fala que dá nome ao fator emergente: a norma do build verde é vivida como pressão permanente sobre quem integra código.

Bots de desenvolvimento no pipeline de CI/CD

5 elos em 1 artigo(s)

> **O que é.** Programas automáticos que atuam no fluxo de trabalho dos desenvolvedores — por exemplo, comentando nas revisões de código, apontando violações de padrão, sugerindo revisores ou respondendo dúvidas frequentes. Prática proposta durante a leitura, ainda não incorporada ao catálogo. Descrição completa na seção 1.

↑ **aumenta** Carga cognitiva (fator −)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Contexto: O elo vem da análise dos desafios relatados pelos usuários de bots na Microsoft (9 entrevistados e 54 respondentes do survey), em que vários bots comentam ao mesmo tempo no mesmo pull request. Os problemas foram recomendações que ocupam muito espaço na tela (D1: 5 de 9 e 21% do survey), sem próximo passo claro (D2: 5 de 9 e 21%) e sem relação com a mudança em questão (D3: 4 de 9 e 42%). Os autores concluem que bots demais causam ruído mesmo quando cada um é útil, e que o desenvolvedor precisa priorizar as recomendações; a reação descrita é ignorar o que não se entende. Bots que indicam a ação exata, como os de segurança e de configuração, não geram esse custo, e os autores mostram nos logs que o engajamento continua alto para bots que apontam problemas urgentes.
Mecanismo: vários bots comentando no mesmo pull request, com relatórios longos e estatísticas sem explicação, poluem a tela de revisão e obrigam o desenvolvedor a filtrar e priorizar o que importa; o que não se entende acaba ignorado.
Condições: muitos bots simultâneos; recomendações extensas, sem próximo passo claro ou sem relação com a mudança; bots que indicam a ação exata (B3) não têm esse custo. **Evidência:** empírica direta.

Seção 6.2, D1, P18 (usuário de bot): *“This cluttered the pull-request review screen that developers use to review comments left by peer developers during code review (P18)”*

Os autores relatam, com base em P18, que recomendações detalhadas de bots como CodeInsights e StaticAnalysis ocupavam boa parte da tela e poluíam a interface de revisão, a mesma usada para ler comentários dos colegas na revisão de código. Mostra a sobrecarga de informação que o desenvolvedor precisa filtrar.

Seção 6.2, D3, usuário não identificado: *“When I see these recommendations, I see a lot of statistics and crash reports. I am not able to connect the dots as to how this would be useful for me.”*

Fala de um usuário sobre o desafio D3 (recomendação sem relação com a mudança): ele vê muitas estatísticas e relatórios de falhas e não consegue entender como aquilo lhe seria útil. Mostra o esforço de interpretação exigido por recomendações sem explicação.

Seção 6.2, D2, P21 (usuário de bot): *“It is not immediately clear to me what I should do with that recommendation, and hence, I mostly ignore it.”*

P21, usuário entrevistado, fala do desafio D2 (sem próximo passo claro): vê as recomendações, mas não sabe o que fazer com elas e por isso costuma ignorá-las. Mostra o desfecho da carga cognitiva: o que exige esforço demais para decifrar é descartado.

Seção 6.2, síntese dos autores: *“observe that too many bots can cause noise, even if individual bots are helpful”*

Conclusão dos autores ao fechar os desafios: ao estudar vários bots atuando juntos, observaram que bots em excesso causam ruído, mesmo que cada bot isolado seja útil. É a condição do elo: o custo cognitivo nasce da soma de bots no mesmo PR, não de um bot específico.

↕ **ambivalente** Carga de trabalho (fator −)

Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*

Contexto: No estudo de caso dos 23 bots de CI/CD da Microsoft, os nove usuários entrevistados e os 54 do survey relataram benefícios e problemas; o benefício mais citado foi pegar o que foi esquecido (B1: 6 de 9 entrevistados e 51% do survey). Sem o bot, esquecer configurações, chamadas de biblioteca ou scripts levava a falhas de build e obrigava a abrir um pull request de correção, o que aumentava a carga de trabalho; bots como Nitpicker, Static analysis e ConfigAlert evitam isso, e outros tiram do revisor o apontamento de miudezas (B4: 2 de 9 e 16%). Em sentido oposto, os autores registram, ao descrever a motivação dos bots de qualidade de código, que suas recomendações podem alongar o pull request pelo trabalho extra de melhoria. O efeito é ambivalente por somar essas duas direções; a menção à distribuição desigual de revisões pelo bot Reviewer aparece só na discussão, sem dado no texto.
Mecanismo: o bot que pega código esquecido evita a falha de build e o pull request de correção, e o que aponta miudezas tira esse trabalho do revisor; já os bots de qualidade de código acrescentam trabalho ao PR, e o bot que escolhe revisores distribuiu a revisão de forma desigual. **Condições:** reduz com bots que lembram configurações, dependências e regras rotineiras (Nitpicker, Static analysis, ConfigAlert); aumenta com bots de qualidade de código, que pedem melhorias além do que o PR exigia, e com o bot de revisores (achado só citado na discussão, sem dado no texto). **Evidência:** empírica direta.

Seção 6.1, B1, P16 e P17 (usuários de bot): *“increasing their workload. Developers found bot recommendations helpful for "preventing a failure from a missed code" (P17) or "saving on pull-request iterations" (P16)”*

Trecho dos autores que junta falas de dois usuários entrevistados: esquecer código obrigava a abrir um pull request de correção, aumentando a carga de trabalho, e os bots ajudam a evitar a falha (P17) e a economizar iterações do PR (P16). Mostra o estado sem o bot e a redução de retrabalho que ele traz, a direção "reduz" do efeito.

Seção 6.1, B4, síntese dos autores: *“Bots take the burden off the code-reviewers to nitpick every minor issue in the pull-request, letting code-reviewers provide feedback on the more important aspects of the review”*

Os autores resumem o valor cultural percebido pelos usuários no benefício B4 (ajuda na revisão de código): os bots tiram dos revisores o fardo de apontar cada pequeno problema, e eles podem se concentrar no que importa. Mostra a redução de carga do lado do revisor.

Seção 5.2, M4, síntese dos autores: *“The recommendations from these bots may sometimes lead to more turnaround time for the pull-request due to the additional work needed to improve code quality”*

Ao descrever a motivação dos bots CodeQuality (que cobram estilo, formatação e boas práticas), os autores admitem que suas recomendações às vezes alongam o tempo do pull request, pelo trabalho extra de melhorar a qualidade do código. Sustenta a direção "aumenta" do efeito; é uma observação dos autores, não fala de usuário.

Seção 8.2, discussão: *“unbalanced workload allocation by the "Reviewer" bot”*

Na discussão sobre relevância e excesso de informação, os autores mencionam ter encontrado distribuição desigual de revisões pelo bot Reviewer, que adiciona revisores automaticamente aos PRs. O achado não aparece nos resultados com dados ou falas, e por isso pesa pouco: só indica que o bot pode concentrar trabalho de revisão em algumas pessoas.

↑ **aumenta** Conflito de papéis (fator −)

Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*

Contexto: O elo vem do fechamento da seção de desafios dos usuários de bots no estudo de caso da Microsoft (9 usuários entrevistados e 54 no survey). Ali os autores sintetizam que vários desafios relatados decorrem de um descompasso de prioridades entre quem desenvolve os bots e quem os usa: os bots DevProductivity e CodeQuality foram feitos para alertar sobre boas práticas, desempenho e atrasos, e isso às vezes conflita com o desejo do usuário de terminar o pull request no prazo. Não há fala de participante, contagem nem dado de survey ligados especificamente a esse ponto, por isso a evidência é inferida a partir da interpretação dos autores. O próprio texto marca a ressalva com "às vezes".

Mecanismo: os bots de produtividade e de qualidade cobram boas práticas e desempenho no pull request, o que concorre com a cobrança por terminar o PR no prazo. **Condições:** bots das categorias DevProductivity e CodeQuality; síntese dos autores sobre os desafios relatados pelos usuários. **Evidência:** inferida.

Em revisão: O texto fala de conflito entre as cobranças do bot e o desejo do próprio usuário de terminar o PR no prazo, não de expectativas incompatíveis vindas de papéis ou pessoas diferentes; o elo talvez se encaixe melhor em pressão de prazo ou carga de trabalho do que em "Conflito de papéis". A evidência é só síntese dos autores, sem fala de participante.

> Seção 6.2, síntese dos autores: *“mismatch in priorities between bot developers and bot users”*
>
> Os autores resumem os desafios D1 a D4 como resultado de um descompasso de prioridades entre desenvolvedores de bots e usuários de bots. Situa a origem do conflito: duas partes com objetivos diferentes atuando sobre o mesmo pull request.

> Seção 6.2, síntese dos autores (continuação): *“However, these may sometimes”*
>
> Fragmento da frase seguinte, em que os autores dizem que os alertas de boas práticas e desempenho dos bots DevProductivity e CodeQuality às vezes entram em conflito com algo. O "às vezes" mostra que o conflito é ocasional, segundo os próprios autores.

> Seção 6.2, síntese dos autores (continuação): *“the pull-request on a deadline”*
>
> Fim da mesma frase: o conflito é com o desejo do usuário de concluir o pull request dentro do prazo. Mostra as duas exigências que concorrem, a qualidade cobrada pelo bot e a entrega no prazo.

↑ **aumenta** Documentação interna (fator +)

Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*

Contexto: Na organização estudada (Microsoft), times especializados em privacidade de dados atendem muitos times de produto e se tornam gargalo, porque a maior parte do suporte consiste em responder perguntas repetidas. Para isso, o time de privacidade criou os bots ThreatModelQuality e SecReview, que verificam o código, abrem work items e respondem às perguntas frequentes em nome dos especialistas, que só entram nas questões não triviais. O elo combina a fala de um usuário (P18), que compara a resposta rápida do bot com a espera de um dia inteiro pela resposta manual, e a fala de um desenvolvedor do bot no time de privacidade (P11), sobre o bot assumir tarefas rotineiras. O benefício de comunicação mais rápida (B2) foi relatado por 3 dos 9 entrevistados e 29% do survey, mas esse número inclui outros bots além do de privacidade.

Mecanismo: o bot da equipe de privacidade responde às perguntas frequentes no próprio work item, e o desenvolvedor não fica um dia parado esperando um especialista; o especialista deixa de ser acionado para as dúvidas repetidas. **Condições:** dúvidas rotineiras e repetidas (FAQ) de um time especialista escasso; as questões não triviais continuam com o especialista humano. **Evidência:** empírica direta.

Em revisão: O fator "Documentação interna" encaixa de forma frouxa: as citações falam de um bot que responde dúvidas no lugar do especialista e encurta a espera por ele, o que pesa mais sobre dependência e tempo de espera por especialistas (ou carga do time de privacidade) do que sobre a documentação em si. Além disso, metade das citações (P11) vem de quem desenvolveu o bot, não de quem o usa.

> Seção 6.1, B2, P18 (usuário de bot): *“quick help on basic privacy questions compared to waiting an entire day when someone had to manually answer them”*
>
> P18 é usuário do bot ThreatModelQuality e fala do benefício B2 (comunicação mais rápida). Ele conta que obtém ajuda rápida em perguntas básicas de privacidade, quando antes esperava um dia inteiro até alguém responder manualmente; descreve o antes e o depois do bot.

> Seção 5.2, M3, P11 (desenvolvedor de bot, time de privacidade): *“has been helping us address security concerns at scale as the bot”*
>
> P11 é um dos desenvolvedores de bot do time de privacidade de dados e explica a motivação M3: usar o bot como meio de comunicação assíncrono tem ajudado o time a tratar questões de segurança em escala. Mostra o efeito do lado de quem antes respondia às dúvidas.

> Seção 5.2, M3, P11 (continuação): *“several routine tasks”*

Continuação da fala de P11: o bot assume várias tarefas rotineiras, como apontar problemas e informar como resolvê-los. É o mecanismo do elo: o conhecimento repetitivo do especialista passa a ser entregue pelo bot, sem depender de uma pessoa disponível.

↑ **aumenta** Integração de novatos (fator +)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Contexto: O artigo é um estudo de caso na Microsoft sobre 23 bots que atuam nos pull requests e work items do pipeline de CI/CD, com entrevistas com 13 desenvolvedores de bots, 2 da plataforma e 9 usuários, survey com 54 usuários e análise de logs. Um dos bots de violação de configuração, o Nitpicker, foi configurado pelos times para lembrar, no próprio pull request, dependências, configurações e atualizações que o repositório costuma exigir. O elo se apoia em duas fontes: um usuário entrevistado (P17), que descreve como o Nitpicker ajuda o novato a aprender as convenções sem ocupar um desenvolvedor sênior, e a menção dos autores de que o time de um desenvolvedor de bot (P3) usava o bot com esse fim. A evidência sobre novatos é de poucos participantes e o artigo não mede o tempo de integração; os autores retomam o ponto na discussão como exemplo de comunicação com novatos.
Mecanismo: o bot de configuração lembra o novato das regras e convenções do repositório no próprio pull request, e ele aprende as nuances sem precisar que um sênior aponte cada uma. **Condições:** bots de violação de configuração/estilo (Nitpicker) configurados pelo time para as convenções do repositório. **Evidência:** empírica direta.

Seção 6.1, B1, P17 (usuário de bot): *"it helps newcomers learn the nuances of"*
P17 é um dos nove usuários de bots entrevistados, falando do benefício B1 (o bot pega o que foi esquecido). Ele conta que o time configurou o Nitpicker para gerar lembretes sobre arquivos de configuração e que isso ajuda os novatos a aprender as sutilezas dessas configurações; é o mecanismo do elo, visto por quem usa o bot.

Seção 6.1, B1, P17 (continuação): *"without burdening a senior developer to point that out"*
Continuação da mesma frase de P17: o aprendizado acontece sem sobrecarregar um desenvolvedor sênior, que antes teria de apontar cada detalhe. Mostra a situação sem o bot (o sênior corrigindo o novato) e o ganho que ele traz para a integração de quem chega.

Seção 5.2, M1, P3 (desenvolvedor de bot): *"violation bot to guide newcomers towards standard team practices"*
Frase dos autores ao descrever a motivação M1 (bots de violação de configuração): o time de P3, um dos desenvolvedores de bots entrevistados, usava um bot desse tipo para conduzir novatos às práticas padrão da equipe. Não é fala literal de P3 e expressa a intenção de quem criou o bot, o que complementa, do lado da motivação, o relato do usuário.

Releases frequentes

5 elos em 4 artigo(s)

> **O que é.** Encurtar o intervalo entre versões do produto: em vez de uma grande versão por ano ou semestre, uma versão a cada poucas semanas ou dias, em calendário fixo. O navegador Firefox, por exemplo, passou de versões anuais para versões a cada seis semanas. Descrição completa na seção 1.

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Contexto: Mesmo estudo de três entrevistas com projetistas líderes de projetos da Solita que tinham pipeline de entrega contínua e podiam implantar em produção todo dia. A frequência real, porém, variava: segundo a Tabela 1, a média era de 7 implantações por mês no projeto A, 5 no B e 2 no C; os projetos A e B implantavam quando o time achava que devia, e no C o cliente exigia que a implantação fosse feita do escritório dele. O elo vem do relato sobre o projeto B, em que as implantações feitas só quando necessárias aumentaram o receio de que a próxima falhasse, e da generalização que os autores fazem a partir disso nas conclusões. É, portanto, o relato de um único entrevistado somado a uma interpretação dos autores (inclusive a ideia de um ciclo que corrói a confiança, apresentada como hipótese), e não uma comparação sistemática entre projetos.
Mecanismo: quanto mais tempo desde a última implantação em produção, menos confiança o desenvolvedor tem na próxima; implantar pouco aumenta o receio de que a próxima falhe e eleva a barreira para implantar, corroendo ainda mais a confiança. **Condições:** contraste entre projetos: no projeto B, que implantava só quando necessário, o receio aumentou; nos de implantação mais frequente não foi relatado. **Evidência:** empírica direta.

Seção 4.2, projeto B: *"Infrequent deployments in project B also increased worries"*
Os autores relatam que no projeto B (aplicação comercial em Java e JavaScript, 4 pessoas, média de 5 implantações por mês, feitas quando o time achava necessário) as implantações pouco frequentes aumentaram a preocupação de que a

próxima implantação em produção falhasse. É a evidência empírica do elo: o receio aparece justamente quando a prática de implantar com frequência não é seguida.

Seção 4.2, projeto B: *"This might also raise the bar for deployments that are not strictly necessary, which would further erode the trust."*

Na sequência, os autores especulam (o verbo é 'might') que esse receio pode elevar a barreira para implantações que não são estritamente necessárias, o que corroeria ainda mais a confiança. A frase descreve um ciclo vicioso proposto por eles, não observado: implantar pouco gera medo, e o medo leva a implantar ainda menos. Logo depois concluem que, para a implantação ser um processo normal e sem estresse, ela precisa ser automatizada e, sobretudo, frequente.

Seção 6 (conclusões): *"the less confident the developers feel to make the next production deployment."*

Fim de uma frase das conclusões cuja primeira parte diz que, quanto mais tempo passou desde a última implantação em produção, menos confiantes os desenvolvedores se sentem para fazer a próxima. É a generalização dos autores a partir do relato do projeto B, apresentada como um dos resultados do estudo, e dá a forma do elo: a confiança cai à medida que cresce o intervalo entre implantações.

↓ **reduz** Carga cognitiva (fator −)

Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*

Contexto: Mesmo estudo de entrevistas com projetistas líderes de três projetos da Solita com entrega contínua. Os desenvolvedores estavam satisfeitos e orgulhosos de poder implantar em produção sempre que quisessem, mas nos projetos B e C as implantações eram feitas só quando havia necessidade e, por isso, nem sempre eram frequentes; o projeto C, uma aplicação interna, tinha a menor média (2 implantações por mês), porque o cliente queria que o desenvolvedor fizesse a implantação a partir do escritório dele. O elo vem da fala do entrevistado do projeto C sobre a confusão causada por várias versões simultâneas da mesma funcionalidade, que os autores explicam como dificuldade de lembrar em que a produção diferia da versão em desenvolvimento. A evidência é de um só entrevistado; o texto, porém, atribui a dificuldade de lembrar às implantações raras dos projetos B e C juntos, e não apenas ao C, como sugere o campo de condições. **Mecanismo:** com implantações raras, fica difícil lembrar em que a versão em produção difere da versão em desenvolvimento, e convivem várias versões da mesma funcionalidade; implantar com frequência mantém a distância entre as duas versões pequena. **Condições:** relatado pelo entrevistado do projeto C, onde o cliente pedia que a implantação fosse feita do escritório dele e a média era de duas implantações por mês. **Evidência:** empírica direta.

Seção 4.2, projeto C: *"the interviewee in project C mentioned"*

Fragmento da frase em que os autores dizem que, apesar da satisfação geral com a possibilidade de implantar a qualquer momento, o entrevistado do projeto C mencionou confusão com múltiplas versões simultâneas da mesma funcionalidade. Identifica a fonte da evidência: um único entrevistado, o projetista líder do projeto C.

Seção 4.2, projetos B e C: *"When the deployments were only made on particular necessity, the deployments were not always that frequent in both projects B and C."*

Os autores explicam a condição que gerava a confusão: nos projetos B e C as implantações eram feitas só por necessidade específica e, por isso, nem sempre eram frequentes. É a situação sem a prática, que permite ler o elo pelo avesso: com implantações raras, cresce a distância entre o que está em produção e o que está em desenvolvimento.

Seção 4.2, projetos B e C: *"how the version in production was different from the current development version."*

Fim da frase segundo a qual, por isso, era difícil lembrar como a versão em produção diferia da versão de desenvolvimento atual. Descreve o esforço mental de manter na cabeça duas versões divergentes do sistema, que é o que liga o elo à carga cognitiva; implantar com frequência mantém essa diferença pequena.

↓ **reduz** Conflito trabalho-família (fator −)

Claes et al. (2017) *— Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content*

Contexto: O estudo, um artigo curto do MSR 2017, minera o Bugzilla (sistema de registro de bugs) do Mozilla Firefox: os autores ligaram os participantes do Bugzilla aos commits do repositório de código para obter o fuso horário e ficaram com 5,86 milhões de comentários de envio de patch e de revisão de código, feitos por 2.637 desenvolvedores. O horário desses comentários serve de indicador (proxy) de trabalho fora do expediente, que os autores tratam como sinal de pressão de prazo e de pouco desligamento do trabalho. Até o Firefox 4, em março de 2011, saía no máximo uma versão principal por ano e versões paralelas continuavam recebendo correções; depois dele, passou a sair uma versão a cada 6 semanas, substituindo a anterior. O elo vem da comparação estatística, antes e depois dessa mudança, da proporção de comentários feitos em fins de semana e à noite (teste de Mann-Whitney com tamanho de efeito). As ressalvas, reconhecidas pelos autores, são que a medida é indireta, que é uma só organização, que não há grupo de comparação para separar o efeito das releases rápidas de outras mudanças da época, e que a fatia de comentários de contratados da Mozilla subiu de 16,7% para 42,5%, o que pode explicar parte da queda, já que voluntários tendem a contribuir no tempo livre.

Mecanismo: com a passagem para um ciclo de release a cada 6 semanas, a fatia da atividade de desenvolvimento feita em fins de semana e à noite caiu, embora o volume em horário comercial tenha crescido; os autores contrariam a hipótese de que prazos mais frequentes aumentariam a pressão e empurrariam o trabalho para fora do expediente. **Condições:** Mozilla Firefox, mudança de releases anuais com versões paralelas para releases a cada 6 semanas em 2011; queda maior nos fins de semana (d = 1.43) que à noite (d = 0.45); considerando só endereços Mozilla, fins de semana d = 0.64 e noite com efeito pequeno a desprezível; parte da diferença pode vir do aumento de desenvolvedores pagos pela Mozilla; trabalho fora de hora medido pelo horário de comentários de patch e revisão; uma só organização. **Evidência:** empírica direta.

> Seção IV.B (RQ2, diferença no padrão de trabalho com releases rápidas): *"There is a statistically significant decrease in the relative amount of comments made outside office hours."*
>
> É o resultado principal da comparação antes e depois da passagem para releases a cada 6 semanas: a proporção de comentários feitos fora do horário comercial (fins de semana e noite, de 20h às 7h) caiu de forma estatisticamente significativa. Os autores acrescentam que o efeito é grande nos fins de semana (d de Cohen = 1,43) e moderado à noite (d = 0,45), enquanto o volume em horário comercial cresceu.

> Seção IV.B (RQ2): *"On average since the release of Firefox 4, a weekend day receives 24.4% of the comments a weekday receives, while before Firefox 4, a weekend day used to receive 48.6% of the comments of a weekday."*
>
> A citação traduz o efeito em números: antes do Firefox 4, um dia de fim de semana tinha quase metade (48,6%) dos comentários de um dia útil; depois, cerca de um quarto (24,4%). A medida é relativa, ou seja, mostra que o fim de semana passou a concentrar uma fatia menor do trabalho, não necessariamente que o volume absoluto de trabalho no fim de semana caiu.

> Seção IV.B, só endereços de e-mail da Mozilla: *"we still observe a large decrease in the activity during weekends"*
>
> Os autores testam a explicação alternativa de que a queda viria apenas da entrada de mais desenvolvedores pagos pela Mozilla, que trabalham em horário comercial. Restringindo aos comentários feitos com e-mail da Mozilla, a queda nos fins de semana continua grande (d = 0,64; de 52,4% para 17,2% de um dia útil), mas a da noite fica pequena a desprezível; a citação sustenta que o elo vale para os contratados, sobretudo quanto ao fim de semana.

> Seção VI (conclusão): *"switching to rapid releases has reduced the amount of work performed outside office hours"*
>
> Na conclusão, os autores contrapõem o resultado à suposição comum de que mais prazos aumentariam a pressão e empurrariam o trabalho para noites e fins de semana, e afirmam que ocorreu o contrário. É a interpretação causal dos autores, apoiada num estudo de psicologia segundo o qual prazos mais frequentes reduzem atrasos; o desenho antes e depois, porém, não isola a prática de outras mudanças.

↓ **reduz** Pressão de prazos (fator −)

Rossi et al. (2016) — *Continuous deployment of mobile software at facebook (showcase)*

Contexto: O artigo é um relato industrial (FSE 2016, trilha Showcase) escrito por engenheiros do Facebook e pesquisadores, que descreve a implantação contínua dos aplicativos móveis e analisa sete anos de dados internos (commits e pushes, crashes, cherry-picks, problemas de produção), com as conclusões conferidas por especialistas da empresa. No Facebook, os desenvolvedores enviam (push) suas mudanças ao branch principal (Master) várias vezes por semana, e a equipe de engenharia de release (RelEng) corta periodicamente um branch de release, que é estabilizado e publicado; só correções aceitas pela RelEng entram depois do corte, por cherry-pick. Ao longo de quatro anos o ciclo do Android caiu de 8 semanas para 1 semana, enquanto o iOS ficou em 2 semanas. O elo vem da análise do comportamento de push em torno do dia do corte: há um pico de pushes nesse dia, que os autores leem como corrida contra o prazo, e a proporção de pushes no dia do corte é menor no Android, com ciclo semanal, o que os autores explicam porque perder o corte custa só uma semana. Não há medida de pressão percebida, a comparação é entre plataformas e não antes e depois, e a explicação é apresentada pelos próprios autores como intuitiva; ao mesmo tempo, mover o corte para domingo aumentou os pushes no fim de semana. **Mecanismo:** com corte do release branch semanal, perder o corte custa só uma semana de espera, então o desenvolvedor não força o push no dia do corte e a corrida contra o prazo diminui. **Condições:** release branch cortado em data fixa; com ciclo mais longo (duas semanas, iOS) a proporção de pushes no dia do corte é maior; mover o corte para o fim de semana reduz ainda mais a corrida, mas aumentou os pushes no fim de semana. **Evidência:** empírica indireta.

> Seção 5.4, Deadline Effects: *"There is a clear increase in the number of pushes to the Master branch on the day the Release branch is cut. This may indicate that developers rush to get their code in before the deadline"*
>
> Os autores descrevem o que os registros de controle de versão mostram: no dia em que o branch de release é cortado, o número de pushes ao branch principal sobe claramente. Eles interpretam o pico como sinal de que os desenvolvedores correm para incluir o código antes do prazo; é a evidência, comportamental e indireta, da pressão de prazo que o ciclo de release cria.

> Seção 5.4, comparação Android × iOS: *"if a developer believes her update will not make the cut, they will only have to wait one week and hence do not force a push"*

Os autores explicam por que o Android, com corte semanal, tem proporcionalmente menos pushes no dia do corte que o iOS, com corte a cada duas semanas: quem acha que não vai chegar a tempo só espera uma semana e não força o envio. É o mecanismo do elo, apresentado pelos autores como explicação intuitiva da diferença entre plataformas, não como algo medido diretamente.

Seção 5.4, corte movido para o fim de semana: *"there is a lower tendency to rush to push updates when the cut date is on a weekend"*

Os autores observam que, depois que o corte passou de quinta-feira para domingo (agosto de 2015 no Android, fevereiro de 2016 no iOS), houve menos corrida para enviar mudanças e menos cherry-picks. A citação mostra que a corrida depende de quando cai o prazo, e não só da duração do ciclo; a mesma seção registra que os pushes no fim de semana subiram com a mudança.

Seção 7, Concluding Remarks: *"Shortening the release cycle improved this, because developers were less likely to rush their software pushes"*

Na conclusão, os autores retomam o padrão de amontoar trabalho no dia do corte, que gera software de qualidade pior, e afirmam que encurtar o ciclo melhorou a situação porque os desenvolvedores, sabendo que outra release viria logo, ficaram menos propensos a correr. É a síntese dos autores que liga diretamente a prática (ciclo mais curto) à menor pressão do prazo.

↑ **aumenta** Pressão de prazos (fator −)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Contexto: Mesmo estudo de caso no Group X. 'Tempo reduzido e falta de profundidade do trabalho' (insatisfação) e 'aumento das cadências de trabalho' (risco) aparecem tanto nos times ágeis quanto nos DevOps (Tabela 2), então não são efeito próprio do DevOps. Na discussão, os autores ligam o problema à velocidade dos ciclos: dividir o projeto em sprints permite entregar mais rápido, mas encurta etapas e não deixa tempo de pausa entre um prazo e outro. O elo vem de falas de participantes e da interpretação dos autores. Ressalva: a primeira fala é de um gerente de produção, papel que não está entre os cinco perfis amostrados.

Mecanismo: a cadência rápida de entregas impõe datas de entrega sem tempo para fazer o trabalho direito; etapas como testes são encurtadas para cumprir o prazo. **Condições:** fator comum aos modos ágil e DevOps, não específico de DevOps; os entrevistados pedem períodos de descanso entre um prazo e outro. **Evidência:** empírica direta.

Findings, Dissatisfaction, fator 2 (PM-DO): *"Deployments too fast: it's sometimes frustrating to have delivery dates imposed on us without having properly or correctly done our jobs."*

Fala de um gerente de produção (PM) de um projeto DevOps, usada para ilustrar a insatisfação 'tempo reduzido e falta de profundidade do trabalho'. 'Implantações rápidas demais' é expressão dele: é frustrante receber datas de entrega impostas sem ter feito o trabalho direito. Mostra o prazo vindo de fora e a qualidade sacrificada.

Findings, Risks, fator 2: *"Increased work cadences, resulting in work overload and tasks prioritization"*

Título do risco 'aumento das cadências de trabalho', que segundo os autores gera sobrecarga e obriga a priorizar tarefas, com risco de negligenciar algumas, como fundamentos de segurança e qualificação sistemática. Liga a cadência acelerada à pressão sobre o que dá tempo de fazer.

Discussion: *"Indeed, some steps such as test phases can be lightened to meet deadlines"*

Interpretação dos autores na discussão sobre otimizar o tempo dos projetos. Para cumprir os prazos, etapas como os testes podem ser aliviadas. É a consequência concreta da pressão de prazo sobre o trabalho.

Discussion: *"According to the interviewees, it seems important to provide rest periods between each deadline."*

No mesmo parágrafo, os autores relatam a proposta dos entrevistados: prever períodos de descanso entre um prazo e outro. Mostra que a pressão é contínua, de sprint em sprint, e indica a condição que a atenuaria.

Site Reliability Engineering

5 elos em 1 artigo(s)

> **O que é.** Abordagem criada no Google que trata a operação como problema de engenharia: profissionais (SREs) usam software e automação para manter os sistemas confiáveis. Dois conceitos centrais são o “orçamento de erro” (quanto de falha é aceitável) e a eliminação de “toil” — o trabalho operacional manual, repetitivo e sem valor duradouro. Descrição completa na seção 1.

↑ **aumenta** Alinhamento do uso do tempo (fator +)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Contexto: Estudo exploratório multimétodo do ICSE 2026: revisão de 61 fontes de literatura cinzenta e estudo de caso na ING, banco europeu que adotou SRE (Site Reliability Engineering, a abordagem do Google para confiabilidade de serviços) e faz oficinas de eliminação de toil, isto é, do trabalho manual, repetitivo, automatizável e sem valor duradouro ligado a manter serviços em produção. O estudo de caso tem 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações, recrutados entre quem passou pelas oficinas; nos códigos, E é engenheiro, C coordenador, e S, M e J indicam mais de 10, 5 a 10 e menos de 5 anos de experiência). Ao tratar dos efeitos do toil (RQ2), os entrevistados da ING concordam com a literatura cinzenta que ele reduz a velocidade e a eficiência dos times, e um coordenador sênior resume que ele afasta da engenharia de verdade. Ao tratar da eliminação (RQ3), os autores relatam que os entrevistados veem o tempo liberado voltar para tarefas de engenharia, encurtar o tempo de entrega e alimentar iniciativas novas. As afirmações sobre eliminação respondem em parte a perguntas hipotéticas e o artigo não mede tempo gasto. Os mesmos entrevistados ressalvam que o custo de manter a automação, inclusive documentação, costuma ser subestimado e que eliminar toil precisa de alto retorno para se justificar.

Mecanismo: automatizar o toil libera o tempo antes gasto em tarefa manual e o devolve ao trabalho de engenharia e a iniciativas novas, que é o que tira o engenheiro da 'engenharia de verdade'. **Condições:** o custo de manter a automação, incluindo documentação, costuma ser subestimado e consome parte do ganho; eliminação exige retorno de investimento alto para se justificar. **Evidência:** empírica direta.

> Seção 4.3.2 (efeitos do toil), CS-1: *“[toil] keeps you from the actual engineering.”*
>
> Fala de CS-1, coordenador de operações sênior (mais de 10 anos de experiência), sobre o efeito do toil: ele afasta da engenharia de verdade. Descreve o estado sem eliminação, em que o tempo vai para tarefa manual em vez do trabalho considerado próprio da função.

> Seção 4.4.2 (efeitos positivos da eliminação), síntese dos autores: *“elimination increases teams’ velocity (cf., reduces teams’ velocity) by freeing up time for engineering tasks”*
>
> Resumo dos autores do que os entrevistados disseram: a eliminação aumenta a velocidade dos times (o cf. remete ao código oposto, o toil reduz a velocidade) porque libera tempo para tarefas de engenharia. É o mecanismo do elo: o tempo volta para o trabalho que os engenheiros valorizam.

> Seção 4.4.2, síntese dos autores: *“as engineers can redirect saved time toward creative problem-solving and new initiatives”*
>
> Continuação do resumo, no efeito aumenta a inovação: os engenheiros podem redirecionar o tempo poupado para resolver problemas de forma criativa e para iniciativas novas. Mostra para onde vai o tempo liberado, segundo os entrevistados.

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Contexto: Estudo exploratório multimétodo do ICSE 2026: revisão de 61 fontes de literatura cinzenta e estudo de caso na ING, banco europeu que adotou SRE (Site Reliability Engineering, a abordagem do Google para confiabilidade de serviços) e faz oficinas de eliminação de toil, isto é, do trabalho manual, repetitivo, automatizável e sem valor duradouro ligado a manter serviços em produção. O estudo de caso tem 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações, recrutados entre quem passou pelas oficinas; nos códigos, E é engenheiro, C coordenador, e S, M e J indicam mais de 10, 5 a 10 e menos de 5 anos de experiência). O elo vem de duas partes do estudo de caso: ao definir toil (RQ1), um coordenador descreve esse trabalho como fazer a mesma coisa todo dia, como dentro de uma caixa, o que os autores leem como limite à criatividade; ao relatar efeitos da eliminação (RQ3), um engenheiro diz que ela torna a vida menos monótona e os entrevistados dizem que eliminar toil força a criatividade. Parte das perguntas sobre eliminação era hipotética (que efeitos você sentiria se gastasse menos tempo com toil), e os autores reconhecem que a passagem pelas oficinas, baseadas na visão do Google, influencia as respostas. Não há comparação medida antes e depois.

Mecanismo: eliminar o toil tira do dia a tarefa repetitiva e monótona e o ato de automatizar exige pensar fora da caixa, o que dá mais variedade e criatividade ao trabalho. **Evidência:** empírica direta.

> Seção 4.4.2 (efeitos positivos da eliminação), EJ-2: *“helps in making our life less monotonous and more flexible for something else.”*
>
> Fala de EJ-2, engenheiro de operações júnior (menos de 5 anos de experiência) na ING; a frase completa diz que a remoção do toil basicamente ajuda a tornar a vida menos monótona e mais flexível para outra coisa. Descreve o depois da prática: sem a tarefa repetitiva, o trabalho ganha variedade. A frase deu título ao artigo.

> Seção 4.4.2, síntese dos autores: *“forces creativity since eliminating toil demands thinking outside of the box as the task is far from trivial”*
>
> Resumo dos autores de um efeito relatado pelos entrevistados da ING: eliminar toil força a criatividade, porque exige pensar fora da caixa, já que automatizar a tarefa está longe de ser trivial. Mostra que a própria atividade de eliminação enriquece o conteúdo do trabalho.

> Seção 4.2.2 (definição de toil), CM-1: *“[D]oing the same thing every day and you’re sort of like in a box.”*
>
> Fala de CM-1, coordenador de operações com 5 a 10 anos de experiência, ao explicar o que é toil: fazer a mesma coisa todo dia, como se estivesse dentro de uma caixa (a frase continua: então você não pensa fora da caixa). Descreve o estado sem eliminação; os autores concluem que o toil afeta a criatividade dos funcionários.

↕ **ambivalente** Desenvolvimento de competências (fator emergente, Individual)
Kafoe et al. (2026) *— "making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*
Contexto: Estudo exploratório multimétodo do ICSE 2026: revisão de 61 fontes de literatura cinzenta e estudo de caso na ING, banco europeu que adotou SRE (Site Reliability Engineering, a abordagem do Google para confiabilidade de serviços) e faz oficinas de eliminação de toil, isto é, do trabalho manual, repetitivo, automatizável e sem valor duradouro ligado a manter serviços em produção. O estudo de caso tem 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações, recrutados entre quem passou pelas oficinas; nos códigos, E é engenheiro, C coordenador, e S, M e J indicam mais de 10, 5 a 10 e menos de 5 anos de experiência). Este elo junta os dois lados que os entrevistados da ING atribuem à eliminação: ela melhora o desenvolvimento pessoal (espelho do código da RQ2 segundo o qual o toil atrapalha o desenvolvimento), mas a automação reduz a transparência, corrói o conhecimento do sistema e tira a experiência prática que forma a habilidade de depuração. O lado negativo é coerente com o que os entrevistados disseram do toil na RQ2: fazê-lo à mão ensina os detalhes do sistema e ajuda os recém-chegados. As três citações são resumos dos autores, não falas literais de participantes, e o artigo não mede aprendizagem.
Mecanismo: o tempo liberado e o desafio de automatizar favorecem o desenvolvimento pessoal, mas a automação tira a prática direta com o sistema, reduz a transparência e corrói o conhecimento do sistema e a habilidade de depuração.
Condições: o ganho de desenvolvimento vem do tempo redirecionado para engenharia; a perda vem de a automação esconder o funcionamento que antes se aprendia fazendo à mão. **Evidência:** empírica direta.

> Seção 4.4.2 (efeitos positivos da eliminação), síntese dos autores: *“participants agree that toil elimination increases morale (cp., reduces morale) and improves personal development”*
>
> Resumo dos autores: os participantes concordam que a eliminação aumenta a moral e melhora o desenvolvimento pessoal (o cp. remete aos códigos opostos do toil, reduzir a moral e atrapalhar o desenvolvimento). Sustenta o lado positivo do elo; o artigo não detalha como o desenvolvimento ocorre, mas o associa ao tempo liberado para engenharia.

> Seção 4.4.2 (efeitos negativos da eliminação), síntese dos autores: *“hinders system comprehension as related automation reduces transparency and erodes the system knowledge”*
>
> Resumo dos autores de um efeito negativo com o qual os entrevistados concordaram: a eliminação prejudica a compreensão do sistema, porque a automação reduz a transparência e corrói o conhecimento sobre ele. É o lado em que a prática reduz o desenvolvimento de competências.

> Seção 4.4.2, síntese dos autores: *“Toil elimination also impacts debugging skills as engineers lose hands-on experience working directly with the system”*
>
> Outro efeito negativo resumido pelos autores: a eliminação afeta a habilidade de depuração, porque os engenheiros perdem a experiência prática de trabalhar diretamente com o sistema. Completa o mecanismo: o que se aprendia fazendo à mão deixa de ser aprendido.

↑ **aumenta** Insegurança no emprego (fator −)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Contexto: Estudo exploratório multimétodo do ICSE 2026: revisão de 61 fontes de literatura cinzenta e estudo de caso na ING, banco europeu que adotou SRE (Site Reliability Engineering, a abordagem do Google para confiabilidade de serviços) e faz oficinas de eliminação de toil, isto é, do trabalho manual, repetitivo, automatizável e sem valor duradouro ligado a manter serviços em produção. O estudo de caso tem 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações, recrutados entre quem passou pelas oficinas; nos códigos, E é engenheiro, C coordenador, e S, M e J indicam mais de 10, 5 a 10 e menos de 5 anos de experiência). Entre os efeitos negativos da eliminação relatados na ING, os autores registram que ela pode ameaçar o emprego de quem trabalha com toil, porque essas pessoas podem se sentir substituíveis quando a tarefa é automatizada; a literatura cinzenta traz o mesmo efeito. A evidência de entrevista é curta: um fragmento de fala sem participante identificado, com o restante formulado pelos autores como possibilidade, e trata da percepção sobre outros engenheiros, não do relato de alguém que tenha temido perder o emprego. A partir disso os autores recomendam não alocar ninguém exclusivamente em toil, para evitar rotatividade e resistência à eliminação.

Mecanismo: quem tem o trabalho centrado em toil passa a se sentir substituível quando a tarefa é automatizada. **Condições:** engenheiros alocados predominantemente em toil; os autores sugerem não alocar ninguém exclusivamente em toil para evitar turnover e resistência à eliminação. **Evidência:** empírica indireta.

> Seção 4.4.2 (efeitos negativos da eliminação), entrevistado não identificado: *“threaten [the engineers’] job”*
>
> Fragmento de fala de um entrevistado da ING inserido pelos autores no texto (o colchete é deles): a eliminação pode ameaçar o emprego dos engenheiros. O artigo não diz qual participante falou; é a única evidência de entrevista para o elo.

> Seção 4.4.2, síntese dos autores: *“engineers working on toil might feel replaceable”*
>
> Explicação dos autores para o fragmento anterior: engenheiros que trabalham com toil podem se sentir substituíveis. Dá o mecanismo, a perda da tarefa que sustentava a função, e o formula como possibilidade (might), não como caso relatado.

> Quadro-resumo da RQ3 (Seção 4.4.3): *“especially among employees dedicated to toil, who may feel threatened by automation”*
>
> Frase do quadro que resume a resposta à RQ3: a eliminação pode baixar a moral, especialmente entre funcionários dedicados ao toil, que podem se sentir ameaçados pela automação. Delimita a condição do elo: vale para quem tem o trabalho centrado em toil.

↕ **ambivalente** Senso de realização (fator +)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Contexto: Estudo exploratório multimétodo do ICSE 2026: revisão de 61 fontes de literatura cinzenta e estudo de caso na ING, banco europeu que adotou SRE (Site Reliability Engineering, a abordagem do Google para confiabilidade de serviços) e faz oficinas de eliminação de toil, isto é, do trabalho manual, repetitivo, automatizável e sem valor duradouro ligado a manter serviços em produção. O estudo de caso tem 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações, recrutados entre quem passou pelas oficinas; nos códigos, E é engenheiro, C coordenador, e S, M e J indicam mais de 10, 5 a 10 e menos de 5 anos de experiência). Toil e eliminação aparecem nos dois sentidos para a sensação de realização. Na RQ2, só no estudo de caso, entrevistados dizem que o toil pode ser recompensador por ser uma vitória rápida e algo que se sabe fazer; na RQ3, coordenadores relatam tanto a satisfação de eliminar o toil e construir a solução quanto a perda dessa recompensa imediata quando ele some. Os autores resumem a ambivalência no próprio título, vida menos monótona contra só riscar coisas da lista. As falas vêm de poucos entrevistados, quase todos coordenadores, e não há medida de satisfação.

Mecanismo: construir a automação que elimina o toil dá a satisfação de resolver o problema; mas eliminar o toil também tira a tarefa rápida e conhecida que permite 'riscar coisas da lista' e sentir que o dia rendeu. **Condições:** aumenta para quem constrói a solução de eliminação; reduz para quem se sente valorizado ao fazer toil ou não gosta do processo de eliminação. **Evidência:** empírica direta.

> Seção 4.4.2 (efeitos positivos da eliminação), CM-2: *“it’s obviously the satisfaction [that] comes from eliminating [toil] and building solutions”*
>
> Fala de CM-2, coordenador de operações com 5 a 10 anos de experiência: a satisfação vem de eliminar o toil e construir soluções. Os autores a usam para mostrar que eliminar toil é recompensador; é o lado em que a prática aumenta o senso de realização, para quem constrói a automação.

> Seção 4.4.2 (efeitos negativos da eliminação), síntese dos autores: *“toil elimination reduces morale given that some engineers feel valued when performing toil”*

Resumo dos autores: a eliminação reduz a moral porque alguns engenheiros se sentem valorizados ao fazer toil ou simplesmente não gostam do processo de eliminação. Mostra o lado negativo e sua condição: depende de a pessoa tirar valor da própria execução do toil.

Seção 4.4.2, CM-3: *“[y]ou can do [toil] quickly and without a lot of thought, and you can just tick things off, and sometimes that really helps with how you perceived your day to have gone”*
Fala de CM-3, coordenador de operações com 5 a 10 anos de experiência, que ilustra o lado negativo: o toil se faz rápido e sem muito pensar, dá para ir riscando itens da lista, e às vezes isso ajuda muito na percepção de como o dia rendeu. Eliminar o toil tira essa fonte de sensação de dia produtivo; a expressão just tick things off também está no título.

Automação de testes

4 elos em 4 artigo(s)

> **O que é.** Os testes que verificam se o software funciona são escritos como programas e executados automaticamente, a cada mudança, em vez de alguém testar à mão. É o que torna possível integrar e entregar com frequência sem perder qualidade. Descrição completa na seção 1.

↑ **aumenta** Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)
Andreyev (2025) *— The Art of Managing Flaky Tests at Scale*
Contexto: Relato de experiência de duas páginas (workshop CCIW, no ICSTW 2025), de um único autor da Okta, sobre uma iniciativa de um ano para controlar testes flaky na CI de um monorepo. Um sistema interno lê os resultados dos testes e marca como flaky o teste que ultrapassa um limiar configurável de falhas, o que abre um item de ação para o time responsável. Antes dos ajustes, a maioria das resoluções era desabilitar o teste e deixar a investigação para depois, e o número de testes desabilitados crescia sem estratégia para revertê-lo. Ao longo do ano o autor variou os limiares e acompanhou a fração de testes corrigidos já na primeira investigação, que subiu de 23,9% para 44,8%. O elo com a fadiga de alertas é a leitura do autor sobre essa única métrica: não há medida de fadiga, de carga ou da percepção dos times, nem números de times, testes ou alertas, e no mesmo período houve outras iniciativas (dashboards, calendário de incidentes da CI, treinamentos, incentivos).
Mecanismo: cada teste flaky que passa do limiar vira item de ação com prazo de um dia útil; com limiar agressivo, os alertas excedem a capacidade dos times para dívida técnica, eles se dessensibilizam e passam a desabilitar o teste em vez de corrigi-lo. **Condições:** limiares de falha agressivos; ajustar o limiar à capacidade dos times atenuou o efeito (correções na primeira investigação quase dobraram); limiar frouxo demais deixa o teste instável no pipeline. **Evidência:** empírica indireta.

Página 1: *“Action items must be addressed within one business day, as all required tests must pass before a change can be merged.”*
Descreve a regra do processo de alertas: cada teste flaky marcado pelo sistema vira item de ação a ser resolvido em até um dia útil, porque nenhuma mudança pode ser integrada com um teste obrigatório falhando. É o que transforma cada alerta em demanda de prazo curto para o time, a base do mecanismo do elo.

Página 1: *“overly aggressive alerting thresholds led to a counterproductive "resolve now, investigate later" mentality”*
Achado do autor ao variar os limiares: limiares agressivos demais levaram a uma mentalidade contraproducente de resolver agora e investigar depois, em que, na primeira análise, o teste era desabilitado temporariamente em vez de estabilizado. Mostra o comportamento que o autor associa ao excesso de alertas; o que foi medido é a proporção de correções, e a mentalidade é interpretação dele.

Página 1: *“alert fatigue significantly impacts long-term process adherence”*
Conclusão apresentada pelo autor como uma percepção central da iniciativa: a fadiga de alertas afeta muito a adesão ao processo no longo prazo. Nomeia o fator emergente do elo, sem medida própria de fadiga.

Página 1: *“Teams have a finite capacity for addressing technical debt, and excessive alerts can lead to desensitization”*
Explicação do autor para o fenômeno: os times têm capacidade finita para tratar dívida técnica, e alertas em excesso levam à dessensibilização. É o mecanismo do elo: o volume de alertas ultrapassa a capacidade, e os times deixam de responder com correção.

Página 1: *“teams in aggregate nearly doubled - from 23.9% to 44.8% - by fine-tuning flaky test thresholds”*
Único dado quantitativo do relato: somando os times, a fração de testes flaky corrigidos já na investigação inicial quase dobrou, de 23,9% para 44,8%, depois do ajuste fino dos limiares. Mostra o outro lado do elo: com alertas ajustados à capacidade, os times corrigem mais e desabilitam menos. O texto não informa o número de testes, os limiares usados nem o peso das outras iniciativas simultâneas.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Contexto: Mesmo estudo de três entrevistas na Solita. Segundo a Tabela 1, os projetos A e C tinham testes end-to-end automatizados executados em navegadores reais, e o projeto B não; nos três, os testes faziam parte do pipeline e os programadores escreviam e alteravam testes junto com as novas funcionalidades, sem achar isso penoso. O elo vem de relatos, resumidos pelos autores, de que esse tipo específico de teste era às vezes um incômodo: demorado, instável por causa dos frameworks e consumidor de tempo para ficar confiável. Não há medida do tempo gasto, e a reação de restringir esses testes à funcionalidade central é contada por um único entrevistado.
Mecanismo: testes end-to-end automatizados em navegador real demoram mais de uma hora e falham às vezes sem motivo lógico, pela instabilidade dos frameworks de teste, e consomem tempo extra para torná-los confiáveis. **Condições:** específico de testes end-to-end em navegador real (projetos A e C); um time passou a escrever testes de navegador só para a funcionalidade central, como forma de conter o custo. **Evidência:** empírica direta.

> Seção 4.1 (projetos A e C): *"Especially automated end-to-end tests that are run with real browsers were mentioned as being a pain for the"*
>
> Os autores relatam que os entrevistados citaram os testes end-to-end automatizados, executados em navegadores reais, como um incômodo (pain) para os desenvolvedores em alguns momentos. Delimita o elo: não se trata da automação de testes em geral, que no parágrafo anterior aparece como fonte de segurança, mas desse tipo de teste.

> Seção 4.1 (projetos A e C): *"Though important, such tests may take more than an hour to run and may fail occasionally without any logical reason because of the unreliability of the test frameworks being used. Often some extra time was spent to improve tests and achieve better reliability."*
>
> Descreve o problema concreto: embora importantes, esses testes podem levar mais de uma hora para rodar e falhar ocasionalmente sem razão lógica, porque os frameworks de teste usados não são confiáveis, e muitas vezes se gastava tempo extra para melhorá-los. É o mecanismo do elo: testes instáveis viram trabalho de manutenção do ferramental, que tira tempo do desenvolvimento.

> Seção 4.1, um entrevistado: *"they stopped writing new browser tests for other than just core functionality at one point."*
>
> Um entrevistado contou que, a certa altura, o time parou de escrever novos testes de navegador para algo além da funcionalidade central. Mostra que o custo era alto o bastante para mudar a prática e funciona como condição do elo: limitar esses testes ao essencial foi a forma de conter a manutenção.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Parry et al. (2022) *— Surveying the Developer Experience of Flaky Tests*
Contexto: Survey online (ICSE-SEIP 2022) com 170 respondentes recrutados no Twitter, no LinkedIn e num fórum regional de empresas de tecnologia de Sheffield, mais a análise temática de 38 threads do StackOverflow sobre testes flaky, isto é, testes automatizados que passam e falham sem mudança no código. As perguntas fechadas usaram escalas de quatro pontos, pontuadas de 0 a 3, e os autores compararam as médias de quem vê testes flaky ao menos mensalmente (pouco mais da metade da amostra) com as de quem os vê menos. O elo junta três resultados: a concordância com a perda de produtividade (SQ4), a frequência das ações tomadas diante de um teste flaky (SQ7) e uma resposta aberta sobre falta de tempo para consertá-los (SQ11). Ressalvas: é percepção autorrelatada, os autores não puderam verificar se todos os respondentes eram de fato desenvolvedores, e as diferenças entre grupos são comparações de médias sem teste estatístico.
Mecanismo: testes automatizados que passam e falham sem mudança no código geram falhas falsas que custam produtividade: o desenvolvedor reexecuta o build, tenta reparar um teste cuja causa muitas vezes não se descobre e acaba sem tempo para consertar todos. **Condições:** depende da frequência com que a suíte apresenta testes instáveis: quem os vê ao menos mensalmente reexecuta mais, tende mais a não agir e repara menos; causas mais citadas são setup/teardown inadequado, dependência de rede e causas desconhecidas. **Evidência:** empírica direta.

> RQ2, conclusão (SQ4): *"They also agree that flaky tests lead to both a loss of productivity and a reduction in testing efficiency."*
>
> Conclusão dos autores para a RQ2 (impactos), a partir das notas médias de concordância com oito afirmações. Perda de produtividade (SQ4.3, média 2,50 numa escala de 0 a 3) e redução da eficiência dos testes (SQ4.2, 2,47) ficaram em segundo e terceiro lugar, logo atrás do impacto sobre a CI. Sustenta o custo de produtividade do elo; é concordância declarada, não produtividade medida.

> RQ4 (SQ7): *"The most common action as scored by all participants was to simply re-run the failing build (SQ7.2)"*
>
> Resultado da pergunta SQ7, que pedia a frequência de oito ações depois de identificar um teste flaky. Reexecutar o build foi a mais comum (média 2,67 de 3), seguida de tentar reparar o teste (2,41); as demais ficaram bem abaixo. Mostra a forma concreta do trabalho extra: a resposta típica à falha falsa é reiniciar o build e esperar que o teste passe dessa vez, como o próprio questionário descreve a ação.

RQ4 (SQ7.1): *“This suggests that developers who experience flaky tests more often are more likely to take no action.”*
Comparação entre grupos na mesma pergunta: quem vê testes flaky ao menos mensalmente deu nota 1,40 a não tomar nenhuma ação, contra 0,91 de quem os vê menos, e também tenta reparar menos (2,23 contra 2,64). Os autores concluem que a exposição frequente favorece a inação. Mostra a condição do elo: quanto mais instável a suíte, mais os testes se acumulam sem conserto.

Tabela 1, SQ11t8 (P64): *“We haven’t got the time to address them all.”*
Fala do participante P64 na pergunta aberta final (SQ11), usada como exemplo do tema SQ11t8, não vale a pena consertar, em que o esforço de reparar testes flaky parece grande demais para o ganho. P64 diz que o time não tem tempo para tratar todos. Mostra que a manutenção da suíte excede o tempo disponível, o que fecha o mecanismo do elo.

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Ahmad et al. (2023) *— An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration*
Contexto: Estudo de caso na Axis Communications, empresa sueca de vídeo-vigilância em rede com mais de 3.000 funcionários, cujos produtos combinam muitas features configuráveis. Com a adoção de integração contínua, os testes de regressão foram automatizados e passaram a rodar ao menos uma vez por dia. Os dados vêm de um único grupo focal de três horas com quatro profissionais seniores de teste (dois engenheiros de automação de testes, um arquiteto de testes e um supervisor de testes de regressão), interpretado por análise temática; o objetivo era avaliar a adoção da priorização de testes por diversidade (DBT), e o fardo de manter a suíte aparece entre os desafios atuais relatados (C3, testes antigos; C5, mudança das diretrizes de escrita; C8, testes instáveis). Não há medida de esforço, e os próprios autores reconhecem que a amostra é pequena, restrita a papéis de teste e a uma empresa.
Mecanismo: a suíte de regressão automatizada, executada ao menos uma vez por dia, acumula testes antigos ou desatualizados que falham quando as regras de escrita mudam ou quando a rede do ambiente de teste falha, e a checagem e atualização é feita à mão, um fardo para a equipe. **Condições:** produto configurável com muitas combinações de features; diretrizes de escrita de teste que mudam ao longo do tempo; pouca documentação de processo; ambiente de teste com rede instável; checagem manual das inconsistências; relato de 4 testadores seniores de uma empresa. **Evidência:** empírica direta.

Seção 4.2, Tema 1 (atividades de teste), síntese dos autores: *“Identifying and handling old or outdated test cases (C3) is also one of their challenges with continuous testing”*
Frase dos autores que apresenta o desafio C3: identificar e tratar casos de teste antigos ou desatualizados é um dos problemas da equipe com o teste contínuo. Eles explicam que o produto muda o tempo todo e que testes não revisados geram inconsistências e redundâncias; é o estado que a prática produz quando a suíte cresce sem curadoria.

Seção 4.2, Tema 1, P1: *“It’s a burden on us.”*
Final da fala de P1, engenheiro de automação de testes há 5 anos na empresa, responsável pelas ferramentas e pela infraestrutura de automação. Antes ele diz que não querem manter um teste antigo que nunca falhou e de que ninguém cuida; a frase citada chama esse acúmulo de fardo para a equipe. É a evidência direta de que manter a suíte pesa sobre quem trabalha.

Seção 4.2, Tema 2 (fatores organizacionais), P3: *“sometimes all the test cases fail and we look at them and we see for obvious reason that this is a very good test case written three years ago and they changed the rules during the time”*
Fala de P3, arquiteto de testes há 8 anos na empresa, ilustrando o desafio C5: às vezes todos os testes falham e a equipe percebe que é um bom teste escrito três anos antes, mas as regras de escrita mudaram nesse meio-tempo; ele continua dizendo que tentam manter a suíte inteira atualizada com as regras. Mostra um mecanismo do fardo: mudar as diretrizes obriga a revisar testes antigos.

Seção 4.2, Tema 3 (infraestrutura de teste), P1: *“Most of the time when we have flaky test cases, it is because of the test environment, the network really not working as it should.”*
Fala de P1 sobre o desafio C8, testes instáveis (flaky, que passam ou falham sem mudança no código): na maioria das vezes a causa é o ambiente de teste, a rede que não funciona como deveria. Mostra outro mecanismo: falhas de infraestrutura produzem falsos alarmes que a equipe precisa investigar.

Seção 5 (discussão), vantagem apoiar a manutenção de artefatos de teste, síntese dos autores: *“Currently, these checks are done manually when issues during execution arrive (e.g., failures), which is detrimental to stability of test cycles (SI2).”*
Frase dos autores na discussão: hoje a checagem de inconsistências no repositório de testes é feita à mão, quando surgem problemas na execução, como falhas, o que prejudica a estabilidade dos ciclos de teste. SI2 é o interesse de stakeholder do modelo TAS (Test Activity Stakeholders) de garantir estabilidade e integridade do sistema durante o desenvolvimento. Confirma que a manutenção é manual e reativa.

Times multifuncionais Dev–Ops

4 elos em 1 artigo(s)

> **O que é.** Desenvolvedores e profissionais de operação passam a trabalhar no mesmo time, com objetivos comuns, em vez de em departamentos separados que se comunicam por chamados. É a dimensão organizacional e cultural do DevOps. Descrição completa na seção 1.

↓ **reduz** Apoio do gestor (fator +)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Contexto: Mesmo estudo de caso no Group X, que tem duas linhas hierárquicas separadas, uma de desenvolvimento e outra de operações, e times que trabalham à distância; os próprios autores dizem que essas condições não eram favoráveis ao DevOps. O time DevOps junta pessoas das duas linhas e funciona por conta própria, fora da cadeia de comando de cada uma. A falta de retorno da hierarquia aparece como insatisfação específica dos times DevOps (Tabela 2) e é agrupada pelos autores sob 'falta de apoio organizacional'. O elo vem de uma única fala de gerente, da interpretação dos autores sobre o motivo e de uma comparação com outras empresas citada na conclusão sem dados nem método descritos; por isso a evidência é indireta.

Mecanismo: o time DevOps reúne gente de várias linhas e funciona à margem das linhas hierárquicas; os chefes de cada linha perdem o controle do time e deixam de dar retorno e atenção. **Condições:** empresa com duas linhas hierárquicas separadas (Dev e Ops) e times remotos; nas comparações com outras empresas, com linha hierárquica unificada e painel comum, o feedback da hierarquia era bem maior. **Evidência:** empírica indireta.

> Findings, Dissatisfaction, fator 6: *“DevOps teams are composed of members of various trade and has an own operating mode that is independent of the hierarchical lines and traditional craft.”*
>
> Descrição dos autores para o fator de insatisfação 'falta de retorno da hierarquia'. O time DevOps reúne pessoas de vários ofícios e tem modo de funcionar próprio, independente das linhas hierárquicas e das profissões tradicionais. É a condição estrutural que afasta o time dos chefes.

> Findings, Dissatisfaction, fator 6 (DM-DO): *“My hierarchy is not very present, so I don’t have too much feedback from my boss.”*
>
> Fala de um gerente de departamento (DM) de um projeto DevOps. Ele diz que sua hierarquia está pouco presente e que recebe pouco retorno do chefe. Mostra o efeito do lado de quem o sente, e que até um gestor fica sem apoio de cima.

> Dissatisfaction and risks: *“Since the DevOps project team is multi-line of services, the superiors have no control over the team and may seem overwhelmed by this cross-functional mode of operation”*
>
> Interpretação dos autores no subtema 'falta de apoio organizacional'. Como o time atravessa várias linhas de serviço, os superiores não têm controle sobre ele e parecem sobrecarregados por esse modo transversal de funcionar, o que gera a falta de retorno. É o mecanismo proposto, não algo que os entrevistados tenham dito.

> Conclusion: *“Comparisons we made with other firms showed that when there is a unified hierarchical line with a common dashboard, feedback from the hierarchy was much more important.”*
>
> Na seção de limitações, os autores dizem que, comparando com outras empresas que têm uma linha hierárquica única e um painel de indicadores comum, o retorno da hierarquia era bem maior. Isso indica que o efeito depende da estrutura com duas linhas separadas; o artigo não descreve como essa comparação foi feita.

↑ **aumenta** Conflito de papéis (fator −)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Contexto: Mesmo estudo de caso no Group X. Os engenheiros de produção são profissionais de operações que, no modo DevOps, passam a integrar os times de projeto, mas continuam responsáveis pela operação e pelos incidentes de outras aplicações. A 'sobrecarga do operador (projeto e gestão de incidentes)' é uma das três insatisfações que só apareceram nos times DevOps (Tabela 2), e os autores a agrupam sob 'gestão de tempo complexa'. O elo vem da descrição dos autores, de uma fala de engenheiro de produção e da interpretação dos autores sobre a exigência de disponibilidade do DevOps; o artigo não diz quantos operadores relataram o problema ('algumas pessoas da produção').

Mecanismo: o operador integrado ao projeto DevOps continua responsável pela operação e pelos incidentes; tem de tocar projeto e incidente em paralelo e o incidente atrapalha as tarefas em que queria se concentrar. **Condições:** operadores que mantêm suporte de nível 1 em outras aplicações; os autores ligam o efeito à alta disponibilidade diária exigida pelo modo DevOps. **Evidência:** empírica direta.

Em revisão: O artigo enquadra este tema como sobrecarga ('operator overload', 'overload of work') e como gestão de tempo complexa, não como demandas incompatíveis entre papéis. Conflito de papéis é leitura defensável, porque o operador acumula o papel de membro do projeto e o de responsável por incidentes, mas Carga de trabalho talvez seja o fator mais fiel

ao texto, ou caiba um segundo elo com ele.

Findings, Dissatisfaction, fator 5: *“Some production people also complain about the overload of work in their department, since they are responsible for DevOps projects and incident management to be conducted in parallel.”*
Descrição dos autores para o fator de insatisfação 'sobrecarga do operador'. Algumas pessoas de produção se queixam do excesso de trabalho porque respondem ao mesmo tempo pelos projetos DevOps e pela gestão de incidentes. Mostra as duas frentes de trabalho simultâneas que dão base ao elo.

Findings, Dissatisfaction, fator 5 (PE-DO): *“We have applications on which we are still operating level 1. We do a lot of things at once, there are tasks on which we would like to focus ... sometimes we are a little disturbed by the incident.”*
Fala de um engenheiro de produção de um projeto DevOps. 'Operar em nível 1' quer dizer fazer o primeiro atendimento de incidentes em aplicações que continuam sob sua responsabilidade; ele faz muitas coisas ao mesmo tempo e os incidentes interrompem as tarefas em que queria se concentrar. É o relato concreto do choque entre o papel no projeto e o papel na operação.

Dissatisfaction and risks: *“The DevOps approach requires a very high level of availability and involvement on a daily basis, which makes it difficult for the operators to manage projects and incidents concomitantly”*
Interpretação dos autores no subtema 'gestão de tempo complexa'. Eles atribuem a dificuldade de conciliar projeto e incidentes ao alto nível de disponibilidade e envolvimento diário que a abordagem DevOps exige. É a condição que, segundo os autores, produz o efeito.

↕ **ambivalente** Reconhecimento no trabalho (fator +)
Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*
Contexto: Mesmo estudo de caso no Group X (54 entrevistas com cinco papéis em 11 projetos ágeis e DevOps, análise temática). O reconhecimento aparece dos dois lados: 'papel reconhecido' é fator de satisfação, sobretudo para arquitetos e engenheiros de produção, enquanto 'papéis expostos a crítica' é fator de insatisfação que só surgiu nos times DevOps e atinge product owners e release managers, papéis novos que nasceram com a abordagem. Os autores explicam que esses papéis são transversais e sem posição hierárquica, o que cria uma 'dupla estrutura hierárquica' e incerteza sobre o lugar deles. O elo junta falas de participantes e a interpretação dos autores na seção sobre insatisfação e riscos. Ressalva: pela Tabela 2, 'papel reconhecido' é fator comum ao ágil e ao DevOps, então o lado positivo não é exclusivo do DevOps; só o lado negativo é.
Mecanismo: o time misto dá visibilidade ao trabalho de Ops, que passa a ser reconhecido pelos demais; ao mesmo tempo, os papéis novos e transversais de PO e RM ficam expostos e precisam justificar suas escolhas de método diante dos outros. **Condições:** aumenta para engenheiros de produção e arquitetos, cujo papel ganha lugar no time; reduz para PO e RM, papéis transversais e sem linha hierárquica, julgados pela abordagem experimental. **Evidência:** empírica direta.

Findings, Satisfaction, fator 4 (PE-DO): *“We are more recognized than before”*
Trecho da mesma fala do engenheiro de produção de um projeto DevOps usada no elo de relações. Aqui importa o começo: ele se sente mais reconhecido do que antes, isto é, do que antes de operações fazer parte do time. É o lado em que o reconhecimento aumenta.

Findings, Dissatisfaction, fator 4: *“They sometimes feel judged by others in their innovative project management approach, feeling that they have to justify their choices.”*
Descrição dos autores para o fator de insatisfação 'papéis expostos a crítica', referente a release managers e product owners. Eles se sentem julgados pelos colegas por conduzirem o projeto de modo inovador e precisam justificar suas escolhas. É o lado em que o reconhecimento diminui.

Findings, Dissatisfaction, fator 4 (PO-DO): *“Sometimes I feel that I have to justify our choices, and finally the methodology, the impression that we are judged in fact.”*
Fala de um product owner (PO, quem prioriza o que o time constrói) de um projeto DevOps. Ele diz que às vezes precisa justificar as escolhas do time e até a própria metodologia, com a impressão de estar sendo julgado. É o relato em primeira pessoa da exposição que os autores descrevem.

Dissatisfaction and risks: *“This factor of dissatisfaction hinders the overall satisfaction felt by a majority of team members about the role to play and its recognition.”*
Interpretação dos autores no subtema 'papéis mais expostos', um dos quatro grupos de insatisfação e risco próprios do DevOps. Afirmam que essa exposição prejudica a satisfação que a maioria do time sente com o papel que exerce e com o reconhecimento dele. É o que liga a crítica sofrida por PO e RM ao fator reconhecimento e justifica o efeito ambivalente.

↑ **aumenta** Relações no trabalho (fator +)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Contexto: Estudo de caso qualitativo no Group X, grande empresa europeia de serviços com 15 mil pessoas em TI, que usa métodos ágeis há mais de 15 anos e DevOps há 8; em 2017 os autores fizeram 54 entrevistas de cerca de 90 minutos com cinco papéis (gerente de departamento, product owner, arquiteto, engenheiro de produção e release manager) em 11 projetos, uns em modo ágil e outros em modo DevOps, e fizeram análise temática. Segundo os autores, sem DevOps o ágil fica restrito ao desenvolvimento e mantém o silo entre quem constrói (Build) e quem opera (Run); no modo DevOps, membros de operações entram no time de projeto desde o início. O elo vem de falas de participantes e da Tabela 2, que separa temas comuns aos dois modos de temas que só apareceram num deles: a riqueza da colaboração entre papéis aparece nos dois, mas a dinâmica de grupo e a capacidade coletiva de colaborar só nos times DevOps. A comparação é pela presença de temas nos discursos, sem contagem nem teste estatístico.

Mecanismo: a entrada de Ops no time de projeto multiplica as trocas e as relações entre papéis e une o time; o engenheiro de produção passa de executor a parceiro. **Condições:** efeito mais forte no modo DevOps que no ágil: nos times ágeis, com o silo Dev/Ops ainda de pé, Ops relatam expectativa frustrada de colaboração com Dev; a dispersão geográfica do time é desfavorável, e mais ainda em DevOps. **Evidência:** empírica direta.

> Findings, Satisfaction, fator 4 (PE-DO): *“We are more recognized than before and now there are more exchanges, there are more relationships; we went from being a performer to a partner position.”*
>
> Fala de um engenheiro de produção (PE, profissional de operações que também faz testes) de um projeto em modo DevOps (DO), usada pelos autores para ilustrar o fator de satisfação 'papel reconhecido'. Ele compara com o período anterior: agora há mais trocas e relações com o resto do time, e o papel de operações passou de mero executor ('performer') a parceiro. É o estado depois da prática, contado por quem vem de operações.

> Findings, Satisfaction, fator 9: *“Through the DevOps approach, the teams seem unified”*
>
> Síntese dos autores para o fator de satisfação 'capacidade coletiva de colaborar e dinâmica de grupo', tirada dos discursos dos times DevOps, que descrevem a equipe como unida, amigável, competente e com boa energia. Mostra o efeito no nível do time inteiro, não só de um papel; é a interpretação dos autores a partir dos termos usados pelos entrevistados.

> Table 2: *“DevOps practices: group dynamics and the collective ability to collaborate”*
>
> Linha da Tabela 2 em que esse fator aparece na coluna de fatores específicos de um modo, marcado como prática DevOps: o tema surgiu nos times DevOps e não nos ágeis. É a base para dizer que o ganho nas relações é mais forte com DevOps do que só com ágil.

> Specific Dissatisfaction for Agile Teams (RM-AG): *“they do not listen to us, because the voice of the run is not important, whereas it should be!”*
>
> Fala de um release manager (RM, responsável por levar versões à produção, do lado de operações) de um projeto em modo ágil (AG), onde Dev e Ops continuam em silos. Ele se queixa de que os desenvolvedores não escutam operações ('a voz do run', isto é, da operação). Os autores registram isso como a única insatisfação específica do modo ágil, a expectativa frustrada de colaboração mais estreita; serve de contraste, mostrando a situação sem a integração Dev–Ops.

> Discussion: *“this dispersion appears as unfavorable (although not an impossible barrier to overcome) to agile and even more to DevOps practices”*
>
> Interpretação dos autores, na discussão sobre coesão do time, a partir do fator de insatisfação 'distância geográfica', comum aos dois modos. Dizem que a dispersão dos membros atrapalha a formação de equipe e é desfavorável ao ágil e mais ainda ao DevOps, embora não seja barreira intransponível. É a condição que enfraquece o elo.

Gestão de incidentes

3 elos em 1 artigo(s)

> **O que é.** O processo organizado para responder quando o sistema falha em produção: quem é acionado, quem coordena, como se diagnostica e como se restaura o serviço. Inclui os roteiros escritos (runbooks) que guiam o diagnóstico. Descrição completa na seção 1.

↑ **aumenta** Carga cognitiva (fator −)

Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*

Contexto: O elo vem da mesma rodada de entrevistas curtas com 10 engenheiros de plantão da Microsoft (Seção 4.4), na pergunta sobre os desafios enfrentados com os TSGs. Na empresa, o TSG é um documento (Word, OneNote, Markdown) com comandos, consultas e instruções em linguagem natural que o plantonista precisa ler, interpretar e executar à mão, copiando e colando comandos; a introdução do artigo afirma que essa execução manual exige entender as instruções e traz risco de erro humano e de fadiga de plantão. A situação descrita é a de um serviço muito customizado, em que o guia genérico não se aplica diretamente a cada incidente. O elo se apoia em um único participante e é relato, sem medida de carga cognitiva.

Mecanismo: o runbook manual não garante a solução, e o plantonista precisa julgar, incidente a incidente, quais passos se aplicam. **Condições:** serviço muito customizado; TSG manual (não executável). **Evidência:** empírica direta.

> Seção 4.4, Q2 (P6): *“I work on a service with a lot of customization. So when incidents arise, I have to manually verify which steps in a TSG will work for that incident. There is no guaranteed solution.”*
>
> P6 é engenheiro de plantão de um serviço com muita customização. Relata que, a cada incidente, precisa verificar manualmente quais passos do TSG valem para aquele caso, porque o guia não garante solução. A citação mostra o mecanismo do elo: o runbook manual deixa para o plantonista, sob a pressão do incidente, o julgamento sobre o que aplicar, em vez de lhe entregar um procedimento que funcione diretamente.

↓ **reduz** Carga de trabalho (fator −)

Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*

Contexto: Estudo industrial na Microsoft (versão arXiv do artigo do ESEC/FSE 2022) em que os autores mineraram, ao longo de 4 meses, os cliques dos engenheiros de plantão nos links de TSGs (Troubleshooting Guides, guias de diagnóstico e mitigação que o plantonista segue ao tratar um incidente), ligando milhares de incidentes (o número exato não foi divulgado por política da empresa) a cerca de 4.800 TSGs. Os TSGs já são prática estabelecida na empresa, com mais de 50 mil guias escritos pelos próprios times e executados à mão pelo plantonista; por isso não há antes/depois da adoção, e sim a comparação entre incidentes que tiveram ou não um TSG aberto durante o tratamento. O elo vem dessa comparação de médias do tempo até a mitigação (TTM), restrita aos incidentes de severidade 1 e 2, que acionam o plantonista na hora, e da premissa dos autores de que esse tempo mede o esforço de plantão. Ressalvas: o artigo fala em diferença significativa, mas não relata teste estatístico nem controle de diferenças entre os dois grupos de incidentes; o esforço não é medido diretamente; e o texto sobre a severidade 1 traz os valores aparentemente trocados (cerca de 36 h com TSG contra 2 h sem), de modo que só a quebra da severidade 2 (de cerca de 18 h para 13 h) é legível.

Mecanismo: o runbook (TSG) ligado ao incidente encurta o tempo de mitigação dos incidentes que acionam o plantonista, e com isso o esforço de plantão gasto por incidente. **Condições:** incidentes de severidade 1 e 2 (com paging); TTM usado pelos autores como proxy do esforço de plantão. **Evidência:** empírica indireta.

> Seção 2.1 (TSGs and Time-to-mitigate): *“TTM, is a reliable proxy for on-call effort”*
>
> Afirmação dos autores que justifica usar o TTM (time-to-mitigate, tempo entre a abertura do incidente e sua mitigação) como medida do esforço do plantonista. Vale só para os incidentes de severidade 1 e 2, que disparam alerta imediato (paging) ao engenheiro de plantão, de modo que o relógio corre enquanto ele trabalha no problema. É a ponte entre o dado medido (tempo) e o fator do elo (carga de trabalho), e é uma premissa, não algo verificado no artigo.

> Seção 2.1 (TSGs and Time-to-mitigate): *“We find that the mean TTM of incidents without TSGs (≈19 hrs) is significantly higher than that of incidents linked to a TSG (≈13 hrs).”*
>
> Resultado principal da mineração: entre os incidentes de severidade 1 e 2, os que tinham um TSG vinculado (identificado pelo clique do plantonista no link do guia) foram mitigados em média em cerca de 13 horas, contra cerca de 19 horas dos que não tinham. A diferença de aproximadamente 6 horas por incidente é o que os autores leem como esforço de plantão poupado pelo runbook. É comparação observacional de médias, sem controle de outras diferenças entre os incidentes.

> Seção 2.1 (TSGs and Time-to-mitigate): *“TSGs tend to significantly reduce effort during incident mitigation”*
>
> Conclusão dos autores ao fim da Seção 2.1, depois de olhar também a quebra por severidade (Figura 1c). Traduz a diferença de TTM em redução do esforço de mitigação, o que fecha o elo entre o uso do runbook e a menor carga de

trabalho do plantonista. É interpretação: o esforço não foi medido em horas trabalhadas, e sim inferido do tempo até a mitigação.

↑ **aumenta** Interrupções no trabalho (fator −)
Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*
Contexto: O elo vem do estudo com usuários no fim do artigo (Seção 4.4): os autores sortearam 30 engenheiros de plantão da Microsoft que tinham mitigado incidentes nos seis meses anteriores, e 10 deles, de 8 times diferentes e com média de 1.852 incidentes tratados, aceitaram entrevistas semiestruturadas de cerca de 15 minutos, com respostas registradas literalmente. As perguntas sobre frequência de uso dos TSGs (Q1) e sobre os desafios que eles trazem (Q2) vieram antes da apresentação da ferramenta AutoTSG. O pano de fundo é a baixa qualidade dos guias manuais: numa pesquisa interna sobre a experiência de plantão, qualidade e cobertura dos TSGs foi a principal dor entre 19 dimensões, e a análise de 428 feedbacks deixados nos próprios guias mostrou como problemas mais frequentes a falta de informação, os links quebrados e os passos incorretos. O elo vale, portanto, para TSGs ruins, e se apoia nas falas de dois entrevistados, sem nenhuma medida de quantas interrupções ocorrem.
Mecanismo: quando o runbook é ruim, confuso ou desatualizado, quem está de plantão recorre aos colegas, e quem conhece o sistema passa a ajudar os pares todos os dias, fora da própria escala. **Condições:** TSGs de baixa qualidade (incompletos, desatualizados, confusos). **Evidência:** empírica direta.

> Seção 4.4, Q1 (P2): *“My usual on-call rotation is bi-weekly. But I end up helping my peers use TSGs every day, because they are bad and confusing.”*
>
> P2 é um dos 10 engenheiros de plantão entrevistados e o único que disse usar TSGs todos os dias; os pesquisadores perguntaram por quê. Ele explica que sua escala de plantão é quinzenal, mas que ajuda os colegas a usar os guias diariamente, porque eles são ruins e confusos. É a evidência direta de que o runbook de baixa qualidade transforma quem conhece o sistema em apoio permanente dos pares, interrompido fora do próprio turno.

> Seção 4.4, Q2 (P7): *“Mostly outdated TSGs or missing the information I need. So I generally discuss with my team for many of the issues.”*
>
> P7, outro engenheiro de plantão, respondendo à pergunta sobre os desafios com os TSGs. Diz que os guias costumam estar desatualizados ou sem a informação de que precisa e que, por isso, discute a maior parte dos problemas com o time. Mostra o outro lado do mecanismo descrito por P2: o plantonista que não encontra a resposta no runbook recorre aos colegas, e são esses pedidos que interrompem quem não está de plantão.

Responsabilidade de operação pelo time

3 elos em 2 artigo(s)

> **O que é.** O mesmo time que desenvolve o software também o mantém funcionando em produção, em vez de entregá-lo a um departamento separado de operações. É a ideia central do DevOps: “quem constrói, opera”. Descrição completa na seção 1.

↑ **aumenta** Conflito trabalho-família (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Contexto: Estudo qualitativo do ICSE-SEIP 2026 com 26 entrevistas semiestruturadas (45 a 60 minutos, em alemão, citações traduzidas para o inglês pelos autores) com membros de times DevOps de duas empresas suíças reguladas: uma seguradora global (Caso 1, participantes 1 a 13) e um banco (Caso 2, participantes 14 a 26), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança e incerteza tecnológicas). Os autores descrevem que membros dos times DevOps, responsáveis também por colocar o software em produção, eram escalados fora do horário normal; o caso típico eram as releases mensais e os lançamentos urgentes, cuja implantação ia para a noite ou o fim de semana e ficava com quem estivesse em melhor posição. O artigo não compara com um período anterior ao DevOps: o elo vem da síntese dos autores e de duas falas, de um gerente de projeto ágil do banco e de um desenvolvedor da seguradora, classificadas como tecno-invasão. A Tabela 3 lista a responsabilidade compartilhada (Duty Days) como coping para este estressor, sem relato de resultado, e a fala do desenvolvedor descreve uma possibilidade, não uma frequência.
Mecanismo: o próprio time DevOps faz as implantações das releases mensais e dos lançamentos urgentes, que são escaladas para a noite e o fim de semana, conforme quem estiver em melhor posição. **Condições:** releases mensais ou urgentes que podem ir para produção a qualquer hora; o artigo cita responsabilidade compartilhada (Duty Days) como coping. **Evidência:** empírica direta.

> Seção 4.2 (tecno-invasão), síntese dos autores: *“Monthly releases or urgent launches often required work to be done in the evenings or on the weekends.”*

Frase dos autores que abre a descrição da tecno-invasão: releases mensais ou lançamentos urgentes frequentemente exigiam trabalho à noite ou no fim de semana. Estabelece a situação concreta em que a responsabilidade do time pela implantação invade o tempo pessoal.

Seção 4.2, Agile Project Manager, Nr. 22: *"So who is deploying right now, on the weekend, for example. Then you just see who is in the best position to do it"*

Fala do participante 22, gerente de projeto ágil no banco (Caso 2), há mais de 15 anos na função. Mostra como a escala é decidida na prática: quando há implantação no fim de semana, vê-se quem está em melhor posição para fazê-la, ou seja, qualquer membro do time pode ser chamado. É o mecanismo do elo: a implantação é tarefa do próprio time.

Seção 4.2, SE (Dev), Nr. 12: *"Theoretically, we could release it at any time during the night or at some point over the weekend"*

Fala do participante 12, desenvolvedor da seguradora (Caso 1) com 10 a 15 anos na função: em tese, a release pode ir para produção a qualquer hora da noite ou em algum momento do fim de semana. Mostra a imprevisibilidade do horário, mas é formulada como possibilidade (theoretically), não como relato de quantas vezes isso acontece.

↑ **aumenta** Desenho do trabalho (fator –, nome neutro*)

Kärpänoja et al. (2016) — *Exploring peopleware in continuous delivery*

Contexto: Mesmo estudo de três entrevistas na Solita. Nos projetos A e B a configuração dos servidores era automatizada como código (IaC, infrastructure-as-code) e mantida pelo próprio time de desenvolvimento, que também respondia por testes, qualidade e entrega; no projeto C não havia gerência de configuração automatizada. Os autores interpretam que deixar o time cuidar da infraestrutura amplia sua responsabilidade e seu controle sobre o sistema, e apoiam isso em falas resumidas sobre motivação: todos disseram que a responsabilidade maior aumentava a motivação, mas um entrevistado acrescentou que ela também aumentava o estresse, por exigir competência mais ampla. A contrapartida vem de dois entrevistados, que se sentiam menos responsáveis por problemas em componentes ou servidores aos quais não tinham acesso. O elo é, assim, interpretação dos autores sobre poucas falas, com efeito ambivalente: mais autonomia e envolvimento e, para uma das três pessoas, mais exigência.
Mecanismo: deixar o time cuidar da infraestrutura, das configurações de servidor e da entrega, além de testes e qualidade, dá mais responsabilidade e controle sobre o sistema e exige um repertório mais amplo de competências; onde o acesso a componentes e servidores era negado, os desenvolvedores se sentiam menos responsáveis pelos problemas ali. **Condições:** projetos com configuração automatizada (IaC) em dois dos três casos; para um entrevistado, a responsabilidade maior também aumentou o estresse, pela exigência de competência mais ampla; o efeito depende de o time ter acesso real aos componentes. **Evidência:** empírica indireta.

Seção 4.1 (os três projetos): *"letting them manage the infrastructure gives them more responsibility and control over the system."*

Os autores comparam a gestão da infraestrutura com a responsabilidade por testes e qualidade: as duas dão ao time mais responsabilidade e controle sobre o sistema. É a formulação do elo com o desenho do trabalho: o time ganha autonomia e passa a cuidar de uma parte maior do sistema, inclusive do que costuma ser tarefa de operação.

Seção 4.3, um entrevistado: *"one interviewee also pointed out that it increased stress."*

Depois de relatar que a responsabilidade maior aumentava a motivação, os autores registram que um dos três entrevistados disse que ela também aumentava o estresse. É a ressalva do elo: o mesmo enriquecimento do trabalho que motiva pode pesar, e a evidência desse lado vem de uma só pessoa.

Seção 4.3: *"When the development team is also responsible for server configurations, testing and quality, wider level of"*

Início da frase que explica esse estresse: quando o time também responde por configurações de servidor, testes e qualidade, exige-se dos programadores um leque mais amplo de competências (expertise). Mostra a outra face do trabalho ampliado: mais variedade de habilidades exigidas, que é ao mesmo tempo fonte de motivação e de pressão.

Seção 4.3, dois entrevistados: *"they sometimes felt careless if problems were related to these components."*

Dois entrevistados contaram que, quando não tinham acesso a alguns componentes de integração ou servidores, às vezes se sentiam descuidados, isto é, pouco responsáveis, se o problema estivesse nesses componentes. É o contraste que sustenta o elo pelo lado negativo: sem acesso, o time não se sente dono daquela parte do trabalho.

Seção 4.3: *"the development team more access and responsibility seems to improve motivation and commitment to the project"*

Conclusão dos autores para a seção: dar ao time de desenvolvimento mais acesso e responsabilidade parece melhorar a motivação e o comprometimento com o projeto e, ao mesmo tempo, com a organização. O verbo 'seems' marca que é uma inferência a partir das falas; é o efeito final esperado do trabalho redesenhado.

↑ **aumenta** Pressão por atualização (fator −)

Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*

Contexto: Estudo qualitativo do ICSE-SEIP 2026 com 26 entrevistas semiestruturadas (45 a 60 minutos, em alemão, citações traduzidas para o inglês pelos autores) com membros de times DevOps de duas empresas suíças reguladas: uma seguradora global (Caso 1, participantes 1 a 13) e um banco (Caso 2, participantes 14 a 26), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança e incerteza tecnológicas). A responsabilidade ampliada do time DevOps aparece duas vezes nos resultados: como a principal fonte de tecno-complexidade, porque o leque de conhecimentos exigido inclui infraestrutura e segurança além do desenvolvimento, e como tecno-invasão, porque, para acompanhar esse leque, as pessoas estudam e programam fora do expediente. Não há comparação antes/depois; o elo vem de falas de desenvolvedores e da síntese dos autores. Como atenuantes, o artigo descreve tempo de treino reservado pela organização, cursos internos e externos, workshops e pair programming para compartilhar conhecimento, e um entrevistado aponta que a maior dificuldade é justamente achar tempo para aprender.
Mecanismo: assumir infraestrutura e segurança além do desenvolvimento amplia o leque de competências exigidas, e para acompanhar as pessoas passam a estudar e programar no tempo livre. **Condições:** atenuado quando a organização reserva tempo de treino, oferece cursos internos e workshops e o time compartilha conhecimento (pair programming); sem tempo alocado, o aprendizado fica para fora do horário. **Evidência:** empírica direta.

> Seção 4.2 (tecno-complexidade), SE (Dev), Nr. 5: *“We do DevOps, which means we do infrastructure, we have to take care of security, there are requirements coming from everywhere that you have to meet somewhere”*
>
> Fala do participante 5, desenvolvedor da seguradora (Caso 1) com 10 a 15 anos na função, definindo o que significa fazer DevOps para o time: cuidar também de infraestrutura e segurança, com exigências vindas de todos os lados. Os autores a apresentam como a fonte mais significativa de tecno-complexidade; para o elo, é o ponto de partida, a prática amplia as competências que cada um precisa ter.

> Seção 4.2 (tecno-invasão), síntese dos autores: *“their skills during personal times in order to keep up with the broad expertise which is required from DevOps members”*
>
> Frase dos autores: vários participantes se sentiam compelidos a desenvolver suas habilidades em tempo pessoal para acompanhar a ampla especialização exigida dos membros de DevOps, indo a conferências, aprendendo ferramentas e programando fora do horário. É a ligação explícita entre a responsabilidade ampliada e a pressão para se atualizar.

> Seção 4.2, SE (Dev), Nr. 16: *“I did a lot of things in my free time. That’s when I started doing a bit of programming and trying things out, learning new things”*
>
> Fala do participante 16, desenvolvedor do banco (Caso 2) há mais de 15 anos na função: fez muita coisa no tempo livre, começou a programar, experimentar e aprender coisas novas. Ilustra o comportamento que resulta da pressão: a atualização acontece fora do expediente.

> Seção 4.3 (coping individual), Application Manager, Nr. 24: *“the biggest challenge is taking the time to do it”*
>
> Fala do participante 24 sobre aprender fazendo (o texto o chama de Application Manager; na Tabela 2 o número 24 é um Agile Project Manager do banco): o maior desafio é arranjar tempo para isso. Os autores concluem que a organização precisa reservar tempo para o desenvolvimento de competências; a citação sustenta a condição do elo, sem tempo alocado o aprendizado recai sobre o tempo pessoal.

Implantação contínua

2 elos em 2 artigo(s)

> **O que é.** Um passo além da entrega contínua: toda alteração que passa nos testes automáticos vai para produção sozinha, sem aprovação manual. Empresas que a adotam lançam dezenas ou centenas de vezes por dia. Descrição completa na seção 1.

↑ **aumenta** Carga cognitiva (fator −)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Contexto: Mesmo estudo de caso no Group X. Pelo critério dos autores, o nível de maturidade DevOps supõe automação das releases e integração contínua, que o nível ágil ainda não alcança; o artigo, porém, não descreve que automação os times usavam. 'Riscos escondidos pela automação' é um dos riscos que só apareceram nos times DevOps (Tabela 2) e forma sozinho o subtema 'riscos da automação'. O elo se apoia em uma única fala de engenheiro de produção e na síntese dos autores; trata-se de risco percebido, não de dano observado, e o tema ocupa poucas linhas do artigo. Os autores concluem que o DevOps é mais arriscado quando o investimento em tempo, em apoio da hierarquia e em automação é insuficiente.
Mecanismo: com a automação das releases, o engenheiro de produção deixa de saber o que mudou na aplicação; defeitos funcionais ficam escondidos e a manutenção posterior fica mais difícil de entender. **Condições:** risco percebido por

engenheiros de produção, que mantêm a aplicação depois; o artigo fala só em automação (de releases, pela definição de maturidade DevOps), sem separar CI e deploy; agravado quando o investimento em tempo e em automação é insuficiente. **Evidência:** empírica direta.

> Findings, Risks, fator 6: *“Individuals evoke risks that may remain hidden by automation and make more complex the application maintenance.”*
>
> Síntese dos autores para o risco 'riscos escondidos pela automação'. Os entrevistados citam riscos que a automação pode deixar ocultos e que tornam mais complexa a manutenção da aplicação. Define o risco e a consequência para quem mantém o sistema.

> Findings, Risks, fator 6 (PE-DO): *“With automation, we have less and less knowledge about the app. Today with automation, I don’t know what has changed and I have no vision of what has been done.”*
>
> Fala de um engenheiro de produção de um projeto DevOps, o único verbatim sobre automação no artigo. Ele diz que, com a automação, sabe cada vez menos sobre a aplicação e não sabe o que mudou nem o que foi feito. É o mecanismo do elo: a mudança passa pelo processo automatizado sem que quem vai manter tenha visão dela.

> Dissatisfaction and risks: *“automation can hide some functional defects in the developed application, which PE’s will have difficulty maintaining later”*
>
> Interpretação dos autores no subtema 'riscos da automação'. A automação pode esconder defeitos funcionais na aplicação, que os engenheiros de produção terão dificuldade de manter depois. Mostra quem arca com o esforço extra de entendimento: o pessoal de operações, mais tarde.

↓ **reduz** Carga de trabalho (fator −)

Zhang et al. (2018) *— One size does not fit all: An empirical study of containerized continuous deployment workflows*

Contexto: O estudo, de métodos mistos e publicado no ESEC/FSE 2018, começa com um survey enviado a 1.000 donos de projetos de código aberto do GitHub que publicam imagens Docker no Docker Hub; 168 responderam, com em média 8,6 anos de experiência em código aberto e 4,3 em CI/CD. Para este elo importa a primeira pergunta aberta, sobre por que fazem implantação contínua (140 respostas), codificada por um autor em duas rodadas de codificação aberta e discutida por todos. A implantação contínua ali consiste em ter a imagem Docker construída e publicada automaticamente a cada mudança no GitHub, seja pelo auto-build do próprio Docker Hub, seja por um serviço de CI como Travis ou CircleCI. O elo vem da categoria de motivação mais citada, a de implantar de forma automática em vez de manual, e de uma fala que descreve o trabalho que deixou de existir. A ressalva é que se trata de motivação declarada para adotar a prática, não de medida da carga de trabalho antes e depois; a parte quantitativa do estudo mede só desfechos técnicos.
Mecanismo: o workflow de CD constrói e publica a imagem automaticamente a cada mudança, eliminando os passos manuais de build e deploy que antes cabiam ao desenvolvedor. **Evidência:** empírica direta.

> Seção 3.2, Tabela 1 (motivações para fazer CD, N=140): *“Helps us deploy automatically instead of doing it manually 60 42.9%”*
>
> É a linha da tabela que resume as respostas abertas sobre por que os donos de projeto fazem implantação contínua: 60 das 140 respostas (42,9%) foram codificadas como implantar automaticamente em vez de manualmente. É a motivação mais frequente e indica que, para quase metade dos respondentes, o ganho principal da prática é tirar das mãos deles uma tarefa manual.

> Seção 3.2, [M1], respondente R79: *“because the project will be built automatically, no need for me to build the image and push it to the server”*
>
> O respondente R79, dono de um projeto que publica imagem Docker, ilustra a motivação M1 (implantar automaticamente): como o projeto é construído sozinho, ele não precisa mais construir a imagem e enviá-la ao servidor. A fala nomeia concretamente os passos manuais que a prática elimina, que é o mecanismo de redução da carga de trabalho.

Postmortem sem culpados

2 elos em 1 artigo(s)

> **O que é.** Depois de uma falha relevante, o time escreve uma análise do que aconteceu e por quê, com foco em causas do sistema e do processo, e não em achar um culpado. O objetivo é aprender e evitar a repetição. Descrição completa na seção 1.

↕ **ambivalente** Carga de trabalho (fator −)

Anandayuvaraj et al. (2026) *— Learning from Software Failures: A Case Study at a National Space Research Center*

Contexto: Estudo de caso qualitativo (ICSE 2026) com 10 entrevistas semiestruturadas, de duração mediana de 53 minutos, com engenheiros de software de pesquisa do departamento de software de voo de um centro nacional de pesquisa espacial, complementadas por 5 entrevistas piloto em outras organizações de alta confiabilidade; as transcrições passaram por codificação em etapas e análise temática. Nessa organização não existe processo formal de postmortem ou retrospectiva: o aprendizado com falhas é informal, e todos os entrevistados relatam falhas que se repetem. Os engenheiros acumulam papéis de desenvolvedor, pesquisador, gerente de projeto e engenheiro de sistemas, com reuniões, burocracia e prazos, e todos apontam a falta de tempo como a principal barreira. O elo vem da seção sobre essa barreira (§5.3.1), em que os participantes comentam como seria adotar reuniões formais de reflexão sobre falhas, uns prevendo peso extra, outros economia de tempo. Ressalva: como a prática não existe ali, os dois sentidos do efeito são expectativas dos entrevistados, não experiência com postmortems realizados, e nada nos dados trata do caráter sem culpados.

Mecanismo: a reunião formal de reflexão sobre falhas soma mais uma reunião e mais burocracia a uma agenda já tomada por projeto, mas evita o retrabalho de resolver de novo as mesmas falhas recorrentes. **Condições:** aumenta quando vira reunião formal e frequente (mensal ou quinzenal) num ambiente já burocrático; reduz quando é leve e encaixada no processo existente, p. ex. nos marcos do projeto; efeitos antecipados pelos participantes, pois a prática não estava adotada na organização. **Evidência:** empírica indireta.

§5.3.1 (S3): *“we have all the bureaucracy, every additional meeting puts a big hole in your calendar”*

S3, engenheiro com 6 a 10 anos de experiência, ao comentar a ideia de criar reuniões formais de reflexão sobre falhas. Diz que o ambiente já tem muita burocracia e que cada reunião a mais abre um buraco grande na agenda; na continuação da fala, prevê que quem propusesse mais uma reunião desse tipo seria odiado. Sustenta o lado em que a prática aumenta a carga: formalizada como mais uma reunião, ela se soma a uma agenda já cheia.

§5.3.1 (S8): *“if it’s something like twice a month or even once a month, for me it will become a little bit tedious.”*

S8, também com 6 a 10 anos de experiência, reage à frequência possível dessas reuniões: se fossem duas vezes por mês, ou mesmo uma, ele as acharia tediosas. Reforça o lado do aumento e indica a condição em que ele aparece: o peso vem da reunião formal e frequente, não da reflexão em si.

§5.3.1 (S1): *“That’s a bad excuse not having the time when we save time by this.”*

S1, com 11 a 15 anos de experiência, no trecho em que vários entrevistados reconhecem que aprender com falhas compensaria. Diz que não ter tempo é uma desculpa ruim, porque o processo economizaria tempo. É o contraponto ao argumento da falta de tempo: a mesma escassez que pesa contra o postmortem seria aliviada por ele.

§5.3.1 (S5): *“It would be helpful to have a process to learn from mistakes...because it takes a lot of time if people make the same mistakes over and over again.”*

S5, com 1 a 5 anos de experiência, que no mesmo trecho descreve os colegas como mergulhados em trabalho de projeto. Diz que um processo para aprender com os erros ajudaria, porque se perde muito tempo quando as pessoas cometem os mesmos erros repetidamente. Explicita o mecanismo do lado da redução: o postmortem cortaria o retrabalho das falhas recorrentes, que o estudo mostra serem comuns na organização (§5.2.1).

§5.3.1 (S9): *“if you do it [with] milestones...then it might be a huge benefit.”*

S9, com 1 a 5 anos de experiência, sugere fazer a reflexão nos marcos (milestones) do projeto, em vez de criar um rito separado, e diz que assim o benefício poderia ser grande. Dá a condição em que o efeito tende à redução da carga: postmortem leve, encaixado no processo que já existe.

↑ **aumenta** Documentação interna (fator +)
Anandayuvaraj et al. (2026) *— Learning from Software Failures: A Case Study at a National Space Research Center*
Contexto: Mesmo estudo de caso no centro de pesquisa espacial, com 10 entrevistas com engenheiros de software de pesquisa e 5 entrevistas piloto em outras organizações de alta confiabilidade. A maioria dos entrevistados (8 de 10) disse que documentar lições aprendidas é ad hoc e fica a critério de cada um; metade disse que elas simplesmente não são documentadas, e o pouco registro que existe se espalha entre issues do GitLab, merge requests, atas de reunião e uma wiki pouco conhecida. Seis dos dez relataram perda de conhecimento com a rotatividade, que um deles descreve como gente entrando e saindo a cada seis meses. O elo é construído pela ausência: os participantes atribuem a falta de registro à inexistência de um processo de postmortem que torne a escrita das lições parte do projeto, e um deles relata o tempo poupado quando esse registro existe. Ressalva: não há comparação com um time ou período que tivesse postmortem; o sentido do efeito é inferido do que falta e do que os entrevistados esperam.
Mecanismo: sem um processo de postmortem que exija registrar as lições, o conhecimento sobre falhas fica tácito, na cabeça de quem resolveu, e se perde quando a pessoa sai; o registro, quando existe, poupa tempo de quem chega.
Condições: evidência pela ausência da prática: os participantes ligam a falta de registro à inexistência do processo, sobretudo em times com rotatividade alta. **Evidência:** empírica indireta.

> §5.1.2 (S4): *“when we close the issue, it is not mandatory to document what did you do...[it’s a] personal decision. We don’t have that process”*
> S4, com 11 a 15 anos de experiência, descreve o que acontece ao fechar uma issue: documentar o que foi feito não é obrigatório, é decisão pessoal, porque não existe esse processo; na continuação, diz que ele mesmo documenta por saber que é importante. Mostra o estado sem a prática: o registro depende da iniciativa individual, e não de um processo de reflexão sobre falhas.

> §5.3.3 (S2): *“[Lessons] should be explicitly documented because experience is just related to one person”*
> S2, com 6 a 10 anos de experiência, falando da perda de conhecimento com a rotatividade dos times. Defende que as lições sejam documentadas explicitamente porque a experiência fica presa a uma pessoa. Liga o problema ao conhecimento tácito: sem registro, a lição vive só na memória de quem resolveu a falha.

> §5.3.3 (S2): *“losing the lesson learned...because there’s no process to say ‘part of your project is to write lesson learned’.”*
> Continuação da mesma fala de S2: perder a pessoa é perder a lição aprendida, porque não há processo que diga que escrever as lições faz parte do projeto. É a passagem que liga diretamente a falta de documentação à ausência do processo de postmortem, ou seja, o próprio elo entre a prática e a documentação interna.

> §5.3.3 (S4): *“I have used what other team members have written [about issues]...it saved me a lot of time”*
> S4 lembra que já usou o que colegas haviam escrito sobre problemas anteriores e que isso lhe poupou muito tempo, sobretudo quando pessoas entram ou saem do time. Mostra o ganho do registro quando ele existe e, portanto, o que um processo de postmortem acrescentaria. É relato de uma pessoa sobre registros feitos por iniciativa individual, não por um processo formal.

ESC/DevOps em geral

1 elo em 1 artigo(s)

> **O que é.** Usada quando o estudo trata da adoção de DevOps ou de Engenharia de Software Contínua como um pacote, sem separar qual prática específica produziu o efeito. Descrição completa na seção 1.

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)
Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*
Contexto: Mesmo estudo de caso no Group X. A variedade de tarefas é o primeiro dos dez fatores de satisfação, aparece nos modos ágil e DevOps (Tabela 2) e corresponde, na Tabela 3, à característica 'variedade de habilidades' do modelo de Hackman e Oldham, que os autores usam como referencial. Para os fatores comuns, os autores escolheram ilustrar com falas de times DevOps, por isso a fala vem de um release manager DevOps. Que a variedade seja maior com DevOps é argumento dos autores na discussão, baseado no escopo mais amplo da atividade, sem dado que compare os dois modos; daí a evidência indireta.
Mecanismo: o escopo mais amplo do trabalho em DevOps faz a pessoa tocar muitas áreas, usar ferramentas e tecnologias variadas e aprender todo dia, o que aumenta a variedade de tarefas. **Condições:** a variedade de tarefas é fator comum aos modos ágil e DevOps; que ela seja maior em DevOps é argumento dos autores, pelo escopo mais amplo, sem dado comparativo. **Evidência:** empírica indireta.

Findings, Satisfaction, fator 1: *“DevOps methodology has satisfied several interviewed individuals, who appreciate testing varied teamwork methods, several tools and technologies, or benefiting from a daily learning mode in their profession.”*
Síntese dos autores para o fator 'variedade de tarefas'. Vários entrevistados apreciam experimentar métodos de trabalho em equipe variados, usar várias ferramentas e tecnologias e aprender no dia a dia da profissão, e os autores dizem que isso vale para os diferentes perfis. Descreve em que consiste a variedade.

Findings, Satisfaction, fator 1 (RM-DO): *“We still touch a lot of areas, so it's an interesting job for that.”*
Fala de um release manager de um projeto DevOps; esse papel, segundo os autores, surgiu com a abordagem DevOps. Ele diz que mexe em muitas áreas e que por isso o trabalho é interessante. É o depoimento direto de que a amplitude do papel torna o trabalho mais variado e atraente.

Discussion: *“With a wider scope of activity, it can be argued that sequential variety of DevOps team is also larger than that of agile teams.”*
Argumento dos autores na discussão, apoiado no conceito de 'variedade sequencial' de Pentland (variedade nas sequências de passos do trabalho, ligada a conhecimento tácito). Com escopo de atividade mais amplo, a variedade sequencial dos times DevOps seria maior que a dos ágeis. É raciocínio teórico, não medição.

Monitoramento e observabilidade

1 elo em 1 artigo(s)

> **O que é.** Os instrumentos que mostram, em tempo real, se o sistema está saudável: métricas, registros (logs), rastreamento de requisições e alertas. Observabilidade é a capacidade de entender o que está acontecendo por dentro a partir desses sinais. Descrição completa na seção 1.

↕ **ambivalente** Carga cognitiva (fator –)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Contexto: Estudo qualitativo do ICSE-SEIP 2026 com 26 entrevistas semiestruturadas (45 a 60 minutos, em alemão, citações traduzidas para o inglês pelos autores) com membros de times DevOps de duas empresas suíças reguladas: uma seguradora global (Caso 1, participantes 1 a 13) e um banco (Caso 2, participantes 14 a 26), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança e incerteza tecnológicas). O monitoramento aparece como fonte de tecno-sobrecarga, o estressor de ter de trabalhar mais e mais rápido por causa da tecnologia: acompanhar incidentes significava vigiar ao mesmo tempo várias caixas de e-mail, o sistema de tickets e as ferramentas de monitoramento, e os dados de monitoramento tinham se tornado excessivos fora de incidentes críticos. Como coping no nível das ferramentas, um desenvolvedor da seguradora descreve alertas automáticos por exceção enviados a uma caixa do time, com os dias de atendimento repartidos entre as pessoas. O elo combina essas falas com a interpretação dos autores (divisão da atenção, fadiga mental, alívio da vigilância); não há medida de carga, e a fala que os autores usam para ilustrar o excesso de dados trata, na verdade, do esforço de manter configurações de monitoramento pelo pipeline.
Mecanismo: acompanhar incidentes por várias caixas de e-mail, sistema de tickets e dados de monitoramento divide a atenção e sobrecarrega; alertas automáticos para uma caixa do time, com os dias repartidos entre as pessoas, tiram o peso da vigilância constante. **Condições:** aumenta quando o monitoramento é espalhado por vários canais e gera volume de dados; reduz quando há alerta automático por exceção e divisão dos dias de atendimento no time. **Evidência:** empírica indireta.

Seção 4.2 (tecno-sobrecarga), SE (Dev), Nr. 12: *“You have to monitor everything that goes wrong, process multiple mailboxes, and operate the ticket system.”*
Fala do participante 12, desenvolvedor da seguradora (Caso 1), descrevendo o acompanhamento de incidentes: monitorar tudo que dá errado, processar várias caixas de e-mail e operar o sistema de tickets. Descreve o monitoramento espalhado por vários canais, que é o lado em que a prática aumenta a carga.

Seção 4.2, interpretação dos autores: *“This constant multitasking, while necessary for ensuring platform stability, can disrupt focus and contribute to mental fatigue.”*
Comentário dos autores logo após a fala do participante 12: esse multitarefa constante, embora necessário para a estabilidade da plataforma, rompe o foco e contribui para a fadiga mental. É a passagem que liga o monitoramento à carga cognitiva.

Seção 4.2 (sobrecarga de informação), síntese dos autores: *“had become a source of overwhelming data, especially outside of critical incidents”*
Afirmação dos autores, atribuída a vários participantes, de que os sistemas de monitoramento, feitos para ajudar a acompanhar o desempenho, tinham virado fonte de dados excessivos, sobretudo fora de incidentes críticos; segundo

eles, esse volume gera sobrecarga cognitiva e dificulta priorizar. A fala usada como ilustração, porém, fala do esforço de mapear arquivos de configuração do monitoramento pelo pipeline, não do volume de alertas.

Seção 4.3 (coping por ferramentas), SE (Dev), Nr. 5: *“With the latest applications, we have really set up monitoring so that if certain exceptions occur, we are notified by email. As a team, we have an application mailbox where everything is sent, and we have divided up the days accordingly.”*
Fala do participante 5, desenvolvedor da seguradora (Caso 1), sobre as aplicações mais recentes: o monitoramento avisa por e-mail quando ocorrem certas exceções, tudo cai numa caixa da aplicação compartilhada pelo time, e os dias de atendimento foram divididos. É o lado em que a prática reduz a carga: alerta por exceção em vez de vigilância contínua, com a responsabilidade repartida.

Seção 4.3, interpretação dos autores: *“These systems reduce the mental burden of constant vigilance”*
Conclusão dos autores sobre essa configuração: esses sistemas reduzem o peso mental da vigilância constante e permitem dividir a responsabilidade no time de forma administrável. Sustenta o sentido de redução do elo ambivalente, mas é interpretação dos autores, não medida.

On-call

1 elo em 1 artigo(s)

> **O que é.** Escala de plantão: em cada período, uma pessoa do time fica responsável por responder a alertas e incidentes do sistema, inclusive à noite e nos fins de semana. É a contrapartida operacional de manter serviços no ar 24 horas. Descrição completa na seção 1.

↑ **aumenta** Conflito trabalho-família (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Contexto: Estudo qualitativo do ICSE-SEIP 2026 com 26 entrevistas semiestruturadas (45 a 60 minutos, em alemão, citações traduzidas para o inglês pelos autores) com membros de times DevOps de duas empresas suíças reguladas: uma seguradora global (Caso 1, participantes 1 a 13) e um banco (Caso 2, participantes 14 a 26), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança e incerteza tecnológicas). O plantão aparece na categoria tecno-invasão, a tecnologia apagando a fronteira entre trabalho e vida pessoal: muitos entrevistados relatam ter de ficar disponíveis o tempo todo, e um engenheiro DevOps do banco descreve como, à noite, quem está de plantão aciona o colega especialista. O artigo não descreve como a escala foi instituída nem um estado anterior a ela; o elo vem dessa fala e da síntese dos autores. Como atenuante, os autores citam a responsabilidade compartilhada em rodízio (Duty Days), mas a fala que a ilustra trata de repartir o atendimento de pedidos, não de horário fora do expediente, de modo que o alívio sobre a vida pessoal não fica demonstrado.
Mecanismo: o plantão 24/7 obriga a ficar disponível o tempo todo e, quando algo quebra à noite, quem está de plantão liga para o colega especialista, de modo que mesmo fora da escala a pessoa se sente sempre ligada. **Condições:** plantão 24/7 com dependência de especialistas; o artigo apresenta a responsabilidade compartilhada em rodízio (Duty Days, escalas divididas) como coping, mas a fala correspondente trata de repartir pedidos, não de horário. **Evidência:** empírica direta.

Seção 4.2 (tecno-invasão), síntese dos autores: *“on-call or standby duties further contribute to this sense of invasion”*
Frase dos autores logo depois de tratarem das implantações fora do horário: o plantão (on-call ou standby, sobreaviso) é apresentado como um segundo fator de invasão da vida pessoal. Situa o elo na categoria tecno-invasão do modelo de Tarafdar, que corresponde ao conflito entre trabalho e família.

Seção 4.2, SE (DevOps), Nr. 23: *“Because we also have standby duty, 24/7, we are very dependent on each other. So it may be that a developer is on standby duty, and if something comes up on the platform at night, it’s normal that they call their colleague who is the specialist in that area”*
Fala do participante 23, engenheiro de software com papel DevOps no banco (Caso 2), há mais de 15 anos na função. Ele descreve o mecanismo: o plantão é 24/7 e, se algo falha na plataforma de madrugada, quem está de sobreaviso liga para o colega especialista naquela área. Mostra que a dependência de especialistas estende a disponibilidade noturna também a quem não está escalado.

Seção 4.2, conclusão dos autores sobre o plantão: *“This constant availability created a sense of being always”*
Interpretação dos autores sobre o conjunto das falas: a disponibilidade constante criava a sensação de estar sempre ligado (a palavra on, entre aspas, vem logo após o trecho citado), mesmo fora do expediente. É o efeito sobre a fronteira entre trabalho e vida pessoal que sustenta o sentido aumenta do elo.

Seção 4.3 (coping no nível do time), SE (Dev), Nr. 7: *“so that not everyone has to constantly process requests”*

Fala do participante 7, desenvolvedor da seguradora (Caso 1), sobre a criação de Duty Days, dias de atendimento atribuídos em rodízio dentro do time, para que nem todos tenham de processar pedidos o tempo todo. Os autores a usam como exemplo de responsabilidade compartilhada para atenuar o plantão, mas ela trata da divisão da carga de atendimento e não fala de noites ou fins de semana; serve como condição atenuante de alcance limitado para este fator.

6. Fichas dos artigos

O que cada artigo estuda e conclui segundo o texto completo, e os elos que sustenta.

Com elo prática → fator (22)

Anandayuvaraj et al. (2026) — Learning from Software Failures: A Case Study at a National Space Research Center

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026

Estudo de caso qualitativo (ICSE 2026) com 10 entrevistas com engenheiros de software de pesquisa num centro espacial nacional, mais 5 entrevistas piloto em outras organizações de alta confiabilidade, sobre como se aprende com falhas (postmortem/retrospectiva). O achado central é a ausência de processo estruturado de postmortem: o aprendizado é informal, as falhas se repetem e o conhecimento se perde com a rotatividade e a documentação fragmentada. O elo com as pessoas é fraco e vem sobretudo de percepções sobre a prática ainda não adotada: os participantes antecipam reuniões de reflexão como burocracia que consome tempo, mas também dizem que ela pouparia o retrabalho de falhas recorrentes; e atribuem a falta de registro das lições à inexistência do processo. O contexto não é de DevOps/entrega contínua, e o lado sem culpados do postmortem não aparece nos dados.

Postmortem sem culpados ↕ **ambivalente** Carga de trabalho (fator −); Postmortem sem culpados ↑ **aumenta** Documentação interna (fator +)

Jeyam et al. (2026) — "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2026

Estudo qualitativo com 26 entrevistas semiestruturadas com membros de times DevOps de duas empresas suíças reguladas (seguradora e banco), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança, incerteza) e por estratégias de coping em quatro níveis. As falas ligam plantão 24/7 e implantações noturnas e de fim de semana à invasão da vida pessoal, a responsabilidade ampliada do DevOps ao estudo fora do horário, o monitoramento por vários canais à sobrecarga cognitiva e a heterogeneidade dos pipelines de CI/CD à incerteza e à troca de contexto; escala dividida (Duty Days), alertas automáticos com dias repartidos, tempo de treino e padronização de pipelines aparecem como atenuantes. A parte de GenAI (estressor e mitigador) não envolve prática de ESC e não gerou efeito.

On-call ↑ **aumenta** Conflito trabalho-família (fator −); Responsabilidade de operação pelo time ↑ **aumenta** Conflito trabalho-família (fator −); Responsabilidade de operação pelo time ↑ **aumenta** Pressão por atualização (fator −); Monitoramento e observabilidade ↕ **ambivalente** Carga cognitiva (fator −); Pipeline de CI/CD ↕ **ambivalente** Carga cognitiva (fator −)

Kafoe et al. (2026) — "making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026

Estudo exploratório multimétodo (ICSE 2026): revisão de literatura cinzenta (61 fontes, blogs e posts de praticantes) seguida de estudo de caso com 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações) na ING, que adotou SRE e faz oficinas de eliminação de toil. O elo com as pessoas vem da eliminação de toil como prática de SRE: os entrevistados relatam trabalho menos monótono, mais tempo para engenharia e desenvolvimento pessoal, mas também perda da 'vitória rápida' que dá sensação de dia produtivo, perda de conhecimento do sistema e de habilidade de depuração, e ameaça ao emprego de quem vive de toil. Burnout, moral e turnover aparecem como desfechos (não como fatores), sem teste quantitativo; a ligação toil → facilita onboarding (CS-1) também aparece, mas o lado da eliminação só vem da literatura cinzenta e ficou fora dos efeitos.

Site Reliability Engineering ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Site Reliability Engineering ↑ **aumenta** Alinhamento do uso do tempo (fator +); Site Reliability Engineering ↕ **ambivalente** Senso de realização (fator +); Site Reliability Engineering ↑ **aumenta** Insegurança no emprego (fator −); Site Reliability Engineering ↕ **ambivalente** Desenvolvimento de competências (fator emergente, Individual)

Kassab et al. (2026) — The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes

FSE Companion 2026 - Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering

Mineração de 73.125 comentários de issues sobre dívida técnica em três projetos Apache (Hadoop, Kafka, Flink), com LDA em 7 temas e classificador de emoção (índice NEI). O tema de dívida de testes e infraestrutura de CI/CD (testes flaky, pipelines de build instáveis) tem NEI mais alto que os temas de API e de código, mas o efeito é pequeno (Cliff's δ = 0,06) e os autores não afirmam causalidade; a comparação é entre tipos de dívida, não entre ter ou não CI. O tema de observabilidade e conectividade tem o NEI mais alto, mas mistura configuração, SSL e conexão, e a ligação com a carga de on-call é afirmação dos autores sem dado: não registrado.

Pipeline de CI/CD ↑ **aumenta** *desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração); Pipeline de CI/CD ↑ **aumenta** Interrupções no trabalho (fator −)

Andreyev (2025) — The Art of Managing Flaky Tests at Scale

2025 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2025

Relato de experiência de duas páginas (workshop CCIW no ICSTW 2025, Okta) sobre uma iniciativa de um ano para controlar testes flaky na CI de um monorepo: os alertas de teste flaky exigem ação em um dia útil, e o autor variou os limiares de alerta medindo a taxa de testes corrigidos na primeira investigação (de 23,9% para 44,8%). O único dado é essa taxa; a leitura de que limiares agressivos geram fadiga de alertas e dessensibilização dos times é do autor, a partir dela. Dashboards, calendário de incidentes da CI, tutoriais, office hours, sorteios e shout-outs teriam mudado a visão dos flaky tests de fardo para responsabilidade coletiva, mas isso é afirmação sem dado e as iniciativas não são práticas de ESC; não gera efeito.

Automação de testes ↑ **aumenta** Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)

Riedl et al. (2025) — How an Agile Software Process Increases Developers' Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model

Business and Information Systems Engineering

Survey transversal com 178 desenvolvedores de times ágeis na Áustria, com mediação moderada (PROCESS, modelos 4 e 7): agilidade, medida por oito práticas do instrumento de Maruping et al. (2009), reduz o estresse medido como desequilíbrio esforço-recompensa (ERI) e assim aumenta indiretamente a satisfação; o efeito vem da percepção de recompensa, não do esforço, e some nos muito sobrecomprometidos. Das oito práticas, só integração contínua e teste unitário são de ESC. Para CI, o efeito sobre o estresse fica no limite (β = -0,0151, p = 0,0553, IC até 0,0002), mas o efeito indireto sobre a satisfação é significativo. A ligação de CI à recompensa é inferência dos autores: a decomposição esforço × recompensa só foi feita para a agilidade agregada. Teste unitário não teve efeito significativo sobre o estresse (p = 0,10) nem efeito indireto sobre a satisfação (IC inclui zero), por isso não entrou como efeito.

Integração contínua ↑ **aumenta** Percepção de recompensa (fator emergente, Organizacional)

Fluri et al. (2024) — On the importance of CI/CD practices for database applications

Journal of Software: Evolution and Process

Revisão sinóptica da literatura seguida de três estudos de caso industriais (Oracle; 20, 7 e 35 desenvolvedores) em que os autores implantaram um pipeline automatizado de CI/CD, com métricas 5 semanas antes e 5 depois e 12 entrevistas de 90 min (dois desenvolvedores por caso, antes e depois). As entrevistas só são resumidas pelos autores, sem falas literais (por sigilo), e uma das autoras era consultora das empresas. Os autores relatam que, depois do pipeline, desenvolvedores e coordenadores de release ficaram com menos carga cognitiva, menos interrupções por troubleshooting de integração e implantação, mais confiança ao implantar e menos dependência do coordenador de release. Não se mediu efeito sobre a hora extra relatada antes (implantações fora do expediente por pacote único de migração), e o lead time não melhorou onde a empresa manteve janelas fixas de release.

Pipeline de CI/CD ↓ **reduz** Carga cognitiva (fator −); Pipeline de CI/CD ↓ **reduz** Interrupções no trabalho (fator −); Pipeline de CI/CD ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Pipeline de CI/CD ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Valenzuela-Toledo et al. (2024) — The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance

Proceedings - 2024 IEEE International Conference on Source Code Analysis and Manipulation, SCAM 2024

Estudo de mineração de repositórios (183 projetos maduros do GitHub, 10 linguagens) sobre a manutenção dos arquivos de workflow do GitHub Actions: volume, taxa de mudança, acoplamento lógico com código de produção e teste, motivos do acoplamento (card sorting de 200 commits) e quem faz a manutenção. Conclui que o pipeline automatizado tem um custo escondido: os workflows mudam todo mês, sobretudo para melhorias de CI/CD e correção de bugs, e essa manutenção manual fica concentrada em cerca de 5% dos desenvolvedores, que também mexem em código de produção e teste. Não mede nada nas pessoas diretamente: o esforço é inferido a partir dos commits, e os autores reconhecem que contar mudanças é só um proxy da manutenção (ameaça à validade de construto). Por isso entra um único elo, com evidência indireta.

Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Ahmad et al. (2023) — An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration

IEEE International Conference on Software Quality, Reliability and Security, QRS

Estudo de caso na Axis Communications: grupo focal de 3 horas com 4 testadores seniores e análise temática dos desafios do teste de regressão em CI e dos efeitos percebidos da priorização de testes por diversidade (DBT), depois de uma demonstração. Quase tudo trata de adoção da técnica e de eficácia dos testes; o único elo com quem trabalha é o fardo de manter a suíte automatizada executada continuamente (testes desatualizados, falhas por mudança de regra e por ambiente de rede, checagem manual). Os efeitos da DBT (liberar tempo para teste exploratório, transparência) são prospectivos, após a demo, e o de teste exploratório se refere a teste manual: não registrados.

Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Asthana et al. (2023) — A Case Study of Developer Bots: Motivations, Perceptions, and Challenges

ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo de caso na Microsoft (ESEC/FSE 2023) de 23 bots de desenvolvimento que atuam nos pull requests do ambiente de CI/CD (análise estática, segurança, configuração, revisores, testes flaky): entrevistas com 13 autores de bots, 2 da plataforma e 9 usuários, survey com 54 usuários e logs de ~1,5 ano de ações dos bots. Os usuários relatam que os bots evitam PRs de correção, tiram do revisor o trabalho de apontar miudezas, orientam novatos sem ocupar um sênior e respondem dúvidas de privacidade que antes esperavam um dia; em contrapartida, muitos bots simultâneos, recomendações longas, sem próximo passo claro ou irrelevantes poluem a revisão e são ignoradas. A prática estudada (bots acoplados ao pipeline de CI/CD, em nome do shift-left) não cabe bem na tabela e vai como prática nova; os bots de segurança poderiam ser lidos como DevSecOps, mas os efeitos relatados não são específicos deles. Os dados de engajamento (RQ3) são só técnicos e não geram efeito.

Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Integração de novatos (fator +); Bots de desenvolvimento no pipeline de CI/CD ↕ **ambivalente** Carga de trabalho (fator −); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Carga cognitiva (fator −); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Documentação interna (fator +); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Conflito de papéis (fator −)

Soltren e Mulligan (2023) — Enabling Pre-Merge CI on your TV

Proceedings - 2023 IEEE 16th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2023

O PDF é só o resumo estendido (1 página) de uma palestra no ICSTW 2023: relato de experiência da Roku sobre migração de Perforce para git com submódulos e adoção de CI pré-merge obrigatório no firmware RokuOS, sem seção de método e sem nenhum dado sobre pessoas. O único dado é técnico: a taxa de builds quebrados caiu de 8,70% para 4,85% em três meses. O efeito sobre quem trabalha (desenvolvimento parado para desfazer mudanças antes do CI pré-merge; processo adicional para quem faz commit quando o merge falha) é narrado pelos próprios autores, por isso os elos são fracos (indireta e inferida).

Integração contínua ↓ **reduz** Manutenção do ambiente de desenvolvimento (fator −); Integração contínua ↑ **aumenta** Carga de trabalho (fator −)

Parry et al. (2022) — Surveying the Developer Experience of Flaky Tests

Proceedings - International Conference on Software Engineering

Survey online com 170 desenvolvedores (Likert de impactos, causas e ações, mais análise temática das respostas abertas) e análise temática de 38 threads do StackOverflow sobre testes instáveis (flaky). Os participantes concordam que testes instáveis atrapalham a CI e causam perda de produtividade; a ação mais comum é reexecutar o build, e quem convive com eles ao menos mensalmente tende mais a não agir e a ignorar falhas possivelmente genuínas. As respostas abertas trazem raiva e frustração (SQ11t2, SQ8t1), que os autores leem como custo psicológico, e apontam diferenças entre o ambiente local e o de CI como causa de instabilidade. O estudo não mede bem-estar; o elo é com o atrito de ferramenta e infraestrutura.

Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −); Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Shetty et al. (2022) — AutoTSG: learning and synthesis for incident troubleshooting

ESEC/FSE 2022 - Proceedings of the 30th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo industrial na Microsoft (ESEC/FSE 2022; o PDF lido é a versão arXiv) sobre Troubleshooting Guides (TSGs, runbooks de incidente) usados por engenheiros de plantão: mineração de cliques ligando ~4800 TSGs a incidentes em 4 meses, codificação aberta de 428 feedbacks sobre TSGs, avaliação técnica da ferramenta AutoTSG e entrevistas curtas com 10 engenheiros de on-call. Incidentes sev 1–2 com TSG têm TTM médio menor (≈13 h contra ≈19 h), que os autores tomam como proxy do esforço de plantão; ao mesmo tempo, TSGs incompletos, desatualizados e confusos fazem o plantonista recorrer a colegas e verificar passos à mão. A parte sobre automação de TSGs (AutoTSG) é só percepção prospectiva dos entrevistados ("automating TSGs would help"), sem uso real, e não gera efeito.

Gestão de incidentes ↓ **reduz** Carga de trabalho (fator −); Gestão de incidentes ↑ **aumenta** Interrupções no trabalho (fator −); Gestão de incidentes ↑ **aumenta** Carga cognitiva (fator −)

Hemon et al. (2018) — Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams

International Conference on Information Systems 2018, ICIS 2018

Estudo de caso qualitativo embutido (ICIS 2018, literatura branca; versão depositada no HAL) numa grande empresa europeia de serviços: 54 entrevistas em profundidade com cinco papéis (DM, PO, AR, PE, RM) em 11 projetos, uns em modo ágil e outros em modo DevOps, mais a fase exploratória, com análise temática. Conclui que DevOps traz mais fontes de satisfação que o ágil (dinâmica de grupo, processo colaborativo, papéis de Ops reconhecidos), mas também insatisfações e riscos específicos: papéis PO/RM expostos a crítica, falta de feedback da hierarquia, sobrecarga dos operadores entre projeto e incidente, riscos escondidos pela automação. A comparação ágil × DevOps é por presença de temas, sem teste estatístico, e 8 dos 10 fatores de satisfação são comuns aos dois modos. Sobre a versão estendida (Hemon-Hildgen et al. 2020, EJIS, sem PDF): este texto basta para os 7 elos registrados, todos com fala de participante ou tabela; mas a parte de automação é curta (um risco e uma fala) e a extensão deve aprofundar automação, compartilhamento e condições de trabalho, então vale obtê-la para refinar mecanismos e condições, sem ser indispensável para os elos.

Times multifuncionais Dev–Ops ↑ **aumenta** Relações no trabalho (fator +); Times multifuncionais Dev–Ops ↕ **ambivalente** Reconhecimento no trabalho (fator +); Times multifuncionais Dev–Ops ↓ **reduz** Apoio do gestor (fator +); Times multifuncionais Dev–Ops ↑ **aumenta** Conflito de papéis (fator −); Implantação contínua ↑ **aumenta** Carga cognitiva (fator −); Releases frequentes ↑ **aumenta** Pressão de prazos (fator −); ESC/DevOps em geral ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Limoncelli (2018) — SQL is no excuse to avoid DevOps

Queue

AVISO: literatura cinza de 1º nível (coluna de opinião no ACM Queue, sem revisão por pares nem método). Thomas Limoncelli, gerente de SRE, defende que banco SQL não impede DevOps e descreve duas técnicas: atualização automática de esquema gerida pela aplicação e código compatível com vários esquemas (expand/contract), que permitem implantar sem janela de manutenção. O único dado é o relato de um amigo sobre o processo manual (implantação de horas no fim de semana, principal fonte de estresse, medo de perder o emprego); o efeito da automação sobre as pessoas é afirmação do autor a partir da experiência profissional, sem caso observado depois da adoção. Por isso os 2 elos são “inferida”, não “empírica indireta”: não há dado em que prática e fator apareçam juntos, só o estado anterior à prática e a previsão do autor.

Entrega contínua ↓ **reduz** Conflito trabalho-família (fator −); Entrega contínua ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Zhang et al. (2018) — One size does not fit all: An empirical study of containerized continuous deployment workflows

ESEC/FSE 2018 - Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo de métodos mistos (ESEC/FSE 2018): survey com 168 donos de projetos OSS que publicam imagens Docker (codificação aberta de motivações, necessidades e barreiras) seguido de regressões de efeitos mistos sobre 855 projetos comparando o workflow de auto-builds do Docker Hub (DHW) com o workflow via serviço de CI (CIW). As regressões medem só desfechos técnicos (frequência de release, builds com erro, estabilidade do Dockerfile, latência); o elo com pessoas vem das falas do survey: a implantação contínua tira do desenvolvedor o trabalho manual de construir e publicar, e o pipeline ao mesmo tempo poupa e cria trabalho de configuração, manutenção e espera de build. Os autores tratam o modo de adoção como trade-off: CIW é mais configurável e confiável, mas tem mais builds com erro e exige mais experiência; DHW é mais simples e de manutenção mais baixa.

Implantação contínua ↓ **reduz** Carga de trabalho (fator −); Pipeline de CI/CD ↕ **ambivalente** Manutenção do ambiente de desenvolvimento (fator −)

Claes et al. (2017) — Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content

IEEE International Working Conference on Mining Software Repositories

Mineração de repositórios (MSR 2017): timestamps de 5,86 milhões de comentários de patch e revisão no Bugzilla do Mozilla Firefox, comparando a proporção de atividade em fins de semana e à noite antes e depois da passagem, em 2011, de releases anuais para releases a cada 6 semanas (e o Firefox com o Firefox OS). Depois das releases rápidas a proporção de trabalho fora do horário comercial caiu, com efeito grande nos fins de semana e menor à noite; restringindo aos contratados da Mozilla, a queda nos fins de semana se mantém e a da noite fica desprezível. Os autores concluem que a suposição de que mais prazos geram mais pressão e mais trabalho fora de hora não se confirmou. O trabalho fora de hora é medido por proxy (horário dos comentários), numa só organização, e o estudo não mede pressão de prazos nem bem-estar diretamente.

Releases frequentes ↓ **reduz** Conflito trabalho-família (fator −)

Pinto et al. (2017) — Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users

Proceedings - 2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2017

Survey online (CHASE 2017) com 158 desenvolvedores que já quebraram ao menos um build em projetos OSS populares que usam Travis CI, com análise qualitativa das respostas abertas sobre causas de quebra de build e benefícios e problemas da integração contínua. Do lado das pessoas, os respondentes relatam que a CI dá confiança para mudar o código, mas cobra configuração e manutenção do ambiente de build, disciplina sob a norma de que o master tem de compilar, esforço extra, deslocamento do trabalho para a infraestrutura de build e dificuldade para novatos. A pressão de tempo aparece como causa de quebra de build, não como efeito da CI, e por isso não foi registrada.

Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −); Integração contínua ↑ **aumenta** Carga de trabalho (fator −); Integração contínua ↑ **aumenta** Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho); Integração contínua ↓ **reduz** Alinhamento do uso do tempo (fator +); Integração contínua ↓ **reduz** Integração de novatos (fator +); Integração contínua ↑ **aumenta** Confiança para alterar o código (fator emergente, Técnico e ferramentas)

Souza e Silva (2017) — Sentiment Analysis of Travis CI Builds

IEEE International Working Conference on Mining Software Repositories

Estudo de mineração de repositórios (MSR Challenge) com 1.262 projetos Java e Ruby do TravisTorrent (609 mil builds, 1 milhão de commits), com sentimento das mensagens de commit medido pelo SentiStrength ajustado e testes de qui-quadrado e Mann-Whitney. Commits seguintes a um build quebrado no servidor de CI são um pouco mais negativos e menos positivos, mas com efeito muito pequeno (Cliff's delta −0,014), e os autores avisam que o efeito real pode ser ainda menor por imprecisão da ferramenta. O sentido inverso (sentimento negativo → build quebrado) é pessoa → pipeline e não gerou registro; commits que mencionam o Travis CI são levemente mais positivos, possivelmente por sarcasmo, sem elo de prática. Nenhum fator intermediário é medido: o registro é direto ao desfecho.

Pipeline de CI/CD ↑ **aumenta** *desfecho:* sentimento negativo expresso nas mensagens de commit

---

Itkonen et al. (2016) — Perceived Benefits of Adopting Continuous Delivery Practices

International Symposium on Empirical Software Engineering and Measurement

Estudo de caso exploratório (entrevistas semiestruturadas com três desenvolvedores do fornecedor e duas pessoas do cliente, análise temática) sobre cinco anos de transição para entrega contínua numa organização fornecedor–cliente que atende um cliente do setor público na Finlândia. Os desenvolvedores relatam menos tarefa manual repetitiva, feedback mais rápido, menor risco de falha na implantação (a que atribuem boa parte da queda do estresse) e menos retrabalho por especificação mal entendida; a adoção, por outro lado, custou esforço de aprendizado acima do previsto. Sobre o cliente: a colaboração não foi registrada como 'aumenta Interação com clientes', porque no catálogo esse fator é o desgaste (requisito que muda, cliente indisponível para tirar dúvida, trabalho feito virando trabalho perdido) e o artigo mostra o contrário: o contato cresceu e esse desgaste caiu. Por isso o elo é 'reduz Interação com clientes', com evidência indireta porque as práticas de colaboração vieram junto com o pipeline e o artigo não as separa; não criei fator emergente de colaboração porque o ganho do lado do desenvolvedor já cabe nesse registro e no de Desenho do trabalho, e duplicar inflaria a matriz. O cansaço de estar disponível todo dia foi relatado só pelo usuário-chave do cliente, que não produz software, e ficou de fora.

Entrega contínua ↓ **reduz** Interação com clientes (fator −, nome neutro*); Entrega contínua ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Entrega contínua ↓ **reduz** Risco de falha na implantação (fator emergente, Técnico e ferramentas); Entrega contínua ↕ **ambivalente** Carga de trabalho (fator −)

Kärpänoja et al. (2016) — Exploring peopleware in continuous delivery

ACM International Conference Proceeding Series

Artigo de workshop (XP '16 Workshops, ACM), derivado de uma dissertação de mestrado: três entrevistas semiestruturadas de cerca de duas horas com projetistas líderes de três projetos web pequenos (4 a 7 pessoas) da Solita, todos com pipeline de entrega contínua automatizado e implantação em produção possível todo dia. Os autores relatam que a prontidão para implantar e corrigir rápido tira o estresse com bugs e implantações, que implantações automatizadas acontecem no horário normal sem hora extra, que a responsabilidade do time por testes, qualidade e infraestrutura dá mais controle e forma um time unido (para um entrevistado, também mais estresse pela exigência de competência mais ampla), que implantar pouco (projetos B e C) aumenta o receio da próxima implantação e a confusão entre versões, e que testes end-to-end em navegador falham sem motivo e consomem tempo. Limitações declaradas: amostra de três pessoas, entrevistas feitas por um único pesquisador que participou de um dos projetos, sem transcrição completa, e desenvolvedores descritos como abertos a mudança.

Entrega contínua ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas); Releases frequentes ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas); Releases frequentes ↓ **reduz** Carga cognitiva (fator −); Entrega contínua ↓ **reduz** Conflito trabalho-família (fator −); Entrega contínua ↑ **aumenta** Relações no trabalho (fator +); Responsabilidade de operação pelo time ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Rossi et al. (2016) — Continuous deployment of mobile software at facebook (showcase)

Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering

Relato industrial (FSE Showcase) com análise quantitativa de 7 anos de dados do Facebook (commits, crashes, cherry-picks, issues de produção) enquanto o ciclo de release do Android caiu de 8 para 1 semana. Conclui que a frequência de deploy não altera produtividade (LoC e pushes por desenvolvedor) nem qualidade — testou e não achou efeito. O único elo com pessoas no corpo do texto é o da corrida para o dia do corte do release branch, que diminui com ciclos mais curtos; a afirmação de que a automação 'reduz a carga do desenvolvedor' aparece só no abstract, e o corpo diz apenas que melhorar ferramentas e automação foi a maior parte do trabalho. Ao mover o corte para domingo, os pushes no fim de semana subiram (iOS 0,3→0,5 e Android 0,175→0,5 por desenvolvedor/dia), mas os autores não discutem isso como efeito sobre as pessoas.

Releases frequentes ↓ **reduz** Pressão de prazos (fator −)

Aprovados sem elo no texto (1)

Fluri et al. (2023) — Measuring the Benefits of CI/CD Practices for Database Application Development
Proceedings - 2023 IEEE/ACM International Conference on Software and System Processes, ICSSP 2023
Artigo do ICSSP 2023 com dois estudos de caso industriais (Oracle; UC1-DWH com 20 desenvolvedores e UC2-BE com 7) em que os autores implantaram um pipeline de CI/CD, com métricas antes e depois e entrevistas de 90 min com dois desenvolvedores por caso, antes e depois; as entrevistas só são resumidas, sem falas literais. É o MESMO estudo e o MESMO dado de Fluri2024 (JSEP), que é a versão estendida: os dois casos são dois dos três de Fluri2024, os números da Tabela IV coincidem com as colunas correspondentes da Tabela 4 de lá e a análise qualitativa é praticamente o mesmo texto ("two use cases" em vez de "three"). Todos os elos que o texto sustenta (pipeline reduz carga cognitiva, reduz interrupções e troca de contexto, aumenta autonomia ao permitir que todos implantem, aumenta a confiança ao implantar) já estão registrados em Fluri2024; o que sobra aqui — hora extra por implantação fora do expediente antes do pipeline, sem medida depois, e janelas fixas de release obrigando a acompanhar a promoção das mudanças — também aparece lá e não sustenta elo novo. Por isso nenhum efeito é registrado aqui, para não contar a mesma evidência duas vezes na matriz.

Reprovados na leitura do texto completo (3)

Gummadi et al. (2026) — Service Level Objective (SLO) Observability with Splunk and Dynatrace in Microservices
2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies, ICAISET 2026
Motivo: pergunta 4 / sem dados próprios: texto conceitual e de recomendações; os efeitos sobre pessoas (fadiga de alertas, menos apagar incêndio) são afirmados com base em documentação de fornecedores e blogs citados.
Artigo conceitual de praticantes sobre observabilidade centrada em SLO com plataformas de mercado em microsserviços: conceitos de SLI/SLO e error budget, configuração das ferramentas, correlação de logs, métricas e traces, alertas guiados por SLO e boas práticas. Não descreve método, amostra nem coleta; os 'case studies' são relatos resumidos de blogs e fornecedores citados. As afirmações de que SLOs reduzem a fadiga de alertas e o trabalho repetitivo de apagar incêndio não vêm de dados do próprio estudo, então não há elo prática → fator a extrair.

Lertpongrujikorn et al. (2026) — Empirical Analysis of Cloud-Edge Infrastructure Complexity: Practitioner Pain Points and Architectural Directions
Proceedings - International Conference on Computer Communications and Networks, ICCCN
Motivo: veículo: preprint arXiv sem revisão (arXiv:2608.08400v1); além disso, pergunta 4 fraca: os dados não ligam uma prática de ESC a um fator.
Estudo de customer discovery (NSF I-Corps) com 101 entrevistas semiestruturadas em 86 organizações, codificadas em 11 categorias de dores e 13 de expectativas sobre infraestrutura cloud-edge. Os achados são frequências: complexidade de implantação (38,6%), dificuldade de onboarding (35,6%) e sobrecarga cognitiva pela co-ocorrência entre complexidade e onboarding; os participantes querem mais automação em pipelines de CI/CD e rollback, mas isso é expectativa, não efeito observado. A ligação engenharia de plataforma → redução de carga cognitiva vem de relatórios citados (Puppet 2023, Greiler et al. 2023), e a seção IV propõe direções arquiteturais sem dado próprio. Mesmo que o veículo fosse aceito, não haveria elo prática → fator evidenciado pelos dados.

Tennakoon e Jayatissa (2025) — Achieving Sustainability in Sri Lankan SME IT Firms through DevOps
Proceedings - International Research Conference on Smart Computing and Systems Engineering, SCSE 2025
Motivo: pergunta 4: a variável dependente é um índice agregado de sustentabilidade; efeito sobre pessoas só no abstract e nas hipóteses.
Survey com 136 profissionais de 42 PMEs de TI do Sri Lanka, analisado por PLS-SEM, com CI/CD, IaC, monitoramento e feedback e colaboração e cultura como preditores de um único construto de sustentabilidade ('Sus'), que mistura as dimensões ambiental, econômica e social. Todos os caminhos dão positivos (p entre 0,004 e 0,049), mas nenhum isola a dimensão social nem mede fator ou desfecho das pessoas. A satisfação, a retenção e a cultura de aprendizado contínuo citadas no abstract não aparecem nos resultados. O texto ainda se contradiz sobre H4 (aceita na Tabela V, 'not validated' na discussão).

Sem PDF (6)

Ferazza e Mersinas (2026) — Socio-technical Friction: An Emergent Grounded Theory of DevSecOps Challenges
Springer (LNCS 2026): sem acesso pela CAPES, só a prévia. No abstract: Teoria fundamentada com 18 entrevistas (9 gestores de cibersegurança e 9 engenheiros de software) sobre os obstáculos à adoção de DevSecOps. Conclui que os engenheiros vivem as iniciativas de segurança como disruptivas, ineficientes e prejudiciais à sua experiência de desenvolvedor, e que o desalinhamento sociotécnico (prioridades desalinhadas, comunicação quebrada) gera atrito.

Claassen e Boekhorst (2015) — Shorter feedback loops by means of continuous deployment
Springer (LNBIP 2015): sem acesso pela CAPES, só a prévia. No abstract: Artigo curto que propõe usar implantação contínua para fechar cedo o ciclo de feedback das funcionalidades. Afirma que a prática também diminui o esforço de implantação dos desenvolvedores, aumentando sua felicidade; o abstract não descreve método nem amostra.

Hemon-Hildgen et al. (2020) — Orchestrating automation and sharing in DevOps teams: a revelatory case of job satisfaction factors, risk and work conditions
Taylor & Francis (EJIS): sem acesso pela CAPES. No abstract: Estudo de caso com 59 funcionários de 12 times ágeis e DevOps de uma mesma empresa, à luz das teorias de características do trabalho e de Herzberg. Conclui que DevOps dá mais satisfação no trabalho que o ágil sozinho, mas amplifica riscos e exige orquestrar automação e compartilhamento, a depender das condições de trabalho.

Jothi et al. (2026) — Empowering Developer Productivity through Platform Engineering: A Transformative Approach to Scalable and Streamlined Software Development
Wiley (capítulo de livro 2026): sem acesso pela CAPES. No abstract: Capítulo de livro acadêmico, conceitual, que discute como a engenharia de plataforma (automação, fluxos padronizados, autoatendimento) melhora a produtividade e a experiência do desenvolvedor, citando redução da carga cognitiva. O abstract não traz dados nem resultados empíricos; a afirmação aparece como tema do capítulo.

Kirpitsas e Pachidis (2026) — KIBO, a new hybrid software development method with enhanced information systems auditing capability
Elsevier: acesso aberto, mas o PDF não baixou pelo navegador. No abstract: Propõe o KIBO, método híbrido que integra Scrum, DevOps e Disciplined Agile Delivery com auditoria contínua, avaliado por 90 especialistas e numa implantação de 7 semanas; o time relatou melhora de qualidade, confiabilidade do processo e satisfação líquida dos desenvolvedores. A satisfação é atribuída ao método como um todo, não a uma prática de ESC isolada.

Kulas et al. (2014) — Practical experience with test-driven development during commissioning of the multi-star AO system ARGOS
SPIE: acesso aberto, mas o PDF não baixou pelo navegador. No abstract: Relato de experiência do uso de TDD, programação em par e integração contínua por engenheiros de software durante o comissionamento do sistema ARGOS, fase de pressão de tempo e estresse. O abstract diz que a equipe conseguiu manter o TDD sob pressão, mas não traz resultados sobre o efeito da integração contínua sobre as pessoas.

7. Para o *snowballing*

Primários citados pelos artigos lidos que ligam ESC a pessoas, estudos secundários da busca 1, as sementes que nenhuma string alcança e os trabalhos citados pelo corpus da etapa 1.

Fonte	Indicado por / por quê
Ståhl e Bosch 2013	**Ahmad et al. (2023)**: Ståhl e Bosch 2013 — benefícios experimentados de CI em empresa, incluindo produtividade dos desenvolvedores
Hilton et al. 2016	**Ahmad et al. (2023)**: Hilton et al. 2016 — usos, custos e benefícios de CI em projetos open source, com survey de desenvolvedores
Leppänen et al. 2015	**Ahmad et al. (2023)**: Leppänen et al. 2015 — entrevistas sobre obstáculos e benefícios da implantação contínua (IEEE Software, cinza de 1º nível)
Kasi et al. 2008	**Anandayuvaraj et al. (2026)**: Kasi et al. 2008 — estudo Delphi ("post mortem paradox") sobre percepções de especialistas de TI: tempo, recursos e barreiras culturais no postmortem
Desouza, Dingsøyr e Awazu 2005	**Anandayuvaraj et al. (2026)**: Desouza, Dingsøyr e Awazu 2005 — experiências conduzindo postmortems de projeto (relatórios versus histórias)
Vieira et al. 2019	**Anandayuvaraj et al. (2026)**: Vieira et al. 2019 — dificuldades técnicas e gerenciais na análise postmortem em projetos de software
Sillito e Pope 2024	**Anandayuvaraj et al. (2026)**: Sillito e Pope 2024 (CHASE) — aprender com lições de incidentes; achados preliminares sobre aprendizado com falhas
Wessel et al. 2021	**Asthana et al. (2023)**: Wessel et al. 2021 — "Don't Disturb Me": ruído e desafios de interação com bots em pull requests de projetos open source
Brown e Parnin 2020	**Asthana et al. (2023)**: Brown e Parnin 2020 — arquitetura de escolha das recomendações de bots e incômodo para o desenvolvedor
Erlenhov et al. 2020	**Asthana et al. (2023)**: Erlenhov et al. 2020 — características e desafios de bots de desenvolvimento na perspectiva de praticantes
Hilton et al. 2017	**Asthana et al. (2023)**: Hilton et al. 2017 — trade-offs da integração contínua (garantia, segurança, flexibilidade) relatados por desenvolvedores
Fluri et al. 2023	**Fluri et al. (2024)**: Fluri et al. 2023 (ICSSP) — versão anterior do mesmo estudo, com dois dos casos; conferir sobreposição antes de incluir
Claps et al. 2015	**Fluri et al. (2024)**: Claps et al. 2015 — desafios técnicos e sociais na adoção de implantação contínua
Shahin et al. 2017	**Fluri et al. (2024)**: Shahin et al. 2017 (ESEM) — investigação empírica dos desafios de implantação contínua, inclusive de equipe e coordenação
Riungu-Kalliosaari et al. 2016	**Fluri et al. (2024)**: Riungu-Kalliosaari et al. 2016 — estudo de caso de benefícios e desafios da adoção de DevOps
Lwakatare et al. 2019	**Fluri et al. (2024)**: Lwakatare et al. 2019 — estudo de caso múltiplo de DevOps em cinco empresas
Shropshire e Sweeney 2017	**Hemon et al. (2018)**: Shropshire e Sweeney 2017 — DevOps e moral da força de trabalho; acharam satisfação em queda após a implantação de DevOps
Shropshire, Menard e Sweeney 2017	**Hemon et al. (2018)**: Shropshire, Menard e Sweeney 2017 — personalidade e atitudes diante de DevOps; aspectos negativos relatados: trabalho mais duro, mais responsabilidade, autoridade difusa, métricas complexas, medo de demissão
Nybom, Smeds e Porres 2016	**Hemon et al. (2018)**: Nybom, Smeds e Porres 2016 — impacto de misturar responsabilidades de Dev e Ops: colaboração, mas também nova fonte de atrito
Wiedemann 2017	**Hemon et al. (2018)**: Wiedemann 2017 — nova forma de colaboração em times de TI na adoção de DevOps (capital social)
Leppänen et al. 2015	**Itkonen et al. (2016)**: Leppänen et al. 2015 — The Highways and Country Roads to Continuous Deployment (IEEE Software); estudo com empresas sobre adoção de implantação contínua, citado por identificar a colaboração estreita entre desenvolvimento, operação e teste

Fonte	Indicado por / por quê
Neely e Stolt 2013	**Itkonen et al. (2016)**: Neely e Stolt 2013 — Continuous Delivery? Easy! Just Change Everything (Agile Conference); relato de experiência de adoção de CD, citado pelo mesmo motivo (cinza de 1º nível, entra com aviso se entrar)
Wiedemann et al. 2019	**Jeyam et al. (2026)**: Wiedemann et al. 2019 — citado como fonte de que plantão e disponibilidade constante geram tecno-invasão e de que a evolução de ferramentas gera tecno-incerteza em times DevOps (ICIS)
Sen 2021	**Jeyam et al. (2026)**: Sen 2021 — citado como fonte de que o ritmo dos pipelines de CI/CD e a automação levam a tecno-sobrecarga quando a carga cresce sem apoio (capítulo de livro)
Rau 2016	**Kafoe et al. (2026)**: Rau 2016 (capítulo 'Eliminating Toil' em Beyer et al., Site Reliability Engineering, O'Reilly) — afirma que reduzir toil aumenta satisfação e que acúmulo de toil causa baixa moral e estagnação de carreira; livro, cinza de 1º nível
Lwakatare et al. 2019	**Kafoe et al. (2026)**: Lwakatare et al. 2019 — DevOps in Practice: estudo de caso múltiplo em cinco empresas; curva de aprendizado íngreme de habilidades operacionais em DevOps
Puppet Labs 2015	**Kafoe et al. (2026)**: Puppet Labs 2015 — State of DevOps Report; relatório de survey ligando DevOps a produtividade e qualidade, cinza
Ziftci e Cavalcanti 2020	**Kassab et al. (2026)**: Ziftci e Cavalcanti 2020 — citado para testes flaky que minam a confiança na automação e geram reexecuções e frustração (provavelmente estudo técnico de causa raiz no Google; conferir se há dado sobre pessoas)
Neely e Stolt 2013	**Kärpänoja et al. (2016)**: Neely e Stolt 2013 (AGILE) — estudo de caso da Rally Software sobre a adoção de entrega contínua pelos engenheiros, com resistência à mudança e o estresse dos "scary Saturdays"
Chen 2015	**Kärpänoja et al. (2016)**: Chen 2015 (IEEE Software) — benefícios e desafios da entrega contínua relatados na prática (cinza de 1º nível)
Puppet 2023	**Lertpongrujikorn et al. (2026)**: Puppet 2023 (State of Platform Engineering Report) — relatório de indústria (cinza) usado pelo artigo para ligar engenharia de plataforma à redução da carga cognitiva dos times
Greiler, Storey e Noda 2023	**Lertpongrujikorn et al. (2026)**: Greiler, Storey e Noda 2023 (DevEx, ACM Queue) — fluxo, ciclos de feedback e carga cognitiva como dimensões da experiência do desenvolvedor; candidato a conferir se liga práticas de entrega a esses fatores
Gruber e Fraser 2022	**Parry et al. (2022)**: Gruber e Fraser 2022 — survey sobre como a instabilidade de testes afeta os desenvolvedores e que apoio eles precisam
Eck et al. 2019	**Parry et al. (2022)**: Eck et al. 2019 — perspectiva de desenvolvedores da Mozilla sobre testes instáveis (classificação e survey com 121 respostas)
Hilton et al. 2017	**Parry et al. (2022)**: Hilton et al. 2017 — survey na Pivotal sobre trade-offs da CI, inclui builds falhando por testes instáveis
Hilton et al. 2016	**Pinto et al. (2017)**: Hilton et al. 2016 — uso, custos e benefícios da integração contínua em projetos open source, com survey de desenvolvedores
Claes et al. 2017	**Riedl et al. (2025)**: Claes et al. 2017 — rapid releases e horário anormal de trabalho (Releases frequentes → jornada)
Müller e Benlian 2022	**Riedl et al. (2025)**: Müller e Benlian 2022 — uso de práticas ágeis de desenvolvimento (teste unitário, refatoração, padrões de código) → fadiga, carga percebida e intenção de sair; verificar se separa CI
Benlian 2022	**Riedl et al. (2025)**: Benlian 2022 — uso de práticas ágeis de ISD → bem-estar diário (avaliação como desafio ou ameaça); verificar se inclui CI
Venkatesh et al. 2020	**Riedl et al. (2025)**: Venkatesh et al. 2020 — uso de métodos ágeis (instrumento de Maruping, com CI) → exaustão via ambiguidade e conflito de papel; verificar resultado por prática
Meier et al. 2018	**Riedl et al. (2025)**: Meier et al. 2018 — estresse no desenvolvimento ágil por prática; verificar se analisa CI

Fonte	Indicado por / por quê
Savor et al. 2016	**Rossi et al. (2016)**: Savor et al. 2016 — implantação contínua no Facebook e na OANDA, com desenvolvedores responsáveis por testar e implantar o próprio código; primário dos mesmos autores, pode ter relato sobre as pessoas
Jiang et al. 2020	**Shetty et al. (2022)**: Jiang et al. 2020 — recomendação de TSGs e efeito na redução do esforço de mitigação de incidentes por engenheiros de plantão
Li et al. 2021	**Shetty et al. (2022)**: Li et al. 2021 — dificuldade dos engenheiros de plantão para mitigar incidentes em serviços com dependências complexas
Díaz et al. 2018	**Tennakoon e Jayatissa (2025)**: Díaz et al. 2018 — estudo de caso exploratório de DevOps na prática; pode trazer efeitos de cultura e colaboração sobre os times
Perera et al. 2016	**Tennakoon e Jayatissa (2025)**: Perera et al. 2016 — impacto da prática de DevOps em organizações de software do Sri Lanka; primário, a conferir se trata de pessoas
Rombaut et al. 2023	**Valenzuela-Toledo et al. (2024)**: Rombaut et al. 2023 — overhead de manutenção que o bot de atualização de dependências (Greenkeeper) gera para os desenvolvedores
McIntosh et al. 2011	**Valenzuela-Toledo et al. (2024)**: McIntosh et al. 2011 — esforço de manutenção de arquivos de build, artefato análogo ao pipeline
Saroar e Nayebi 2023	**Valenzuela-Toledo et al. (2024)**: Saroar e Nayebi 2023 — survey com desenvolvedores sobre percepção, motivações e desafios do GitHub Actions
Zhao et al. 2017	**Valenzuela-Toledo et al. (2024)**: Zhao et al. 2017 — impacto da adoção de CI sobre as demais práticas de desenvolvimento
Neely e Stolt 2013	**Zhang et al. (2018)**: Neely e Stolt 2013 — relato de adoção de entrega contínua citado por menos trabalho fora de hora e mais confiança
Claps, Berntsson Svensson e Aurum 2015	**Zhang et al. (2018)**: Claps, Berntsson Svensson e Aurum 2015 — desafios técnicos e sociais na transição para implantação contínua
Chen 2015	**Zhang et al. (2018)**: Chen 2015 — benefícios e desafios da entrega contínua numa organização
Hilton et al. 2017	**Zhang et al. (2018)**: Hilton et al. 2017 — trade-offs da integração contínua (garantia, segurança, flexibilidade) com entrevistas a desenvolvedores
Widder et al. 2018	**Zhang et al. (2018)**: Widder et al. 2018 — por que projetos abandonam o Travis CI
Pérez-Sánchez et al. (2024) A Taxonomy on Human Factors that Affect DevOps Adoption	Mapeamento sistemático (21 estudos) que cria uma taxonomia de 59 fatores humanos que afetam a adoção de DevOps, com foco em cultura. A direção é pessoas → adoção, não prática → pessoas; candidato a snowballing pelos primários que tratam de cultura e fatores humanos em DevOps.
Lunde e Colomo-Palacios (2020) Continuous practices and technical debt: A systematic literature review	Revisão sistemática sobre como práticas contínuas e DevOps afetam a dívida técnica (fator do catálogo), com impactos positivos e negativos; conclui que podem reduzi-la se bem aplicadas. Não trata de pessoas diretamente, mas os primários sobre práticas contínuas e dívida técnica podem servir ao snowballing.
Zimmermann (2017) Microservices tenets: Agile approach to service development and deployment	Artigo de visão geral que revisa introduções populares a microsserviços e os compara com SOA; afirma que microsserviços e os paradigmas associados (inclusive a abordagem DevOps contínua) aumentam a carga cognitiva dos arquitetos. Não é estudo primário e o efeito é atribuído a microsserviços, não a uma prática de ESC.
Jha et al. (2023) From theory to practice: Understanding DevOps culture and mindset	Revisão sistemática sobre cultura e mentalidade DevOps (princípios, práticas, ferramentas, desafios), que diz ajudar a entender o fenômeno pela perspectiva de fatores humanos e identifica três temas culturais: colaboração, melhoria contínua e automação. Candidato a snowballing pelos primários sobre cultura DevOps.
Pérez-Sánchez et al. (2025) A theory on human factors in DevOps adoption	Mapeamento sistemático seguido de clusterização e avaliação por questionário (n=15) que identifica 59 fatores humanos relacionados ao DevOps e propõe uma teoria sobre fatores humanos na adoção de DevOps; a direção estudada é fatores humanos → adoção, não DevOps → pessoas. Candidato a snowballing pelos primários resenhados.
Laukkanen e Mantyla (2015) Build Waiting Time in Continuous Integration-An Initial Interdisciplinary Literature Review	Revisão interdisciplinar da literatura sobre tempo de espera do build em integração contínua; classifica os efeitos da espera em específicos de CI, cognitivos e emocionais e sugere reduzir a percepção da espera com feedback e outras atividades durante a integração. Candidato a snowballing pelos primários sobre efeitos cognitivos e emocionais da espera do build.

Fonte	Indicado por / por quê
Westrum (2004), *A typology of organisational cultures*	Base da ligação práticas → cultura generativa citada por Trinkenreich et al. (2023)
Relatórios DORA / *Accelerate*	Dor do *deploy*, retrabalho e *burnout* associados às capacidades de entrega contínua
Hilton et al. (2017); Fagerholm et al. (2017); Greiler et al. (2022)	Sementes sem prática de ESC no resumo; nenhuma string as recupera

8. Referências (candidatos da busca 1)

Ahmad, Azeem; Neto, Francisco Gomes De Oliveira; Enoiu, Eduard Paul; Sandahl, Kristian; Leifler, Ola (2023). An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration. *IEEE International Conference on Software Quality, Reliability and Security, QRS*. DOI: 10.1109/QRS60937.2023.00041

Anandayuvaraj, Dharun; Singla, Tanmay; Hammadeh, Zain Alabedin Haj; Lund, Andreas; Holloway, Alexandra; Davis, James C. (2026). Learning from Software Failures: A Case Study at a National Space Research Center. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026*. DOI: 10.1145/3744916.3773149

Andreyev, Oleg (2025). The Art of Managing Flaky Tests at Scale. *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2025*. DOI: 10.1109/ICSTW64639.2025.10962486

Asthana, Sumit; Sajnani, Hitesh; Voyloshnikova, Elena; Acharya, Birendra; Herzig, Kim (2023). A Case Study of Developer Bots: Motivations, Perceptions, and Challenges. *ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3611643.3616248

Claassen, Arjan; Boekhorst, Laurens (2015). Shorter feedback loops by means of continuous deployment. *Lecture Notes in Business Information Processing*. DOI: 10.1007/978-3-319-18612-2_24

Claes, Maelick; Mantyla, Mika; Kuutila, Miikka; Adams, Bram (2017). Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content. *IEEE International Working Conference on Mining Software Repositories*. DOI: 10.1109/MSR.2017.3

Ferazza, Francesco; Mersinas, Konstantinos (2026). Socio-technical Friction: An Emergent Grounded Theory of DevSecOps Challenges. *Lecture Notes in Computer Science*. DOI: 10.1007/978-3-032-20018-1_22

Fluri, Jasmin; Fornari, Fabrizio; Pustulka, Ela (2023). Measuring the Benefits of CI/CD Practices for Database Application Development. *Proceedings - 2023 IEEE/ACM International Conference on Software and System Processes, ICSSP 2023*. DOI: 10.1109/ICSSP59042.2023.00015

Fluri, Jasmin; Fornari, Fabrizio; Pustulka, Ela (2024). On the importance of CI/CD practices for database applications. *Journal of Software: Evolution and Process*. DOI: 10.1002/smr.2720

Gummadi, Venkata Pavan Kumar; Chilamkurthi, Lakshmi Sujatha; Kavuri, Srikanth (2026). Service Level Objective (SLO) Observability with Splunk and Dynatrace in Microservices. *2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies, ICAISET 2026*. DOI: 10.1109/ICAISET66439.2026.11541542

Hemon, Aymeric; Monnier-Senicourt, Laetitia; Rowe, Frantz (2018). Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams. *International Conference on Information Systems 2018, ICIS 2018*.

Hemon-Hildgen, Aymeric; Rowe, Frantz; Monnier-Senicourt, Laetitia (2020). Orchestrating automation and sharing in DevOps teams: a revelatory case of job satisfaction factors, risk and work conditions. *European Journal of Information Systems*. DOI: 10.1080/0960085X.2020.1782276

Itkonen, Juha; Udd, Raoul; Lassenius, Casper; Lehtonen, Timo (2016). Perceived Benefits of Adopting Continuous Delivery Practices. *International Symposium on Empirical Software Engineering and Measurement*. DOI: 10.1145/2961111.2962627

Jeyam, Dharneeka; Wiedemann, Anna; Schwabe, Gerhard; Güney, Kadircan (2026). "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2026*. DOI: 10.1145/3786583.3786899

Jothi, C. Saranya; Surya, E.; Rabiya, M. Syed; Lalitha, B.; Roselinkiruba, R. (2026). Empowering Developer Productivity through Platform Engineering: A Transformative Approach to Scalable and Streamlined Software Development. *Platform Engineering: Concepts, Challenges, and Applications*. DOI: 10.1002/9781394395910.ch1

Kafoe, Tom; Ochoa, Lina; Siravuru, Sharath; Serebrenik, Alexander (2026). "making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026*. DOI: 10.1145/3744916.3764540

Kassab, Mohamad; Sohail, Izah; Fernandes, João Paulo (2026). The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes. *FSE Companion 2026 - Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. DOI: 10.1145/3803437.3805542

Kirpitsas, Ioannis; Pachidis, Theodore (2026). KIBO, a new hybrid software development method with enhanced information systems auditing capability. *Information and Software Technology*. DOI: 10.1016/j.infsof.2025.107958

Kulas, M.; Borelli, Jose Luis; Gäsler, Wolfgang; Peter, Diethard; Rabien, Sebastian; Orban De Xivry, Gilles; Busoni, Lorenzo; Bonaglia, Marco; Mazzoni, Tommaso; Rahmer, Gustavo (2014). Practical experience with test-driven development during commissioning of the multi-star AO system ARGOS. *Proceedings of SPIE - The International Society for Optical Engineering*. DOI: 10.1117/12.2056218

Kärpänoja, Pauli; Virtanen, Antti; Lehtonen, Timo; Mikkonen, Tommi (2016). Exploring peopleware in continuous delivery. *ACM International Conference Proceeding Series*. DOI: 10.1145/2962695.2962708

Lertpongrujikorn, Pawissanutt; Nguyen, Hai Duc; Salehi, Mohsen Amini (2026). Empirical Analysis of Cloud-Edge Infrastructure Complexity: Practitioner Pain Points and Architectural Directions. *Proceedings - International Conference on Computer Communications and Networks, ICCCN*. DOI: 10.1109/ICCCN69946.2026.11662594

Limoncelli, Thomas A. (2018). SQL is no excuse to avoid DevOps. *Queue*. DOI: 10.1145/3291276.3300018

Parry, Owain; Kapfhammer, Gregory M.; Hilton, Michael; Mcminn, Phil (2022). Surveying the Developer Experience of Flaky Tests. *Proceedings - International Conference on Software Engineering*. DOI: 10.1109/ICSE-SEIP55303.2022.9793965

Pinto, Gustavo; Reboucas, Marcel; Castor, Fernando (2017). Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users. *Proceedings - 2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2017*. DOI: 10.1109/CHASE.2017.13

Riedl, René; Oettl, Christian; Stangl, Fabian J.; Hevner, Alan R. (2025). How an Agile Software Process Increases Developers' Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model. *Business and Information Systems Engineering*. DOI: 10.1007/s12599-024-00919-x

Rossi, Chuck; Shibley, Elisa; Su, Shi; Beck, Kent; Savor, Tony; Stumm, Michael (2016). Continuous deployment of mobile software at facebook (showcase). *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. DOI: 10.1145/2950290.2994157

Shetty, Manish; Bansal, Chetan; Upadhyayula, Sai Pramod; Radhakrishna, Arjun; Gupta, Anurag (2022). AutoTSG: learning and synthesis for incident troubleshooting. *ESEC/FSE 2022 - Proceedings of the 30th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3540250.3558958

Soltren, Jose; Mulligan, Kyle (2023). Enabling Pre-Merge CI on your TV. *Proceedings - 2023 IEEE 16th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2023*. DOI: 10.1109/ICSTW58534.2023.00028

Souza, Rodrigo; Silva, Bruno (2017). Sentiment Analysis of Travis CI Builds. *IEEE International Working Conference on Mining Software Repositories*. DOI: 10.1109/MSR.2017.27

Tennakoon, Hiruni; Jayatissa, Yehemini (2025). Achieving Sustainability in Sri Lankan SME IT Firms through DevOps. *Proceedings - International Research Conference on Smart Computing and Systems Engineering, SCSE 2025*. DOI: 10.1109/SCSE65633.2025.11031052

Valenzuela-Toledo, Pablo; Bergel, Alexandre; Kehrer, Timo; Nierstrasz, Oscar (2024). The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance. *Proceedings - 2024 IEEE International Conference on Source Code Analysis and Manipulation, SCAM 2024*. DOI: 10.1109/SCAM63643.2024.00029

Zhang, Yang; Vasilescu, Bogdan; Wang, Huaimin; Filkov, Vladimir (2018). One size does not fit all: An empirical study of containerized continuous deployment workflows. *ESEC/FSE 2018 - Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3236024.3236033